



FLASHCARD DESKTOP APP

INDIVIDUELLE ABSCHLUSSARBEIT BLJ

PrimeSoft Group
Bahnhofstrasse 4
CH-8360 Eschlikon

Git-Repository: [Individuelle Abschlussarbeit BLJ breian](#)

Bretscher Jan
jb@letsbuild.ch

Inhaltsverzeichnis

1	Einleitung	3
1.1	Änderungstabelle	3
1.2	Aufgabenstellung	4
1.3	Projektbeschreibung	4
1.4	Risiken & Bedenken	5
2	Planung	6
2.1	Terminplan	6
2.2	Entscheidungsmatrix	7
2.2.1	Technologie	7
2.3	Datenbank	8
3	Hauptteil	9
3.1	Datenbank	9
3.1.1	ERD und Struktur	9
3.1.2	Datenbank aufsetzen	11
3.1.3	Testdaten einfügen	11
3.1.4	CRUD-Operationen & Joins	12
3.2	WPF-Applikation und Serverbackend umsetzen	15
3.2.1	Projekt aufsetzen & Codeumgebung	15
3.2.2	Falscher Plan zur Verbindung mit der Datenbank	16
3.2.3	Lösung, um mit Datenbank zu verbinden	16
3.2.4	Codeerklärung Prototyp mit Authentifizierung	17
3.2.5	Decks und Cards anzeigen V2.1	22
3.2.6	Codeerklärung Decks und Cards anzeigen V2.1 (Final-Version)	22
3.2.7	Codeerklärung Decks hinzufügen V2.2	27
3.2.8	Codeerklärung Cards hinzufügen & bearbeiten V2.3	29
3.2.9	Codeerklärung Learn-Cards V2.4	33
3.2.10	Codeerklärung Import & Export V2.5	36
3.2.11	Login/Registrierung V3.0	39
3.2.12	Karte als Favorit markieren	43
3.2.13	Bearbeiten vom Konto	44
3.2.14	Mitarbeiter für ein Deck einladen	46
3.2.15	Konten Folgen	48
3.3	Testplan/Testfälle mit Ergebnissen	52
3.4	Funktionsbeschreibung	53
3.5	Mögliche Erweiterungen	54
3.6	Fazit	55
3.7	Arbeitsjournal	55
3.7.1	Tag 1 - 04.06.25	55
3.7.2	Tag 2 - 05.06.25	56
3.7.3	Tag 3 - 06.06.25	56
3.7.4	Tag 4 - 11.06.25	56
3.7.5	Tag 5 - 12.06.25	57
3.7.6	Tag 6 - 13.06.25	57
3.7.7	Tag 7 - 18.06.25	58
3.7.8	Tag 8 - 19.06.25	58
3.7.9	Tag 9 - 20.06.25	58
3.7.10	Tag 10 - 25.06.25	59
3.7.11	Tag 11 - 26.06.25	59

3.7.12 Tag 12 - 27.06.25	59
4 Anhang.....	60
4.1 Quellenverzeichnis.....	60
4.1.1 Online-Ressourcen.....	60
4.1.2 KI-Tools.....	60
4.1.3 Bibliotheken und Frameworks	60
4.2 Glossar	61
4.3 Checkliste Individuelles Abschlussprojekt Dokumentation 2025.....	62
4.4 Bilderverzeichnis	63

1 Einleitung

1.1 Änderungstabelle

Datum	Was?	Version
04.06.25	Git-Repository erstellt Git-Project mit Milestones & Issues erstellt Projekt bestätigen lassen Dokumentation erstellt Einleitung der Dokumentation geschrieben	1
05.06.25	Datenbank aufgesetzt Testdaten eingefügt Im Hauptteil den Abschnitt Datenbank abgeschlossen	1.1
06.06.25	C#/WPF/.Net-Projekt aufgesetzt Fehlüberlegung dokumentiert	1.3
11.06.25	Sicherheitsschema gezeichnet Datenbankverbindung hergestellt Code und Sicherheitskonzept dokumentiert	2
12.06.25	Decks anzeigen Cards anzeigen Strukturänderung MainWindow.xaml → Index.xaml Programmierschritte dokumentiert	2.1
13.06.25	Decks hinzufügen Cards hinzufügen Programmierschritte dokumentiert	2.2
18.06.25	Cards hinzufügen Programmierschritte dokumentiert	2.3
19.06.25	Cards hinzufügen dokumentiert Cards Lernen Statistik vom Ergebnis des gelernten Import & Export Weitere Schritte dokumentiert	2.5
20.06.25	Import Files als json & csv Export Files als json & csv Programmierschritte dokumentiert	3.0

	Registrierungs Funktionalität Login Funktionalität Allgemeine Verwendung/Speicherung der wichtigen Variablen	
25.06.25	Login/Registrierung dokumentiert Benutzer kann sich eingeloggt lassen Favoriten Karten hinzufügen	3.1
26.06.25	Favoriten dokumentiert Benutzer Löschen Mitarbeiter hinzufügen	3.2
27.06.25	Mitarbeiter hinzufügen Follower hinzufügen/anzeigen Arbeitsschritte dokumentieren Dokumentation Finalisieren (Anhang schreiben, Text überarbeiten) Präsentation erstellen Code kommentieren/Headerkommentare schreiben	3.3

1.2 Aufgabenstellung

Das Ziel von der individuellen Abschlussarbeit des BLJ's ist es, dass man dem Lehrbetrieb und den Coaches zeigen kann, was man im Basislehrjahr gelernt hat. Es ist empfohlen, dass man ein Thema oder Themenbereich bearbeitet, welcher einem im Lehrbetrieb später auch erwartet. Die PrimeSoft arbeitet viel mit Microsoft zusammen, weshalb ich später viel mit C# machen werde. Es macht deshalb nur Sinn, dass ich eine Windows Applikation schreiben werde, welche ich mit C# umsetzen werde.

1.3 Projektbeschreibung

Mein Projekt ist eine Windows Desktop-Applikation: eine Flashcard-App. Die Grundidee ist ähnlich wie bei Quizlet: Man kann verschiedene Decks erstellen und darin Karten speichern, um diese zu lernen. Zudem kann man eigene Benutzerkonten anlegen, sodass jeder Nutzer nur auf seine eigenen Decks zugreifen kann.

Als Backend dient eine relationale Datenbank auf einem MySQL-Server, den mein Vater hostet. Ich habe Administratorzugriff auf diesen Server, was mir ermöglicht, das Projekt selbstständig umzusetzen.

Wir haben im ZLI bereits an einer mobilen Version einer Flashcard-App gearbeitet. Mein Ziel ist es, diese später mit derselben Datenbank zu verknüpfen, um die Decks sowohl auf dem Laptop als auch auf dem Handy lernen zu können. Dies ist jedoch ein optionales Ziel, da es wahrscheinlich den zeitlichen Rahmen sprengen würde.

Der Fokus meines Projekts liegt grösstenteils auf der Datenbank, der Datenbankverbindung aus der Applikation und den SQL-Abfragen. Ein weiterer Schwerpunkt ist die gesamte WPF-Applikation (Windows Presentation Foundation Application), die ich mit C# und dem .NET-Framework umsetze.

Da ich später bei meinem Betrieb grösstenteils mit C# arbeiten werde, ist dieses Projekt ideal, um praktische Erfahrungen zu sammeln. Ich habe bisher wenig bis keine Erfahrung mit C#, daher kann ich viel Neues lernen. Im ZLI haben wir relationale Datenbanken nicht sehr detailliert behandelt, weshalb ich

umso interessierter bin, mein Wissen in diesem Bereich zu vertiefen. Ich finde Datenbanken einen sehr interessanten und wichtigen Fachbereich der Informatik und schätze die Möglichkeit, mein Wissen darin erweitern zu können.

1.4 Risiken & Bedenken

Auf den MySQL-Server kann ich nur vom gleichen Netzwerk zugreifen, indem auch der Server ist. Ich muss jedoch vom ZLI ausarbeiten, was bedeutet, dass ich entweder eine VPN-Verbindung ins lokale Netzwerk zu Hause herstellen muss oder ich muss alles, was ich im ZLI mache in einem Lokalen Docker-Container umsetzen und danach zu Hause auf den Richtigen Server übertragen muss. Ich würde die erste Variante bevorzugen, weil ich so flexibler und einfacher arbeiten könnte. Ich bin jedoch froh, dass es sicher einen Plan B gibt, auch wenn dieser etwas umständlicher ist.

Was ich auch noch nicht genau weiss, wie ich es umsetzen kann ist die Verbindung zur Datenbank aus der WPF-Applikation heraus. Ich habe beim Frühlingsferienprojekt bereits eine WPF-Applikation erstellen und mit C# arbeiten dürfen. Ich arbeite einfach nach meinem Zeitplan und schaue vorzu, wie ich vorankomme. Ich bin jedoch sehr zuversichtlich, was das Projekt und die Datenbankverbindung angehen.

2 Planung

2.1 Terminplan

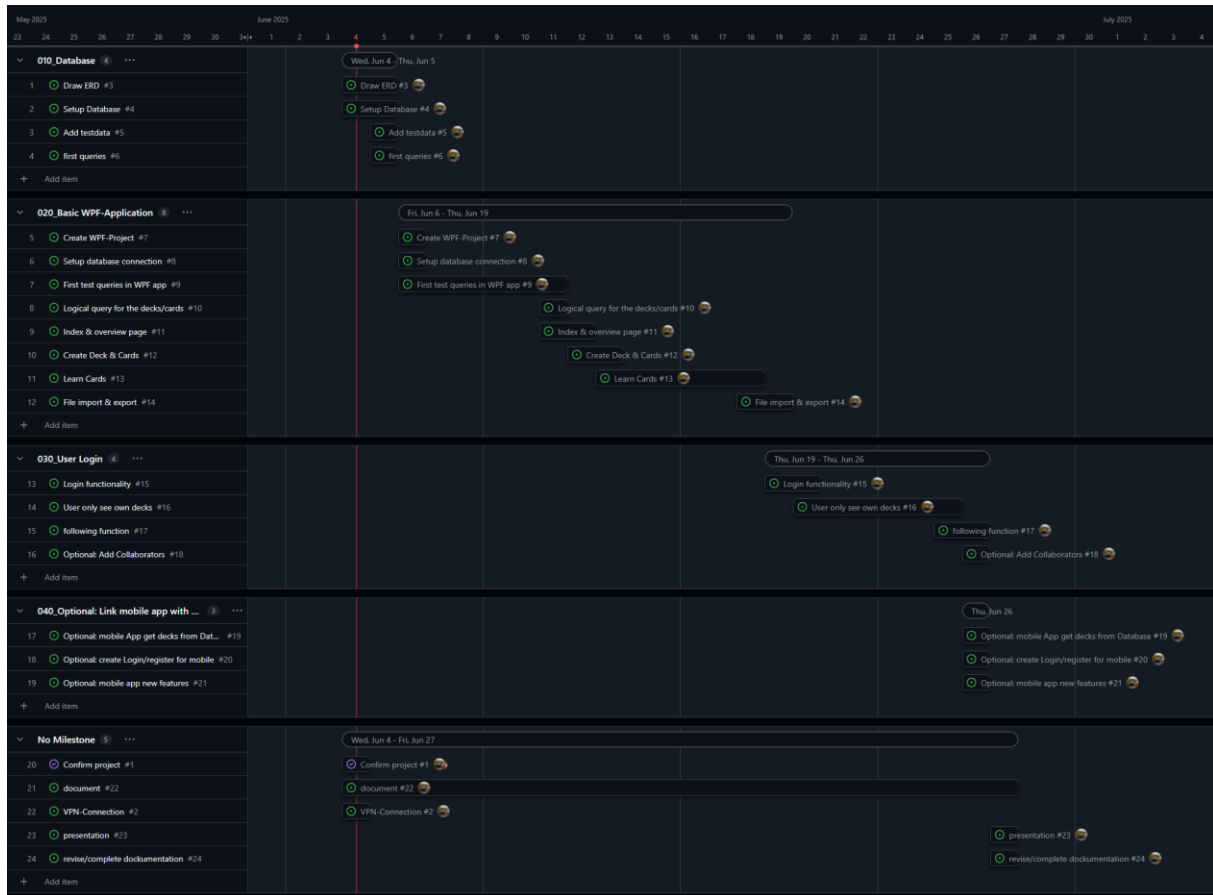


Abbildung 2-1: Roadmap

Dies ist der geplante Zeitplan für das Projekt. Ich habe drei Milestones hinzugefügt, welche ich sicher erreichen möchte/werde. Der Vierte habe ich als optional drin, weil ich denke, dass es zeitlich nicht ganz für diesen reichen wird. Die drei Milestones beinhalten einige Issues, welche ich zu erledigen habe. Ich habe pro Issue zwischen $\frac{1}{4}$ und $\frac{3}{4}$ Tag gerechnet.

Der **erste Milestone** dreht sich um die **Datenbank**. Ich werde zuerst ein ERD-Zeichnen, welches ich danach 1:1 übernehmen kann und die Datenbank damit umsetzen. In der Datenbank werden alle Decks und Karten verwaltet und die Userprofile. Wenn ich die Datenbank umgesetzt habe, werde ich einige Testdaten einfügen und grundlegende SQL-Abfragen durchführen, um zu testen, ob alles wie erwartet funktioniert. Diesen Milestone möchte ich am zweiten Tag bereits erledigt haben.

Der **zweite Milestone** ist die **Grundsätzliche WPF-Applikation**. Dies ist zum einen ein WPF-Projekt zu erstellen, zum einen auch bereits in C# zu coden. Ich habe für diesen Milestone am meisten Issues erstellt, und auch am meisten Zeit eingerechnet. Ich habe mir hierfür 6 Tage Zeit gegeben. Mit 'Grundsätzliche WPF-Applikation' ist gemeint, dass man die App theoretisch bereits verwenden könnte. Die Grundfunktionen sind:

- Alle Kartenstapel anzeigen
- In Kartenstapeln suchen
- Alle Karten/Quiz anzeigen
- In Karten/Quiz suchen

- Kartenstapel erstellen
- Kartenstapel bearbeiten
- Karten/Quiz erstellen
- Karten/Quiz bearbeiten
- Karten lernen
- Lieblingskarten markieren
- Karten als JSON oder CSV exportieren
- Kartenstapel als JSON oder CSV exportieren
- Karten als JSON oder CSV importieren
- Kartenstapel als JSON oder CSV importieren

Der **dritte Milestone** ist die **Login-Funktion**. Hier werde ich die Benutzerverwaltung hinzufügen. Als Benutzer kann man dann sich anmelden oder registrieren, sodass man danach nur noch auf seine eigenen Decks Zugriff hat. Eine weitere Funktion wird Folgen sein. In diesem Milestone werde ich die follow-Funktion hinzufügen, dass verschiedene User einander folgen können und auch zusammen als Gemeinschaftswerk an einem Deck arbeiten können. Diesen Milestone möchte ich in 4 Tagen erledigt haben. Ich denke, dass dies auch eher sportlich und wahrscheinlich knapp machbar sein wird.

Als Optionaler Milestone habe ich mir gesetzt, dass ich bei meiner Mobile Applikation auch ein Login erstelle, sodass man Geräte übergreifend auf seine Decks zugreifen kann. Ich bin mir jedoch ziemlich sicher, dass dies leider zeitlich nicht mehr drin liegt.

2.2 Entscheidungsmatrix

2.2.1 Technologie

KRITERIUM	C#/WPF/.NET	JAVA/JAVAFX	ELECTRON/JS	PYTHON/PYQT
WINDOWS-INTEGRATION	5/5	3/5	2/5	3/5
GUI-KOMFORT & LOOK	4/5	3/5	4/5	3/5
LERNAUFWAND	3/5	3/5	4/5	4/5
PERFORMANCE	5/5	4/5	2/5	3/5
COMMUNITY/SUPPORT	5/5	4/5	4/5	3/5
LOKALE DATENHALTUNG	5/5	4/5	3/5	4/5
BENUTZERVERWALTUNG	5/5	4/5	4/5	3/5
PERSÖNLICHER NUTZEN	5/5	3/5	4/5	4/5
ZUSAMMEN	37/40	28/40	27/40	27/40

- C# mit WPF ist ideal für eine Windows-native App.
- .NET bietet umfassende Bibliotheken für Benutzerverwaltung, Dateispeicherung, UI-Design und Sicherheit.
- WPF (Windows Presentation Foundation) ist etabliert und leistungsfähig für moderne UIs.
- Perfekte Integration in das Windows-Ökosystem (Look & Feel, Registry, Dateisystem).

2.3 Datenbank

KRITERIUM	NOSQL DATENBANK	RELATIONALE DATENBAK
DATENSTRUKTUR	3/5	5/5
SKALIERBARKEIT	4/5	2/5
DATENÜBERTRAGUNGS SICHERHEIT	2/5	5/5
ABFRAGEKOMPLEXITÄT	2/5	5/5
ENTWICKLUNGSKOMFORT	4/5	4/5
ZUSAMMEN	15/25	21/25

3 Hauptteil

3.1 Datenbank

3.1.1 ERD und Struktur

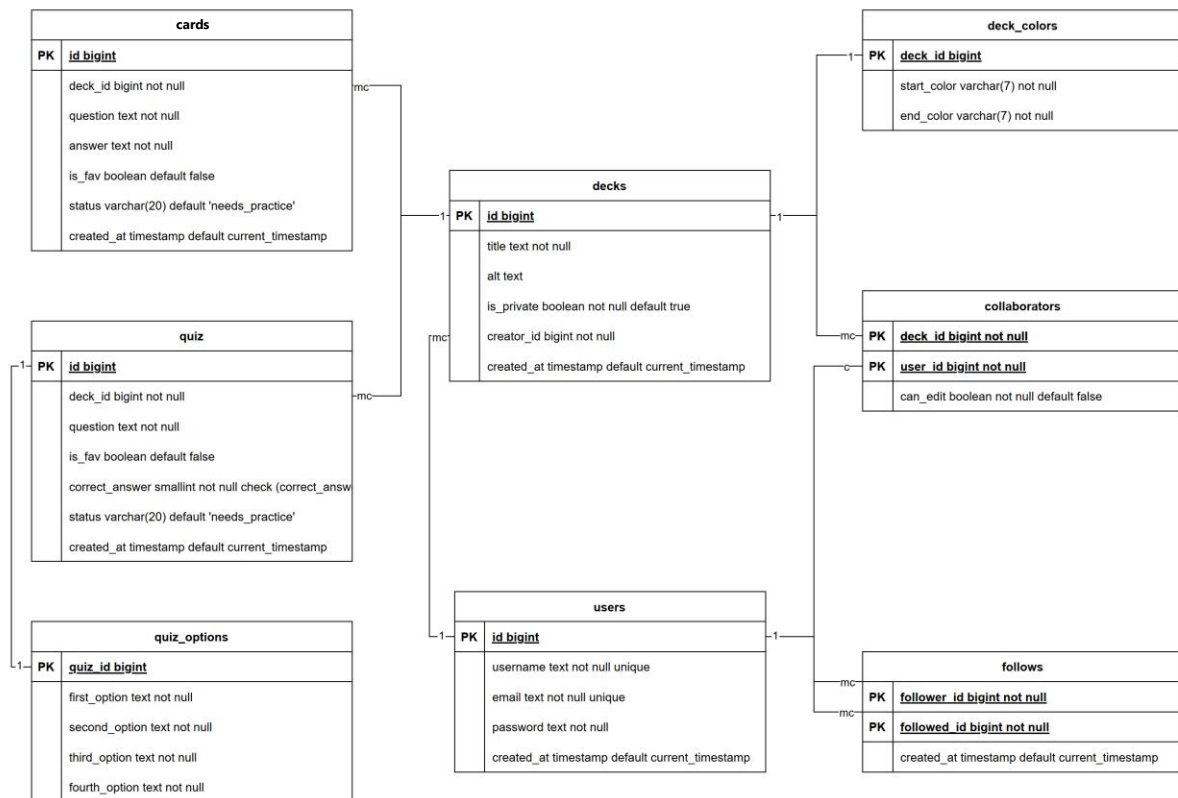


Abbildung 3-1: ERD der FlashCard Datenbank

Dieses ERD beschreibt die Struktur einer Datenbank für eine Lernkarten-Anwendung. Die Kernentitäten sind **users** (Benutzerverwaltung) und **decks** (Lernkartenverwaltung). Ein Benutzer kann Decks erstellen, entweder alleine oder in Zusammenarbeit mit anderen Nutzern (**collaborators**). Jedes Deck enthält Karten, die entweder als einfache Frage-Antwort-Paare (**cards**) oder als Multiple-Choice-Quiz (**quiz** mit **quiz_options**) gestaltet sein können. Zusätzlich können Benutzer einander folgen (**follows**), und Decks haben individuelle Farben (**deck_colors**).

3.1.1.1 Überlegungen zu den Entitäten & Attributen

Ein User muss sich mit einem username, einer email und einem password registrieren. Diese Daten werden im entsprechenden Datensatz gespeichert. Die id und der Zeitstempel (created_at) werden dabei automatisch bei der Registrierung gesetzt.

Ein User kann beliebig vielen anderen Nutzern folgen. Das läuft über die follows-Tabelle, in der jeweils die follower_id und followed_id eingetragen werden.

Die Hauptfunktion für Users ist das Erstellen von Decks. Dabei gibt es zwei Möglichkeiten: Entweder man erstellt ein Deck alleine oder gemeinsam mit einem anderen User. Wenn man jemanden zur Zusammenarbeit einlädt, wird in der collaborators-Tabelle die deck_id des entsprechenden Decks sowie die user_id des eingeladenen Users gespeichert. Ob ein Deck Mitarbeitende hat oder nicht, erkennt man

darán, ob es einen passenden Eintrag in der collaborators-Tabelle gibt. Wenn nicht, ist es einfach ein privates Deck ohne Mitwirkende.

Ein Deck besitzt verschiedene Attribute: title, alt (als Alternativtext), creator_id, ein card_type, ein is_private-Schalter (ob das Deck privat ist) sowie ein created_at-Zeitstempel. Das Attribut is_private habe ich bisher nicht aktiv eingeplant – es ist daher standardmässig auf true gesetzt. Es könnte aber gut für spätere Erweiterungen genutzt werden.

Jedes Deck hat ausserdem zwei Farben (Start- und End-Farbe), die in der Tabelle deck_colors gespeichert sind. Diese Farben werden über die deck_id dem jeweiligen Deck zugeordnet.

Innerhalb eines Decks befinden sich Karten – entweder als klassische Frage-Antwort-Karten (cards) oder als Multiple-Choice-Quizfragen (quiz). Der Unterschied liegt in der Struktur: Bei cards gibt es genau eine Frage und eine Antwort. Bei einem quiz gibt es eine Frage mit vier Antwortmöglichkeiten und einem Index, der die korrekte Antwort angibt. Die vier Optionen habe ich bewusst in eine eigene Tabelle (quiz_options) ausgelagert, um keine Array- oder Listenstruktur im quiz selbst zu verwenden.

3.1.1.2 Die drei Normalformen

1NF (Erste Normalform):

- Alle Attribute enthalten atomare Werte (keine Listen/Mehrfachwerte)
- Keine sich wiederholenden Gruppen

2NF (Zweite Normalform):

- Alle Teile eines Eintrags, die nicht der Schlüssel sind, müssen vom *ganzen* Schlüssel abhängen und nicht nur von einem Teil davon.

3NF (Dritte Normalform):

- Keine Information darf von einer anderen Nicht-Schlüssel-Information abhängen; alles hängt direkt vom Hauptschlüssel ab.

3.1.1.3 Meine Umsetzung der Normalformen

1NF:

Es gibt keine wiederholten Gruppen von Attributen innerhalb einer Tabelle. Ich habe z.B. die Tabelle quiz_options erstellt, dass ich kein Array aus den Optionen in quiz habe. Dasselbe gilt bei den Farben (deck_colors).

2NF:

Die 2. Normalform ist nur relevant, wenn eine Tabelle einen zusammengesetzten Primärschlüssel hat. Z.B. hat die Tabelle follows einen zusammengesetzten Primärschlüssel aus follower_id und followed_id. Das Attribut created_at hängt hier von beiden Attributen ab.

Die meisten anderen Tabellen haben einen normalen int als PK und betreffen deshalb diese Normalform nicht.

3NF:

Die transitiven Abhängigkeiten werden vermieden durch z.B. eine separate Tabelle deck_colors statt der Farben direkt in decks zu schreiben.

Jedes Nicht-Schlüsselattribut bezieht sich direkt auf den Primärschlüssel!

3.1.2 Datenbank aufsetzen

Nachdem ich das ERD gezeichnet habe, schrieb ich einen passenden SQL-Code, welcher die Datenbank 1:1 so umsetzte. Dieser Teil war eine reine Fliessarbeit, weil ich mir die gesamte Struktur, die Beziehungen und Flags beim ERD bereits überlegt habe.

```
create table decks (
  id bigint primary key,
  title varchar(55) not null,
  alt varchar(55),
  card_type smallint not null,
  is_private boolean not null default true,
  creator_id bigint not null,
  created_at timestamp default current_timestamp,
  foreign key (creator_id) references users (id) on delete cascade
);
```

Hier ist ein kurzes Snippet aus dem Code. Die Tabelle decks wird erstellt mit allen Attributen. In der zweitletzten Zeile wird ein Fremdschlüssel gesetzt, welcher auf den User verweist, der das Deck erstellt hat.

In der CMD konnte ich mit `mysql -u jan -h hercules.net.letsbuild.ch -p --ssl-verify-server-cert=FALSE` mit dem MySQL-Server verbinden. Dies funktioniert jedoch nur, wenn ich im lokalen Netzwerk bin (gleiches wie der Server). Hierfür musste ich meinen Laptop über eine VPN-Verbindung im Netzwerk zu Hause verbinden. Dies funktionierte einwandfrei. Zuerst musste ich mit

```
CREATE DATABASE FlashCards;
```

eine neue Datenbank erstellen und danach mit

```
USE FlashCards;
```

diese DB auch verwenden. Danach konnte ich mein gesamtes SQL-Script einfügen.

3.1.3 Testdaten einfügen

Ich liess mit AI-Testdaten in Form von SQL-Coder erstellen und konnte diese auch einfach einfügen.

```
INSERT INTO cards (id, deck_id, question, answer, is_fav, status) VALUES
(1001, 101, 'Hello', 'Hallo', FALSE, 'learned'),
(1005, 102, 'Fläche eines Kreises?', 'pi * r^2', TRUE, 'learned'),
(1009, 105, 'Être', 'Sein', FALSE, 'learned'),
(1010, 106, 'H2O', 'Wasser', FALSE, 'learned'),
(1011, 108, 'Was ist ein "loop"?', 'eine Wiederholung', TRUE, 'learned'),
(1012, 101, 'Water', 'Wasser', FALSE, 'needs_practice');
```

Hier ist ein einfaches INSERT INTO Beispiel für die Tabelle 'cards'. Ich hatte hier das Problem, dass Gemini zuerst den einen Datensatz so einfügen wollte:

```
(1011, 108, 'Was ist ein 'loop'?', 'eine Wiederholung', TRUE, 'learned'),
```

Das Problem sind die Singlequotes ('). Umgekehrt ist das Problem, wenn ich aussen Doublequotes (") habe, dann darf ich innen nur Singelquotes haben. Mein Fazit also:

Es müssen innen und aussen unterschiedliche Anführungszeichen sein.

```
-- Funktioniert nicht!
(1011, 108, 'Was ist ein 'loop'?', 'eine Wiederholung', TRUE, 'learned')
(1011, 108, "Was ist ein "loop"?", "eine Wiederholung", TRUE, "learned")
-- Funktioniert
(1011, 108, 'Was ist ein "loop"?', 'eine Wiederholung', TRUE, 'learned')
(1011, 108, "Was ist ein 'loop'? ", "eine Wiederholung", TRUE, "learned")
```

Ich denke, dass die einzige Lösung ist, dass ich Strings in Doublequotes schreibe und man dann nur einfache Anführungszeichen verwenden darf. Ansonsten lief alles reibungslos ab und die Datenbank ist nun mit Testdaten befüllt.

3.1.4 CRUD-Operationen & Joins

Ich habe einige testabfragen gemacht, um die Funktionalität der Datenbank zu testen. Die Abfragen, die ich gemacht habe, kann ich wahrscheinlich später in meiner Applikation gut verwenden.

Wichtig: die 'id' ist in den meisten Fällen Auto Increment.

```
-- Benutzer erstellen
INSERT INTO users (username, email, password)
VALUES ('max_mustermann', 'max@example.com', 'secure123');

-- Benutzer Login
SELECT id, username FROM users
WHERE email = 'max@example.com' AND password = 'secure123';

-- Benutzer aktualisieren
UPDATE users
SET username = 'new_username', email = 'new@example.com'
WHERE id = 1;

-- Benutzer löschen
DELETE FROM users WHERE id = 1;
```

Hier sind User Interaktionen gemacht. Mit INSERT INTO kann man einen neuen User erstellen. Hier werden diese Daten in die Datenbank hinzugefügt.

Fürs Login kann man grundsätzlich beim User den benutzernamen und das Passwort abfragen und mitgeben. Somit kann man nicht nur wie hier die id und den username zurückgeben, sondern auch die Decks.

Mit UPDATE kann ein User-Datensatz aktualisiert werden. Hier wird z.B. der bestehende username durch new_username und die email durch new@example.com ersetzt beim Datensatz mit der id 1.

DELETE löscht den benutzer ganz normal aus der Datenbank.

```
-- Deck erstellen
INSERT INTO decks (title, alt, is_private, creator_id)
VALUES ('Mathe Basics', 'Deck für Grundrechenarten', false, 1);

-- Farben hinzufügen
INSERT INTO deck_colors (deck_id, start_color, end_color)
VALUES (
  (SELECT id FROM decks ORDER BY id DESC LIMIT 1),
  '#FF0000',
  '#00FF00'
);
```

Das Deck erstellen läuft gleich ab. Beim Hinzufügen der Farben ist es jedoch etwas komplizierter. In der Datenbank ist das Attribut 'deck_id' als Primary Key eingetragen. Dies bedeutet, dass dieser Wert Unique sein muss. Dies ist auch gut so, weil ein Deck nur zu genau einer Farbkombination gehören kann. Hier ist es also vorteilhaft, mit einer Subquery zu arbeiten. Die Subquery selektiert die 'id' des letzten Datensatzes aus 'decks'. Wenn ich also ein neues deck erstelle, bekommt es eine id, welche vom Auto Increment immer die höchste sein muss. Wenn ich dann gleich eine Farbe hinzufügen möchte, dann wird automatisch die letzte Zahl ausgesucht.

```
-- alle Decks eines Benutzers anzeigen
SELECT d.id, d.title, d.alt, dc.start_color, c.end_color
FROM decks d
LEFT JOIN deck_colors c ON d.id = dc.deck_id
WHERE d.creator_id = 1 OR d.id IN (
    SELECT deck_id FROM collaborators WHERE user_id = 1
);

-- Deck aktualisieren
UPDATE decks
SET title = 'Updated Title', is_private = true
WHERE id = 1;

-- Deck löschen
DELETE FROM decks WHERE id = 1;
```

Bei der ersten Abfrage werden alle Decks eines Benutzers mit id, titel, alternativ Text und den beiden Farben dargestellt. Es wird nach der id in deck.creator_id und in collaborators.user_id gesucht. Somit werden auch diese Decks angezeigt, die man nicht selbst erstellt hat, sondern bei denen man 'nur' Mitarbeiter ist.

Das Deck aktualisieren und löschen funktioniert grundsätzlich gleich wie bei den Usern.

```
-- Karte hinzufügen
INSERT INTO cards (deck_id, question, answer, is_fav)
VALUES ((SELECT id FROM decks ORDER BY id DESC LIMIT 1), 'Was ist 2+2?',
    '4', true);

-- Quiz hinzufügen
-- Quiz-Frage erstellen
INSERT INTO quiz (deck_id, question, correct_answer)
VALUES ((SELECT id FROM decks ORDER BY id DESC LIMIT 1), 'Was ist die
Hauptstadt von Deutschland?', 1);
-- Optionen hinzufügen
INSERT INTO quiz_options (quiz_id, first_option, second_option,
third_option, fourth_option)
VALUES ((SELECT id FROM quiz ORDER BY id DESC LIMIT 1), 'Berlin',
'München', 'Hamburg', 'Köln');
```

Die Karte fügt man gleich wie wie ein User hinzu.

Das Quiz ist interessanter, aber auch sehr ähnlich, wie decks und deren Farben. Hier ist 'quiz_id' auch der Primary Key. Es darf also auch wieder nur ein quiz_options zu einem quiz gehören. Dies ist eigentlich dasselbe, wie den decks.

```
-- alle Karten eines Decks anzeigen
-- Einfache Karten
SELECT id, question, answer, is_fav FROM cards WHERE deck_id = 1;

-- Quizze
SELECT q.id, q.question, qo.first_option, qo.second_option,
qo.third_option, qo.fourth_option, q.correct_answer
FROM quiz q
JOIN quiz_options qo ON q.id = qo.quiz_id
WHERE q.deck_id = 1;
```

Hier werden einzeln von Type cards und quiz die zum deck 1 gehören dargestellt, mit Frage und Antwort/en. Die Abfrage so ist noch ziemlich simpel. Das Ziel später ist dann jedoch dies zu kombinieren und alle Karten, egal ob card oder quiz, in einer Tabelle darzustellen.

```
-- Favoriten markieren
UPDATE cards SET is_fav = true WHERE id = 1;

-- karte/quiz löschen
DELETE FROM cards WHERE id = 1;
DELETE FROM quiz WHERE id = 1;
```

Dies ist wieder weniger interessant. Hier werden cards/quiz geändert und gelöscht.

Wenn man einen Datensatz aus quiz löschen möchte, muss man gut aufpassen. Quiz hängt über einen FK mit quiz_options zusammen. Wenn man nun zuerst den Datensatz in quiz löscht, kommt ein Fehler. Theoretisch müsste man also immer zuerst den betreffenden Datensatz in quiz_options löschen, sodass der Foreign Key gelöscht wird, und erst danach in quiz löschen.

Bei dieser Datenbank ist dies jedoch kein Problem, denn ich habe beim Installieren der Datenbank mit folgendem erweitert:

```
FOREIGN KEY (quiz_id) REFERENCES quiz (id) ON DELETE CASCADE
```

Das 'ON DELETE CASCADE' macht, dass wenn eine Komponente der Verbindung (hier: entweder 'quiz_id' oder 'id') gelöscht wird, wird das Gegenstück auch gelöscht.

```
-- mitarbeiter hinzufügen
INSERT INTO collaborators (deck_id, user_id, can_edit)
VALUES (1, 2, true);

-- Folgen
INSERT INTO follows (follower_id, followed_id)
VALUES (1, 2);
```

Beim Mitarbeiter hinzufügen und Folgen muss man etwas beachten. Hier setzt sich der Primary Key aus den Values von deck_id und user_id oder follower_id und followed_id. Das heisst es darf kein Datensatz bereits vorhanden sein, bei dem deck_id = 1 und user_id = 2 ist. Dasselbe gilt für die Followers.

```
-- alle mitarbeiter eines Decks anzeigen
SELECT
    collaborator.username AS username_collaborator,
    d.title AS deck_title,
    creator.username AS username_creator
FROM collaborators c
JOIN users collaborator ON c.user_id = collaborator.id
JOIN decks d ON c.deck_id = d.id
JOIN users creator ON d.creator_id = creator.id
WHERE c.deck_id = 1;
```

Hier wird der Benutzername der Collaborators, der Decktitel und der Benutzername des erstellers vom deck mit der id = 1 angezeigt.

Mit JOIN users collaborator und JOIN users creator kann man Fehler vermeiden, da die Namen gut gewählt sind und klar, welcher user es ist. Unübersichtlicher wäre es gewesen, wenn man es zweimal mit JOIN users ... geschrieben hätte.

```
-- Anzahl Follower pro Benutzer
SELECT u.username, COUNT(f.follower_id) AS follower_count
FROM users u
LEFT JOIN follows f ON u.id = f.followed_id;
```

Diese SQL-Abfrage gruppiert die Datensätze der follows-Tabelle nach dem Benutzer, dem gefolgt wird (followed_id). Für jeden dieser Benutzer wird dann die Anzahl der Zeilen (d.h. die Anzahl der Follower) ermittelt und als Ergebnis zurückgegeben. Das Ergebnis ist eine Liste von Benutzern und der jeweiligen Anzahl ihrer Follower

3.2 WPF-Applikation und Serverbackend umsetzen

3.2.1 Projekt aufsetzen & Codeumgebung

Ich werde dieses Projekt mit Visual Studio 2022 umsetzen. Folgend habe ich ein paar wichtige Unterschiede gegenüber VS-Code aufgelistet.

Kriterium	VS-Code	Visual Studio 2022
Plattform	Windows, macOS, Linux	Nur Windows
Entwicklungsumgebung	Editor	Komplette IDE
Sprachen	Viele, per Erweiterung	Fokus auf C#, C++, .NET
Zielgruppe	Webdevs, DevOps	.NET-/Windows-Stack
Projekttypen	Web, Skripte, leichtgewichtig	Desktop, Mobile, Enterprise

Da ich eine Windowsapplikation mit .Net, WPF und C# umsetzen möchte, eignet sich Visual Studio 2022 am besten geeignet.

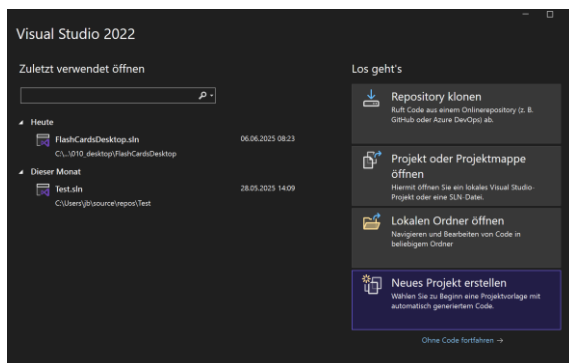


Abbildung 3-3: Neues Projekt wird erstellt

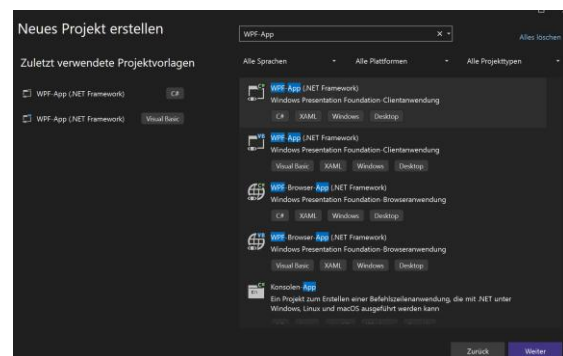


Abbildung 3-2: Die passende Projekt Vorlage auswählen

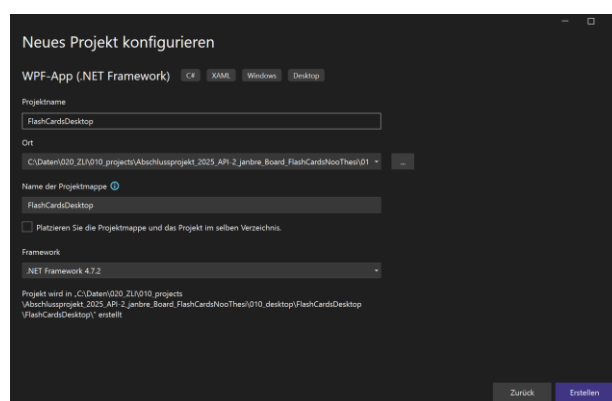


Abbildung 3-4: Titel und Speicherort für Projektmappe

3.2.2 Falscher Plan zur Verbindung mit der Datenbank

Ich wollte direkt von der C# App her auf die Datenbank zugreifen. Es gibt eine Client-Datenbank-Bibliothek, um eine Verbindung zum MySQL-Server aufzubauen. Ich habe es mit dieser Bibliothek versucht, es konnte jedoch keine Verbindung aufbauen. Mir war nicht direkt klar, woran dies liegt. Ich setzte deshalb in meinem Docker einen Mysql Container auf, um die Verbindung mit einem Lokalen Server zu testen. Auch dies hatte nicht funktioniert. Hier lag es wahrscheinlich daran, dass der Server nicht über Localhost:3306 lief, sondern über eine interne IP in Docker. Ich verfolgte dies jedoch nicht weiter, weil ich später bei der Applikation die Datenbankverbindung sowieso nicht so machen kann, weil der Client sonst immer im Server-Netzwerk oder mit einer VPN-Verbindung verbunden sein muss, was unrealistisch ist. Ich habe sehr viel Zeit in dieses Problem investiert. Nun habe ich jedoch eine Lösung, wie ich dieses Problem umgehen kann.

3.2.3 Lösung, um mit Datenbank zu verbinden

Die Lösung ist über ein Serverseitiges PHP-Script. Der Client kann über http-Requests verschiedene Funktionen im PHP aufrufen. Ich kann hierfür die Library 'System.Net.Http' verwenden. Die System-Library ist eine Standardbibliothek für .Net. Diese Bibliothek ermöglicht mir sehr einfach eine http-Abfrage. Mit dieser Lösung muss ich mich nun jedoch um die Sicherheit kümmern, da nicht jeder, der auf das PHP-Script zugreifen kann auch die Datenbank Abfragen darf.

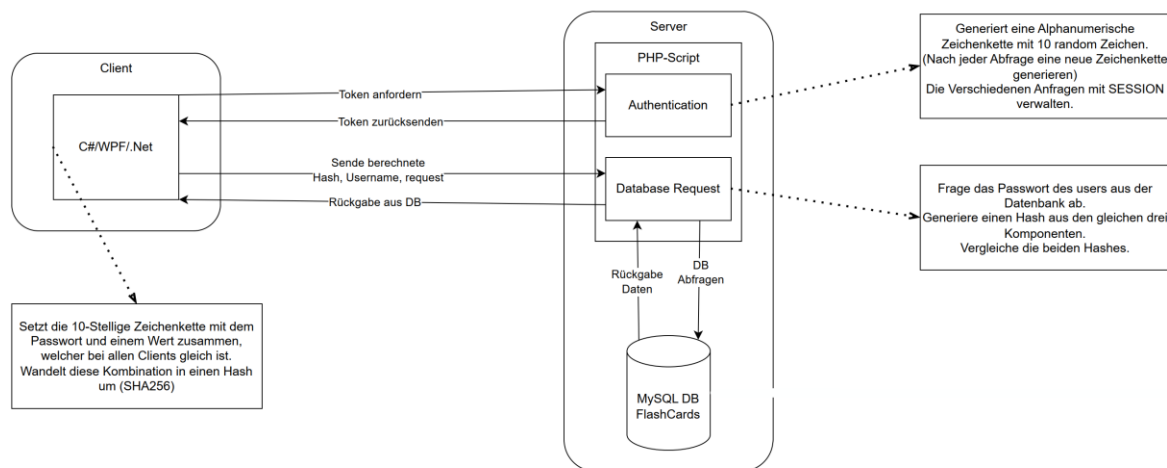


Abbildung 3-5: Datenbankabfrage Netzwerkschema

Bei diesem Schema habe ich kurz festgehalten, wie die Netzwerkkommunikation funktionieren sollte. Der Client fordert als erstes einen Token vom Server an. Der Server generiert eine Random Zeichenkette mit 10 Zeichen (gross-/klein-Buchstaben und Zahlen). Dieser Token wird dem Client zurückgesendet. Der Client generiert aus dem erhaltenen Token, dem Passwort des Benutzers und einem fixen 10-stelligen Code ('Salt'), welcher bei allen Clients gleich ist, einen Hash (SHA256). Dieser Hash wird zusammen mit dem Benutzernamen und einer Aktion (z.B. getDecks) wieder an den Server gesendet. Nun weiss das PHP-Script den Benutzernamen und kann somit das Passwort dieses Benutzers aus der Datenbank abfragen. Der Server erstellt nun einen gezeichneten Hash aus diesen drei Komponenten. Wenn der vom Client geschickte Hash mit dem Server-generierten übereinstimmt, wird die Aktion weiterverfolgt, die Datenbank abgefragt und in JSON-Format zurück an den Client gesendet. Wenn die Hashes nicht übereinstimmen, kommt direkt ein Error zurück.

3.2.4 Codeerklärung Prototyp mit Authentifizierung

3.2.4.1 MainWindow.xaml.cs V1

Ich habe vier Files für die Hauptfunktion für die Datenbank Abfrage. Eine WPF-Applikation ist immer so aufgebaut, dass es ein MainWindow.xaml gibt, welches für das UI zuständig ist. XAML ist ähnlich wie HTML eine Markup Language für das Darstellen vom UI (Extensible Application Markup Language). Für die Logik gibt es dann ein MainWindow.xaml.cs. Hier kann man die ganze Logik verwalten.

UI des Prototyps

```

21 ///////////////////////////////////////////////////////////////////
22 // token + baseCode + password -> SHA256 Hash (Authentication)
23 ///////////////////////////////////////////////////////////////////
24
25 namespace FlashCards
26 {
27     /// <summary>
28     /// Interaktionslogik für MainWindow.xaml
29     /// </summary>
30     2 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
31     public partial class MainWindow : Window
32     {
33         private string token;
34         private string sessionId;
35         private string hashedToken;
36         private string baseCode = "4gdrsh92z7";
37
38         0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
39         public MainWindow()
40         {
41             InitializeComponent();
42         }
43
44         1 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
45         private async void connect_button_clicked(object sender, RoutedEventArgs e)
46         {
47             try
48             {
49                 using (HttpClient tokenClient = new HttpClient())
50                 {
51                     var responseToken = await getToken.GetTokenAsync(tokenClient);
52
53                     token = responseToken.token;
54                     sessionId = responseToken.sessionID;
55
56                     Console.WriteLine($"Token: {token} \n SessionId: {sessionId}");
57                 }
58
59                 hashedToken = generateHash.GenerateSHA256Hash(token, baseCode, password.Password);
60
61                 Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");
62
63                 using (HttpClient requestClient = new HttpClient())
64                 {
65                     var responseData = await sendRequest.SendRequest(requestClient, request.Text, user.Text, hashedToken, sessionId);
66                     string data = responseData;
67
68                     Console.WriteLine($"Response Data: {data}");
69                 }
70             }
71             catch (Exception ex)
72             {
73                 Console.WriteLine($"Error: {ex.Message}");
74             }
75         }
76     }
77 }
78
79

```

Abbildung 3-6: MainWindow.xaml.cs

Zuoberst in diesem Code sind die Instanzvariablen. Diese werden dort definiert, damit überall in dieser Klasse darauf zugegriffen werden kann. Der 'baseCode' ist der Salt Wert, welcher vor dem Hashen angehängt wird. Der Konstruktor ist auf Zeile 38. Hier wird das UI aus MainWindow.xaml geladen. Danach kommt ein Event-Listener. Wenn man auf den 'Send-Request'-Button drückt, wird diese Methode aufgerufen. Auf der Zeile 47 wird mit der 'System.Net.Http' Bibliothek ein http-Client erstellt und dieser dann der GetTokenAsync-Methode mitgegeben. Diese wird weiter unten genauer beschrieben (getToken.cs). Von dieser Methode kommt ein Array zurück, welches etwa so aussehen könnte:

```
{
  "token": "DLpVFfdBBR",
  "sessionID": "qpp0t8srnbaueonl1iae9k8v06"
}
```

Der Token wird einzeln ausgelesen und dann fürs Debugging geprintet. Danach wird die GenerateHash-Methode ausgeführt, mit allen drei Komponenten als Parameter. Mehr dazu auch weiter unten (generateHash.cs). Auch hier wird alles in die Konsole geprintet zu Debugging Zwecken. Wenn man nun den Hash hat, wird die SendRequest-Methode aufgerufen, mit den Parametern: http-Client, den request (z.B. getCards), den userName, den gehasheden Token und die SessionID. Mehr hierzu auch weiter unten (sendRequest.cs).

All dies ist in einem Try. Wenn während einem Vorgang ein Fehler kommt, wird direkt abgebrochen und es geht in den catch, wo der Fehler noch genau in die Konsole geprintet wird.

3.2.4.2 getToken.cs

```
10 namespace FlashCards
11 {
12     2 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
13     internal class getToken
14     {
15         private static string baseUrl = "https://jan-bretscher.ch/01_zli/FlashCards/databaseRequest.php";
16
17         0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
18         public getToken()
19         {
20             // Constructor logic if needed
21         }
22
23         1 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
24         internal static async Task<dynamic> GetTokenAsync(HttpClient client)
25         {
26             var response = await client.GetStringAsync(
27                 $"{baseUrl}?action=getToken");
28             return JsonConvert.DeserializeObject(response);
29         }
30     }
31 }
```

Abbildung 3-7: getToken.cs

Die GetTokenAsync-Methode sendet eine http-Anfrage an die baseUrl. Hier liegt das PHP-Script, welches die Anfragen bearbeitet. Es wird noch der Parameter action=getToken angehängt. Hierzu später noch mehr. Es wird ein String erwartet, weshalb das Script einen String bekommt und diesen dann in JSON-Format umwandelt und zurückgibt.

3.2.4.3 generateHash.cs

```

9      namespace FlashCards
10     {
11         1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
12         internal class generateHash
13         {
14             1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
15             public static string GenerateSHA256Hash(string input, string baseCode, string password)
16             {
17                 using (var sha256 = SHA256.Create())
18                 {
19                     string fullInput = input + baseCode + password;
20
21                     byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(fullInput));
22                     StringBuilder builder = new StringBuilder();
23                     foreach (var b in bytes)
24                     {
25                         builder.Append(b.ToString("x2"));
26                     }
27                     return builder.ToString();
28                 }
29             }
30         }

```

Abbildung 3-8: generateHash.cs

In der GenerateSHA256Hash-methode wird die System.Security.Cryptography-Bibliothek verwendet. Als erstes wird ein neues Objekt, sha256 instanziiert. Danach alle Komponente zusammengenommen, welche gehashed werden. Danach wird die Zeichenfolge 'fullInput' mit dem SHA256-Algorithmus zu einer Bitfolge umgewandelt und danach zu einem String zusammengesetzt. Dieser String wird dann in einen Hexadezimalen, für uns 'lesbaren' String umgerechnet.

3.2.4.4 sendRequest.cs

```

10     namespace FlashCards
11     {
12         1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
13         internal class sendRequest
14         {
15             private static string baseUrl = "https://jan-bretscher.ch/01_zli/FlashCards/databaseRequest.php";
16
17             1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
18             internal static async Task<dynamic> SendRequest(HttpClient client, string request, string user, string token, string sessionID)
19             {
20                 var response = await client.GetStringAsync(
21                     $"{baseUrl}?action=getData&request={request}&user={user}&token={token}&sessionID={sessionID}");
22                 return JsonConvert.DeserializeObject(response).ToString();
23             }
24         }

```

Abbildung 3-9: sendRequest.cs

Diese Klasse funktioniert sehr ähnlich, wie getToken. Der einzige Unterschied sind die vielen Parameter, welche hier mitgeschickt werden. Es werden die Anfrage, der Username, der gehasheden Wert und die SessionID mitgegeben. Die SessionID ist wichtig, dass der Server denselben Token verwendet, wie er diesem Client zur Verfügung gestellt hat für die Berechnung des Hashs verwendet. Mehr hierzu weiter unten (databaseRequest.php).

3.2.4.5 databaseRequest.php

```
$mysql_host = 'herkules.net.letsbuild.ch:3306';
$mysql_user = 'jan';
$mysql_password = 'VEezZ85d';
$mysql_database = 'FlashCards';

$fh = fopen('./log.txt', 'a');

$action = $_GET['action'] ?? '';
$requestMessage = $_GET['request'] ?? null;
$user = $_GET['user'] ?? null;
$clientToken = $_GET['token'] ?? null;
$sessionID = $_GET['sessionID'] ?? null;

$baseCode = '4gdrsh92z7';
```

Hier werden die allgemeinen Variablen definiert. Dies sind alle Credentials für die Datenbankverbindung, das Error Handling log File, die verschiedenen Parameter, welche mit dem http-Request mitgegeben werden und dem baseCode (=Salt).

Abbildung 3-10: CodeSnippet databaseRequest.php allgemeine Variablen

```
// Session nur starten wenn SessionID übergeben wurde
if ($sessionID) {
    session_id($sessionID);
}
session_start();
```

Abbildung 3-11: CodeSnippet databaseRequest.php Sessionverwaltung

Der Client ruft das PHP-Script zweimal auf. Das erste mal um einen Token zu bekommen, das zweite mal, um den Request zu senden. Wenn er einen token anfordert, ist er noch in keiner SESSION, deshalb wird eine neue gestartet. Beim

zweiten mal gibt es jedoch bereits eine SESSION für diesen Client und deshalb wird die SESSION mit der alten ID wieder geöffnet.

```

////////////////////////////////////
/// action === getToken
////////////////////////////////////
if ($action === 'getToken') {

    // Neue Session starten
    session_regenerate_id(true);

    $_SESSION['token'] = generateRandomString(10);

    echo json_encode([
        'token' => $_SESSION['token'],
        'sessionID' => session_id()
    ]);
    exit;
}

```

Abbildung 3-12: CodeSnippet databaseRequest.php action === 'getToken'

Wenn der Parameter vom http-Request 'action=getToken' lautet, wird dieser If ausgeführt. Die aktuelle Session, welche oben gestartet wurde, wird nun mit session_regenerate_id(true); neu erstellt. Dies dient als Sicherheitsvorkehrung, dass nicht die wiedergeöffnete Session verwendet wird, sondern nur ein Klon. Danach wird die Funktion generateRandomString(10); ausgeführt. Diese gibt einen Wert von 10 Zeichen zurück, welche in der Session-Variablen 'token' gespeichert wird. Als Rückgabewert wird der Token und die sessionID zurückgegeben.

```

////////////////////////////////////
/// action === getData
////////////////////////////////////
if ($action === 'getData') {
    $_SESSION['user'] = $user;

    // Prüfen ob alle erforderlichen Parameter vorhanden sind
    if (empty($user) || empty($clientToken) || empty($sessionID)) {
        $erfolg = fwrite($fh, date(DATE_RFC2822) . " : Fehlende Parameter : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode(['error' => 'Fehlende Parameter']);
        exit;
    }

    // Prüfen ob Session existiert und gültig ist
    if (!isset($_SESSION['token']) || !isset($_SESSION['user'])) {
        $erfolg = fwrite($fh, date(DATE_RFC2822) . " : Ungültige Session : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode(['error' => 'Ungültige Session']);
        exit;
    }

    // Prüfen ob der Benutzer übereinstimmt
    if ($_SESSION['user'] !== $user) {
        $erfolg = fwrite($fh, date(DATE_RFC2822) . " : Benutzer stimmt nicht überein : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode(['error' => 'Benutzer stimmt nicht überein']);
        exit;
    }

    // Passwort aus Datenbank holen
    $password = getUserPassword($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user);

    if (is_array($password) && !isset($password['error'])) {
        $erfolg = fwrite($fh, date(DATE_RFC2822) . " : " . $password['error'] . " : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode($password);
        exit;
    }

    // Server-seitigen Hash generieren
    $fullToken = $_SESSION['token'] . $baseCode . $password;
    $serverHash = hash('sha256', $fullToken);

    // Client-Hash mit Server-Hash vergleichen
    if ($clientToken !== $serverHash) {
        session_destroy();
        $erfolg = fwrite($fh, date(DATE_RFC2822) . " : Authentifizierung fehlgeschlagen : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode(['error' => 'Authentifizierung fehlgeschlagen']);
        exit;
    }

    switch ($requestMessage) {
        case 'getCards':
            $result = getCards($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user);
            break;

        case 'getDecks':
            $result = getDecks($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user);
            break;

        default:
            $erfolg = fwrite($fh, date(DATE_RFC2822) . " : Unbekannte Anfrage : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
            echo json_encode(['error' => 'Unbekannte Anfrage']);
            exit;
    }

    // Erfolgreiche Authentifizierung - hier würdest du die eigentliche Datenbankabfrage machen
    $data = array(
        "result" => $result ?? [],
        "serverHash" => $serverHash,
        "clientToken" => $clientToken,
        "fullToken" => $fullToken,
        "status" => "success",
        "user" => $_SESSION['user'],
        "request" => $requestMessage,
        "message" => "Authentifizierung erfolgreich"
    );
    // Hier würdest du die eigentlichen Ergebnisse der Datenbankabfrage hinzufügen

    echo json_encode($data);

    // Session nach erfolgreicher Anfrage zerstören
    session_destroy();
    exit;
}

```

Abbildung 3-13: CodeSnippet databaseRequest.php 'action===getData'

Wenn die action===getData lautet, wird dieser Code teil ausgeführt. Als erstes wird die Session variable 'user' mit dem mitgegebenen username befüllt. Danach wird überprüft, ob der Client alle erforderlichen Parameter mitgeschickt hat, ob die Session existiert und gültig ist und ob der Session-user mit dem mitgegeben Benutzer Namen übereinstimmt. Falls etwas nicht stimmt, wird ein Fehler zurückgesendet, ein Kommentar ins Logfile geschrieben und der Code wird unterbrochen. Danach wird die Funktion getUserPassword aufgerufen, welche das Passwort des Benutzers aus der Datenbank ausliest, welcher als Parameter mitgegeben wurde. Auch hier wird die Passwortvariable geprüft, dass kein Fehler entsteht. Mit dem Schlüsselwort 'hash' kann ein Hash erstellt werden. Wenn der ClientToken nicht mit dem vom Sever berechneten Hash übereinstimmt, wird ein Fehler zurückgegeben und der Code unterbrochen. Falls diese jedoch gleich sind, kommt der switch zum Zug. Der Client schickt als Parameter einen request-Schlüsselwort mit, welches in diesem Switch vermerkt sein muss. Fürs Testing gibt es erst zwei Möglichkeiten. Entweder 'getCards' oder 'getDecks'. Je nachdem wird eine andere Funktion aufgerufen, welche unten erläutert wird. Das Ergebnis wird als Array zurückgegeben (\$data). In diesem JSON-Array \$data enthält noch viele weitere Ausgaben, welche jedoch nur für Debugging Zwecke mitgesendet wurden. Wichtig ist hier 'result'. Zum Schluss wird

die Session restlos gelöscht, sodass sich niemand mehr mit dieser id anmelden kann.

```

////////////////////////////////////
/// function getCards
////////////////////////////////////

function getCards($mysql_host, $mysql_user, $mysql_password,
    $dbh = createDatabaseConnection($mysql_host, $mysql_user

    $sql = "SELECT * FROM cards";
    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        mysqli_close($dbh);
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);

    if (!$result) {
        mysqli_stmt_close($stmt);
        mysqli_close($dbh);
        return ['error' => 'Fehler bei der Abfrage'];
    }

    $cards = [];
    while ($row = mysqli_fetch_assoc($result)) {
        $cards[] = $row;
    }

    mysqli_stmt_close($stmt);
    mysqli_close($dbh);

    return $cards;
}

```

Abbildung 3-14: CodeSnippet databaseRequest.php getCards

Alle Funktionen, welche die Datenbank abfragen, sind sehr ähnlich aufgebaut. Ich habe deshalb versucht hier möglichst viel zusammen zu nehmen. Dies ist jedoch nur die Verbindung zum Server herstellen. Deshalb wird nun bei allen DatenbankabfrageFunktionen (z.B. getUserPassword, getCards, getDecks) als erstes createDatabaseConnection aufgerufen. In SQL wird die individuelle SQL-Query gespeichert, welche dann mit mysqli_stmt_execute ausgeführt wird. Wenn etwas zurückgekommen ist, wird jeder Datensatz in zusammengekommen in einem Array, welches dann zurückgegeben wird. Die mysql-Verbindung wird nach der Abfrage wieder geschlossen.

Mysqli ist die neuere Syntax für mysql. Das 'i' steht für Improved und die mysqli-Funktion steht seit PHP v5 zur Verfügung. Diese neue Version erlaubt auch Objektorientierte Datenbankabfragen. Ich verwende dies in meinem Projekt jedoch nicht, weil ich denke, dass es nur unnötig kompliziert wird.

3.2.5 Decks und Cards anzeigen V2.1

Bei dieser Funktionalität hat die Applikation einen grossen Schritt nach vorne gemacht mit dem Design. Nun kommt man beim Start der App auf die Indexseite, wo man alle Decks sieht, welche dem angemeldeten User gehören oder bei denen er Mitarbeiter ist. Auf dieser Seite gibt es eine Suchfunktion, um nach den Deck-Titeln zu suchen. Wenn man aufs Deck drückt, werden alle zugeordneten Karten und quizze in einer Übersichtsliste angezeigt. Auch hier kann man die Karten mit einer Suchfunktion nach den Fragen filtern. Weiter ist auf der Indexseite auch die Löschen Funktionalität vorhanden, mit der man ein ganzes Deck mit allen weiteren Entitäten, wie den Karten löschen kann.

3.2.6 Codeerklärung Decks und Cards anzeigen V2.1 (Final-Version)

3.2.6.1 Index.xaml & Index.xaml.cs

Das MainWindow.xaml hat nun keinen Nutzen mehr. Die gesamte Funktionalität ist in Index.xaml(.cs) übergegangen. Dies aus benennungsgründen, damit alles etwas übersichtlicher wurde. In diesem File hat es drei grosse Erweiterungen gegeben, sodass es nun auf der Endgültigen Version steht. Die Funktionen sind: Alle decks, welche dem Benutzer gehören oder bei denen er mitarbeitet im UI darstellen, in den angezeigten Decks suchen und einzelne Decks vollständig löschen.

Darstellen der Decks im UI

Das UI sieht wie im Bild rechts dargestellt aus. Die Logik und Funktionalität ist grundsätzlich dasselbe, wie beim Prototyp. Der Client schickt via http-Request einen Token anfordert. Mit diesem Token berechnen beide einen Authentifizierungshash. Danach sendet der Client eine Anfrage an den Server, welcher die Datenbank abfragt und in JSON-Format zurückgibt.

Das ganze wird wie folgt ins UI-übertragen:

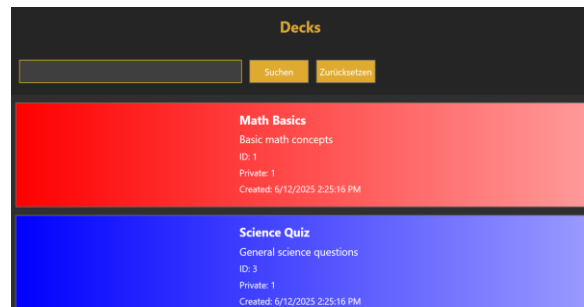


Abbildung 3-16: Index.xaml Final-UI

```
private async void getDecks()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionId = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionID: {sessionId}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _baseCode, _password);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await sendRequest.SendRequest(requestClient, "getDecks", _user, hashedToken, sessionId, "0");
            allDecks = JsonConvert.DeserializeObject<List<DeckList>>(responseData.ToString());
            filteredDecks = new List<DeckList>(allDecks);
            this.DataContext = filteredDecks;
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}

public class DeckList
{
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public int id { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string title { get; set; }
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string alt { get; set; }
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public int is_private { get; set; }
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public DateTime created_at { get; set; }
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string start_color { get; set; }
    0 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string end_color { get; set; }
}
```

Abbildung 3-15: Index.xaml.cs getDecks()-Funktion & DeckList-Class

Die Funktionalität der Datenbankabfrage entspricht im Wesentlichen dem Prototyp, wobei ich die JSON-Rückgabe nun nicht mehr in die Konsole ausgabe. Stattdessen wird die DeckList-Klasse verwendet, um die Rückgaben zu speichern. Mit DeserializeObject (Newtonsoft.Json) wird das JSON-Array in eine Sammlung von DeckList-Objekten umgewandelt, die in die Listen allDecks und filteredDecks geschrieben wird. Diese Listen dienen der Suchfunktion.

Im Gegensatz zum Prototyp habe ich auch die Datenbankabfrage angepasst. Als Parameter gibt man beim Methodenaufruf SendRequest den username mit, welcher hier für die Filterung der Daten benötigt wird. Es werden: id, titel, Alternativtext, ob das Deck privat ist, wann es erstellt wurde und die Beiden Farben abgefragt. Dies von den Decks, bei denen die userId mit der vom mitgegebenen User übereinstimmen. Es wird aber auch nach Collaborator gesucht, beiden Decks wo user_id mit dem User übereinstimmen.

```
SELECT d.id, d.title, d.alt, d.is_private, d.created_at, dc.start_color,
dc.end_color
FROM decks d
LEFT JOIN deck_colors dc ON d.id = dc.deck_id
WHERE d.creator_id = (
    SELECT u.id FROM users u WHERE u.username = ?) OR d.id IN (
    SELECT deck_id FROM collaborators WHERE user_id = (
        SELECT u.id FROM users u WHERE u.username = ?));
```


Um das Ganze im UI darzustellen, wird Binding verwendet. Binding ist das Schlüsselwort, um Daten aus dem Code-Behind ins Frontend zu bringen. Mit ItemsControll wird die Quelle der Daten so festgelegt, dass sie aus dem Code behind stammen. Das Ganze liegt in einem ScrollViewer, damit man scrollen kann, wenn es zu viele Decks wären. Für jedes Element aus DeckList wird ein neuer Button erstellt, welche auf die Deckübersicht weiterleitet. Der Button hat einen Farbverlauf, wie er in der Datenbank steht (start_color, end_color) und in TextBlöcken stehen die weiteren Informationen zum Deck mit verschiedenen Styles. Das ContextMenu wird fürs Löschen des Decks verwendet.

```

<ScrollViewer Grid.Row="2" VerticalScrollBarVisibility="Auto" Padding="5">
  <ItemsControl ItemsSource="{Binding}">
    <ItemsControl.ItemTemplate>
      <DataTemplate>
        <Button Margin="5" Padding="10" Click="DeckButton_Click" Tag="{Binding id}">
          <Button.Background>
            <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
              <GradientStop Offset="0" Color="{Binding start_color}" />
              <GradientStop Offset="1" Color="{Binding end_color}" />
            </LinearGradientBrush>
          </Button.Background>
          <StackPanel Orientation="Vertical">
            <TextBlock Text="{Binding title}" FontWeight="Bold" Foreground="White" FontSize="16" Margin="0,0,0,5" />
            <TextBlock Text="{Binding alt}" Foreground="White" FontSize="14" Margin="0,0,0,5" />
            <TextBlock Text="{Binding id, StringFormat: ID: {0}}" Foreground="White" Margin="0,0,0,5" />
            <TextBlock Text="{Binding is_private, StringFormat: Private: {0}}" Foreground="White" Margin="0,0,0,5" />
            <TextBlock Text="{Binding created_at, StringFormat: Created: {0}}" Foreground="White" Margin="0,0,0,5" />
          </StackPanel>
          <Button.ContextMenu>
            <ContextMenu>
              <MenuItem Header="Delete this Deck" Click="DeleteDeck_Click" Tag="{Binding id}" />
            </ContextMenu>
          </Button.ContextMenu>
        </Button>
      </DataTemplate>
    </ItemsControl.ItemTemplate>
  </ItemsControl>
</ScrollViewer>

```

Abbildung 3-17: Index.xaml Ausschnitt von Decksanzeige

Routing zwischen den Seiten

Wenn man auf den Button eines Decks drückt, wird dieser EventListener ausgeführt. Er öffnet ein neues Window, nämlich Deck.xaml. Bei Buttonclick wird dem Eventlistener den Parameter deckId mitgegeben. Dieser wird wiederum beim Instanzieren des neuen Fensters mitgegeben, mit weiteren Informationen zum aktuellen Fenster. Diese sind dazu da, dass das neue Fenster gleich gross und am gleichen Ort erscheint. Dieses neue Objekt von Deck wird dargestellt (.Show(); und das aktuelle Index-Fenster geschlossen (.Close();).

```

private void DeckButton_Click(object sender, RoutedEventArgs e)
{
    if (sender is Button btn && btn.Tag is int deckId)
    {
        var deckWindow = new Deck(this.Left, this.Top, this.Width, this.Height, this.WindowState, deckId);
        deckWindow.Show();
        this.Close();
    }
}

```

Abbildung 3-18: Index.xaml.cs Routing nach Deck.xaml

Nach Decks suchen

Im UI gibt es ein Inputfeld (TextBox), in welcher man einen Text eingeben kann, um nach einem Deck zu suchen. Weiter sind die Buttons 'Suchen' und 'Reset' vorhanden. Bei jedem ButtonClick wird eine Methode im Code-Behind ausgeführt. Wenn sich in der TextBox etwas ändert, oder der Suchen-Button gedrückt wird, dann wird

```

1 Verwendet 10 Änderungen | 10 Änderungen | 10 Änderungen
private void SearchBox_TextChanged(object sender, TextChangedEventArgs e)
{
    FilterDecks();
}

1 Verwendet 10 Änderungen | 10 Änderungen | 10 Änderungen
private void ResetButton_Click(object sender, RoutedEventArgs e)
{
    FilterDecks();
}

1 Verwendet 10 Änderungen | 10 Änderungen | 10 Änderungen
private void FilterDecks()
{
    SearchBox.Text = string.Empty;
    filteredDecks = new List<Deck>(allDecks);
    this.DataContext = filteredDecks;
}

2 Verwendet 10 Änderungen | 10 Änderungen | 10 Änderungen
private void FilterDecks()
{
    if (allDecks == null) return;
    string searchText = SearchBox.Text.Trim().ToLower();
    if (string.IsNullOrEmpty(searchText))
    {
        filteredDecks = new List<Deck>(allDecks);
    }
    else
    {
        filteredDecks = allDecks
            .Where(deck => deck.title.ToLower().Contains(searchText))
            .ToList();
    }
    this.DataContext = filteredDecks;
}

```

Abbildung 3-19: Index.xaml.cs Suchfunktion-Logik

FilterCards(); aufgerufen. Hier wird als erstes überprüft, ob die Liste allCards Leer ist, welche bei getDecks befüllt wurde mit allen Ergebnissen aus der Datenbank. Danach wird der eingegebene Text von Leerzeichen befreit und alles wird klein geschrieben. Wenn das Textfeld keinen Text beinhaltet, die Liste mit allen Decks neu instanziiert und als filteredDecks gespeichert. Wenn jedoch ein Suchbegriff eingegeben wurde, die Liste 'filteredDecks' mit den Elementen aus allDecks befüllt, welche den Suchtext beinhalten. Wenn der Reset-Button gedrückt wurde, dann wird der Inhalt der Textbox geleert und die Liste filteredDecks mit allDecks befüllt.

```

<Border Grid.Row="1" Background="White" Padding="10">
  <StackPanel Orientation="Horizontal">
    <TextBox x:Name="SearchBox"
      Width="300"
      Height="30"
      Margin="5"
      TextChanged="SearchBox_TextChanged"
      Background="White"
      Foreground="White"
      BorderBrush="Black"
      Padding="5" />
    <Button
      Width="80"
      Height="30"
      Margin="5"
      Click="SearchButton_Click"
      Background="Black"
      Foreground="White" />
    <Button
      Width="80"
      Height="30"
      Margin="5"
      Click="ResetButton_Click"
      Background="Black"
      Foreground="White" />
  </StackPanel>
</Border>

```

Abbildung 3-20: Index.xaml für die Suchfunktion

Decks löschen

Mit Rechtsklick auf ein Deck kann man es Löschen. Mit einem ContextMenu konnte bei Rechtsklick eine Auswahl von den MenuItems. Bei mir ist es nur eine Möglichkeit, nämlich 'Delete this Deck'. Wenn man dort draufdrückt, wird die DeleteDeck_Click-Methode ausgeführt. Auch hier wird die DeckId als Parameter mitgegeben. Es kommt noch eine MessageBox, ob man wirklich löschen will und falls ja gedrückt wird, wird die deleteDeck()-Methode ausgeführt. Diese ist sehr ähnlich wie getDecks. Der Unterschied ist, dass der Parameter request=deleteDeck lautet. Dies löst im PHP-Script eine andere Funktion aus.

```
<Button.ContextMenu>
  <ContextMenu>
    <MenuItem Header="Delete this Deck" Click="DeleteDeck_Click" Tag="{Binding id}"/>
  </ContextMenu>
</Button.ContextMenu>
```

Abbildung 3-21: Index.xaml Ausschnitt für Löschfunktion

```
private void DeleteDeck_Click(object sender, RoutedEventArgs e)
{
    var menuItem = sender as MenuItem;
    var deckId = menuItem?.Tag?.ToString();

    if (!string.IsNullOrEmpty(deckId))
    {
        var result = MessageBox.Show(
            "Sind Sie sicher, dass Sie dieses Deck löschen möchten?",
            "Deck löschen",
            MessageBoxButton.YesNo,
            MessageBoxImage.Question);

        if (result == MessageBoxResult.Yes)
        {
            deleteDeck(deckId);
            getDecks();
        }
    }
}
```

Abbildung 3-22: Index.xaml.cs Löschfunktion Event

Die deleteDeck Funktion im PHP ist ähnlich aufgebaut, wie die Get-abfragen. Zuerst wird eine Verbindung zur Datenbank hergestellt. Danach beginnt eine Transaction. Dies ermöglicht mehr als nur eine Abfrage. Da man hier mehrere Tabellen löschen muss, welche alle zu einem Deck gehören, eignet sich dies ganz gut. Im Array \$queries werden alle Datenbankinteraktionen definiert. Und danach mit einer foreach-Schleife jede Query ausgeführt.

3.2.6.2 Deck.xml und Deck.xml.cs

Im deck.xml(.cs) ist es grundsätzlich dasselbe, wie im Index.xml. Hier werden alle Karten eines Decks angezeigt, wessen Id per Parameter bei Seitenaufruf mitgegeben wurde. Hier sind noch einige Variablen mehr in der CardList klasse. Auch die SQL-Abfrage hierfür ist einiges komplexer, weil alle cards und alle quiz des Decks angezeigt werden. Es sind sozusagen zwei Queries und beide habe eine andere Ausgabe. Mit dem Schlüsselwort UNION ALL können zwei verschiedene Abfragen zu einer zusammengehängt werden. Die Suchfunktion und das Darstellen der Daten funktionierten grundsätzlich sehr ähnlich wie oben beschrieben. Neu ist auch, dass man mit dem Backbutton

```
SELECT
'card' AS type,
c.id,
c.question,
c.answer,
NULL AS correct_answer,
NULL AS first_option,
NULL AS second_option,
NULL AS third_option,
NULL AS fourth_option,
c.is_fav,
c.status,
c.created_at,
d.title
FROM cards c
JOIN decks d ON c.deck_id = d.id
WHERE c.deck_id = ?

UNION ALL

SELECT
'quiz' AS type,
q.id,
q.question,
NULL AS answer,
q.correct_answer,
q.first_option,
q.second_option,
q.third_option,
q.fourth_option,
q.is_fav,
q.status,
q.created_at,
d.title
FROM quiz q
JOIN quiz_options qo ON q.id = qo.quiz_id
JOIN decks d ON q.deck_id = d.id
WHERE q.deck_id = ?;
```

Abbildung 3-24: SQL-Abfrage für cards und quiz

Deck Seite wird hier ein neues Window instanziiert, welches die Indexseite darstellt. Dies auch wieder mit dem Parameter, welche die Grösse des Fensters angeben. Hier war das Problem, dass beim ersten Erstellen des Windows diese Parameter nicht erlaubt sind. Wenn ein Window ganz zu Beginn gestartet wird, darf der Konstruktor keine Werte verlangen, da das Objekt zu Beginn leer instanziiert wird. Die Lösung hierfür waren also 2 verschiedene Konstruktoren, einen mit den Parametern und einen ohne. Wenn die Applikation als erstes gestartet wird, werden die Parameter per Default gesetzt. Wenn jedoch das Fenster später während dem Laufen der Applikation aufgerufen wird, dann werden die Konstruktor Parameter verwendet, welche mitgegeben werden.

```
function deleteDeck($mysql_host, $mysql_user, $mysql_password, $mysql_database, $mysql_id) {
    $db = createDatabaseConnection($mysql_host, $mysql_user, $mysql_password, $mysql_database, $mysql_id);

    $mysql_begin_transaction($db);

    try {
        $queries = [
            "DELETE FROM collaborators WHERE deck_id = ?",
            "DELETE FROM deck_colors WHERE deck_id = ?",
            "DELETE FROM cards WHERE deck_id = ?",
            "DELETE FROM quiz WHERE deck_id = ?",
            "DELETE FROM quiz_options WHERE quiz_id IN (SELECT id FROM quiz WHERE deck_id = ?)",
            "DELETE FROM decks WHERE id = ?"
        ];

        foreach ($queries as $sql) {
            $stmt = $mysql_prepare($db, $sql);
            if ($stmt) {
                throw new Exception("SQL Prepare fehlergeschlagen: " . $mysql_error($db));
            }
            $mysql_stmt_bind_param($stmt, "i", $deck_id);
            if ($mysql_stmt_execute($stmt)) {
                throw new Exception("SQL Ausführung fehlergeschlagen: " . $mysql_stmt_error($stmt));
            }
            $mysql_stmt_close($stmt);
        }

        $mysql_commit($db);
        $mysql_close($db);
        return ["success" => true];
    } catch (Exception $e) {
        $mysql_rollback($db);
        $mysql_close($db);
        return ["error" => $e->getMessage()];
    }
}
```

Abbildung 3-27: databaseRequest.php request==deleteDeck

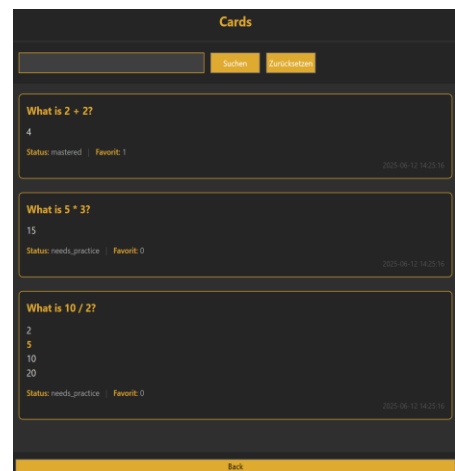


Abbildung 3-26: Deck.xml UI mit dargestellten Karten

```
private void BackButton_Click(object sender, RoutedEventArgs e)
{
    var indexWindow = new Index(this.Left, this.Top, this.Width, this.Height, this.WindowState);
    indexWindow.Show();
    this.Close();
}
```

Abbildung 3-25: Deck.xml.cs routing auf Indexpage

```
public Index() : this(100, 100, 800, 450, WindowState.Normal)
{
    InitializeComponent();
    getDecks();
}

2 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
public Index(double left, double top, double width, double height, WindowState state)
{
    InitializeComponent();
    getDecks();

    this.Left = left;
    this.Top = top;
    this.Width = width;
    this.Height = height;
    this.WindowState = state;
}
```

Abbildung 3-23: Index.xml.cs Konstruktoren

3.2.7 Codeerklärung Decks hinzufügen V2.2

Auf einer weiteren zusätzlichen Seite können Decks hinzugefügt werden. Für die Decks können Farben, Titel und eine Beschreibung eingegeben werden.

3.2.7.1 CreateDeck.xaml, CreateDeck.xaml.cs und postData.cs

Farbauswahl

Ein Deck zu erstellen ist ziemlich simpel. Man muss einfach alle Felder ausfüllen und Create Deck drücken. Jedes Deck hat zwei Farben, aus welchen ein Farbverlauf entsteht beim Anzeigen auf der Indexseite. Für diese ColorPicker-funktion konnte ich verwerde diese Bibliothek: <https://github.com/dsa/wpf-color-picker>. Es war sehr einfach diese zu installieren, das sie mit dem NuGet-Packagemanager erhältlich ist. Es hatte eine gute Dokumentation und ein gutes Beispiel, für die Implantation.

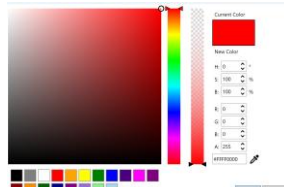


Abbildung 3-29: Colorpicker UI

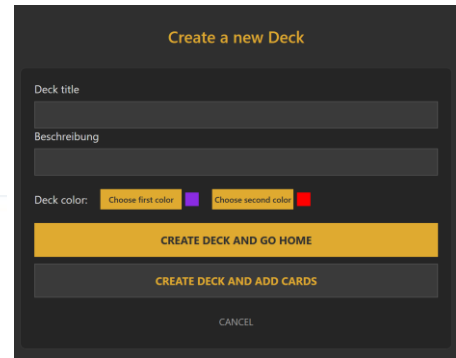


Abbildung 3-28: Deck erstellen UI

einfach eine Library installieren. Ich

Ich musste die Farbauswahl zweimal implementieren, da zwei separate Farben benötigt wurden. Beim Klicken auf den *Choose Color*-Button wird eine neue Instanz des ColorPickerDialog erstellt und das Dialogfenster geöffnet. Nach der Farbauswahl wird der entsprechende Setter mit dem gewählten Wert aktualisiert, der dann für weitere Verarbeitung genutzt werden kann. Als Standardwerte sind Violett für die erste und Rot für die zweite Farbe voreingestellt.

```
public static readonly DependencyProperty FirstColorProperty =
    DependencyProperty.Register(nameof(FirstColor), typeof(Color), typeof(CreateDeck), new PropertyMetadata(Colors.BlueViolet));

public static readonly DependencyProperty SecondColorProperty =
    DependencyProperty.Register(nameof(SecondColor), typeof(Color), typeof(CreateDeck), new PropertyMetadata(Colors.Red));

// Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
public Color FirstColor
{
    get => (Color)GetValue(FirstColorProperty);
    set => SetValue(FirstColorProperty, value);
}

// Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
public Color SecondColor
{
    get => (Color)GetValue(SecondColorProperty);
    set => SetValue(SecondColorProperty, value);
}

// Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
private void PickFirstColorButton_Click(object sender, RoutedEventArgs e)
{
    var dialog = new ColorPickerDialog(FirstColor);
    dialog.Owner = this;
    var res = dialog.ShowDialog();
    if (res.HasValue && res.Value)
    {
        FirstColor = dialog.Color;
    }
}

// Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
private void PickSecondColorButton_Click(object sender, RoutedEventArgs e)
{
    var dialog = new ColorPickerDialog(SecondColor);
    dialog.Owner = this;
    var res = dialog.ShowDialog();
    if (res.HasValue && res.Value)
    {
        SecondColor = dialog.Color;
    }
}
```

Abbildung 3-30: Code für colorpicker

Eine besondere Herausforderung bestand darin, die ausgewählte Farbe in der Benutzeroberfläche anzuzeigen. Dafür habe ich eine neue Klasse ColorToBrushConverter erstellt, die auf Elementen der Bibliothek basiert und von einem Beispiel in der Dokumentation inspiriert war. Im XAML-Frontend musste ich diese Klasse zunächst in den Window.Resources registrieren, um sie anschließend nutzen zu können. Damit ließ sich ein Rechteck mit der dynamisch ausgewählten Farbe füllen.

```
public class ColorToBrushConverter : IValueConverter
{
    // Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is Color color)
            return new SolidColorBrush(color);
        return Brushes.Transparent;
    }

    // Verweise | Jan Bretscher, Vor 2 Stunden | 1 Autor, 1 Änderung
    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}
```

Abbildung 3-32: ColorToBrushConverter Klasse

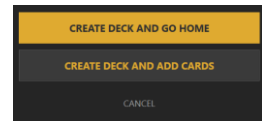
```
<Window.Resources>
{
    <Local:ColorToBrushConverter x:Key="ColorToBrushConverter"/>
}
</Window.Resources>

<Rectangle Fill="{Binding FirstColor, Converter={StaticResource ColorToBrushConverter}}" Width="20" Height="20" Margin="5"/>
```

Abbildung 3-31: Xaml für Farbvorschau

Aktionsbuttons

Es gibt drei verschiedene Aktionsbuttons auf dieser Seite. Create & go home, create & add cards oder cancel. Wenn man das Deck created, wird die Methode createDeck aufgerufen, welche wie gewohnt einen Token anfordert, diesen Hashed und danach die AddDeck-Methode aufruft.



Aktionsbuttons im UI

```
private async void createDeck()
{
    try
    {
        if (string.IsNullOrWhiteSpace(title.Text))
        {
            MessageBox.Show("Bitte geben Sie einen Titel für das Deck ein.", "Fehler", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionId = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionId}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _baseCode, _password);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await addDeck.AddDeck(requestClient, "addDeck", _user, hashedToken, sessionId,
                removeHashtagColor.RemoveHashtagColor(FirstColor.ToString()), removeHashtagColor.RemoveHashtagColor(SecondColor.ToString()), title.Text, alt.Text);
            Console.WriteLine(responseData.ToString());
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

Abbildung 3-33: CreateDeck.xaml.cs createDeck()-Methode

Ich habe hier sehr viele Parameter, welche ich mitgeben muss. Dies sind: addDeck, username, hashedToken, sessionId, firstColor, secondColor, title und Beschreibung. Es ist bestimmt nicht die sauberste Lösung mit diesen vielen Parametern, jedoch eine zuverlässige und sichere Möglichkeit. Mit title.Text und alt.Text werden die Textfelder ausgelesen. Zuoberst im Try wird überprüft, ob das Textfeld title einen Inhalt hat. Wenn nicht, kommt ein Fehler-Window. Ich hatte grosse Probleme mit dem Übertragen der Parameterwerte per http-Request. Dies lag daran, dass in der URL-Hashtags ('#') und Under Lines ('_') nicht erlaubt waren. Da ich die Farben jedoch als Hex Wert übertragen habe (Format: '#FFFFFF') störte dies die URL. Bei einem # wird ein 'anker' (link) erwartet, wie er in HTML z.B. gegeben werden könnte. Ich musste also eine Lösung finden, um die Farben ohne den # zu übertragen, danach jedoch trotzdem richtig in der Datenbank abspeichern. Ich habe hierfür also eine neue Klasse erstellt, welche mit Regex den # vom String entfernt. Der Pattern ist die Vorlage des Strings also zuerst ein # und danach 8 Zeichen von 0-9, a-f und A-F. Danach wird überprüft, ob der String mit der Vorlage übereinstimmt. Falls ja, dann wird das erste Zeichen entfernt ('#') und der neue String zurückgegeben.

```
public static string RemoveHashtagColor(string color)
{
    string pattern = @"#[0-9a-fA-F]{8}";
    Match match = Regex.Match(color, pattern);
    if (match.Success)
    {
        string hexColor = match.Groups[1].Value;
        Console.WriteLine(hexColor);
        return hexColor;
    }
    Console.WriteLine("Keine gültige Farbe gefunden.");
    return "";
}
```

Abbildung 3-34: removeHashtagColor.cs Regex-Funktion

3.2.7.2 AddDecks in databaseRequest.php

Als die Farben noch mit einem # zuvor übergeben wurde, war die Ausgabe ab den Farben immer der Default Wert. Wenn kein gültiger Parameter übergeben wurde, dann wird einfach der Default Wert daneben verwendet. Da nun die Farben keine # mehr haben, müssen diese wieder angefügt werden. Diese mache ich mit einer einfachen Konkatination. Danach wird die Funktion

```
$startColor = $_GET['startColor'] ?? '#ffffff';
$endColor = $_GET['endColor'] ?? '#000000';
$title = $_GET['title'] ?? null;
$alt = $_GET['alt'] ?? $title;
```

Abbildung 3-35: Parametervariablen definieren

```
case 'addDeck':
    $startColor = '#' . $startColor;
    $endColor = '#' . $endColor;
    $result = addDeck($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $startColor, $endColor, $title, $alt);
    break;
```

Abbildung 3-36: action = addDeck

addDeck aufgerufen, wo die SQL-Abfrage stattfindet.

Um ein Deck hinzuzufügen, müssen zwei Tabellen angepasst werden. Dies sind 'decks' und 'deck_colors'. Zuerst werden die Werte, welche als Parameter mitgegeben wurden, in decks hinzugefügt. Wenn dies erfolgreich ausgeführt werden konnte, werden die Werte in deck_colors hinzugefügt. Wenn beides erfolgreich ablief, wird ein success und die deckId zurückgegeben.

```
function addDeck($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $startColor, $endColor, $title, $alt) {
    $dbh = createDatabaseConnection($mysql_host, $mysql_user, $mysql_password, $mysql_database, fopen('./log.txt', 'a'));
    $error = null;
    $deckId = null;

    // Erste Transaktion: Deck erstellen
    $sql1 = "INSERT INTO decks (title, alt, is_private, creator_id) VALUES (?, ?, FALSE, (SELECT id FROM users WHERE username = ?))";
    $stmt1 = mysqli_prepare($dbh, $sql1);

    if ($stmt1 && mysqli_stmt_bind_param($stmt1, "sss", $title, $alt, $user) && mysqli_stmt_execute($stmt1)) {
        $deckId = mysqli_insert_id($dbh);
        mysqli_stmt_close($stmt1);

        // Zweite Transaktion: Farben hinzufügen
        $sql2 = "INSERT INTO deck_colors (deck_id, start_color, end_color) VALUES (?, ?, ?)";
        $stmt2 = mysqli_prepare($dbh, $sql2);

        if ($stmt2 && mysqli_stmt_bind_param($stmt2, "iss", $deckId, $startColor, $endColor) && mysqli_stmt_execute($stmt2)) {
            mysqli_stmt_close($stmt2);
        } else {
            $error = 'Fehler beim Hinzufügen der Deck-Farben: ' . ($stmt2 ? mysqli_stmt_error($stmt2) : mysqli_error($dbh));
        }
    } else {
        $error = 'Fehler beim Erstellen des Decks: ' . ($stmt1 ? mysqli_stmt_error($stmt1) : mysqli_error($dbh));
    }

    mysqli_close($dbh);

    return $error
        ? ['error' => $error]
        : ['success' => true, 'deckId' => $deckId];
}
```

Abbildung 3-37: databaseRequest.php addDeck-Funktion

3.2.8 Codeerklärung Cards hinzufügen & bearbeiten V2.3

Die Bezeichnung addCards stimmt nicht ganz. Es sollte eigentlich create and edit Cards sein. Zuerst war die Funktionalität nur beim Karten hinzufügen. Ich erweiterte die Klasse danach jedoch so, dass auch die Bearbeitung von den Karten möglich ist. Hierfür werden die Karten zuerst aus der Datenbank abgefragt. Danach werden sie im «Bearbeitungsmodus» dargestellt, wo man sie ändern, löschen oder neue hinzufügen kann. Mit 'Save changes' wird alles gespeichert.

3.2.8.1 AddCards.xaml.cs & AddCards.xaml

```
5 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
public abstract class BaseCard
{
    [JsonPropertyName("question")]
    4 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Question { get; set; } = "";
}

3 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
public class Card : BaseCard
{
    [JsonPropertyName("answer")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Answer { get; set; } = "";
}

3 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
public class QuizCard : BaseCard
{
    [JsonPropertyName("option1")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Option1 { get; set; } = "";

    [JsonPropertyName("option2")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Option2 { get; set; } = "";

    [JsonPropertyName("option3")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Option3 { get; set; } = "";

    [JsonPropertyName("option4")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string Option4 { get; set; } = "";

    [JsonPropertyName("correctIndex")]
    2 Verweise | Jan Bretscher, Vor 31 Minuten | 1 Autor, 1 Änderung
    public string CorrectIndex { get; set; }
}
```

Abbildung 3-39: AddCards.xaml.cs Klassen für die Darstellung der Daten

Datenspeicherung für UI-Anzeige

Diese drei Klassen sind für die Speicherung aller Karten. Das Frontend (AddCards.xaml) greift auf die Daten aus diesen Klassen zu, um sie darzustellen. Die 'BaseCard'-Klasse ist für beide Karten-Typen. Die Gemeinsamkeit ist die Frage. Die Card und QuizCard Klasse erben also Question von der BaseCard Klasse. Da alle anderen Attribute unterschiedlich sind, können diese nicht in BaseCard verallgemeinert werden.

Abbildung 3-38: addCards UI

Bestehende Karten abrufen

Die einzelnen Karten werden sehr ähnlich wie bei [Deck.xaml.cs](#) abgerufen, da es auch die gleichen Daten sind, die benötigt werden. Zuerst wird ein Token angefordert, danach gehashed und mit dem request=getCards Parameter an den Server geschickt. Aus der Rückgabe (JSON) wird eine Liste aus Dictionaries erstellt (cardsData). Mit einer foreach-Schleife wird diese Liste abgearbeitet. Das 'item' ist dann wie ein Dictionary mit den entsprechenden Daten drin. Wenn also der type des Dictionarys == 'card' ist, wird ein neues Card-Objekt und bei einem quiz wird ein QuizCard-Objekt instanziiert. Die weiteren Daten aus dem Dictionary werden dann in diesem Objekt gespeichert.

```
private async void getCards()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionID = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionID}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _baseCode, _password);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await sendRequest.SendRequest(requestClient, "getCards", _user, hashedToken, sessionID, _deckId.ToString());
            var cardsData = System.Text.Json.JsonSerializer.Deserialize<List<Dictionary<string, object>>>(responseData.ToString());

            foreach (var item in cardsData)
            {
                if (item["type"].ToString() == "card")
                {
                    Cards.Add(new Card
                    {
                        Question = item["question"].ToString(),
                        Answer = item["answer"].ToString(),
                    });
                }
                else if (item["type"].ToString() == "quiz")
                {
                    Cards.Add(new QuizCard
                    {
                        Question = item["question"].ToString(),
                        Option1 = item["first_option"].ToString(),
                        Option2 = item["second_option"].ToString(),
                        Option3 = item["third_option"].ToString(),
                        Option4 = item["fourth_option"].ToString(),
                        CorrectIndex = item["correct_answer"].ToString(),
                    });
                }
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

Abbildung 3-41: AddCards.xaml.cs getCards -> bereits bestehende Karten abrufen

Buttonsteuerung

In diesem Fenster gibt es einige Buttons, welche für vieles verwendet werden. Als erstes gibt es die Buttons, um Karten und Quiz hinzuzufügen. Hier wird jeweils ein neues Objekt der Klassen Card/QuizCard instanziiert. Dieses Objekt wird an der Collection Cards angefügt. Die Objekte sind jedoch noch leer, da sie noch nicht befüllt wurden. Beim Löschen wird das Objekt aus der Collection gelöscht, welches beim Buttonclick als Parameter mitgegeben wurde. Bei den beiden weiteren Buttons kommt man wieder zurück auf die Index Seite. Mit dem SaveChanges-Button werden die Daten zuvor noch gespeichert.

```
public ObservableCollection<BaseCard> Cards { get; set; } = new ObservableCollection<BaseCard>();
private void CreateCardButton_Click(object sender, RoutedEventArgs e)
{
    Cards.Add(new Card());
}

// Versteckt das Breitscher, Vor 1 Stunde | 1 Autor, 2 Änderungen
private void CreateQuizButton_Click(object sender, RoutedEventArgs e)
{
    Cards.Add(new QuizCard());
}

// Versteckt das Breitscher, Vor 1 Stunde | 1 Autor, 1 Änderung
private void RemoveCard_Click(object sender, RoutedEventArgs e)
{
    if (sender is Button btn && btn.DataContext is BaseCard card)
    {
        Cards.Remove(card);
    }
}

// Versteckt das Breitscher, Vor 1 Stunde | 1 Autor, 2 Änderungen
private async void SaveChangesButton_Click(object sender, RoutedEventArgs e)
{
    await SendCardsToApiAsync();
    goHome();
}

// Versteckt das Breitscher, Vor 1 Stunde | 1 Autor, 1 Änderung
private void BackButton_Click(object sender, RoutedEventArgs e)
{
    goHome();
}

// Versteckt das Breitscher, Vor 1 Stunde | 1 Autor, 1 Änderung
private void goHome()
{
    var indexWindow = new Index(this.Left, this.Top, this.Width, this.Height, this.WindowState);
    indexWindow.Show();
    this.Close();
}
```

Abbildung 3-40: AddCards.xaml.cs Buttonfunktionen

Frontend (xaml)

Das Frontend muss für diesen Zweck sehr flexibel sein. Hier werden die beiden Vorlagen (DataTemplates) in den Window.Resources definiert. Wenn es eine Karte ist, dann werden zwei Textfelder angezeigt und bei einem Quiz sind es sechs. Dies ist nur eine Vorlage für die Daten. Mit diesem Code würde noch nichts dargestellt werden. Unten im Scrollviewer werden die Daten aus der Collection Cards genommen und oben je nach type wird ein anderes Formular dargestellt.

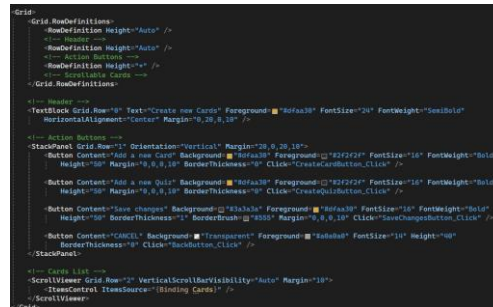


Abbildung 3-43: AddCards.xaml Anzeige

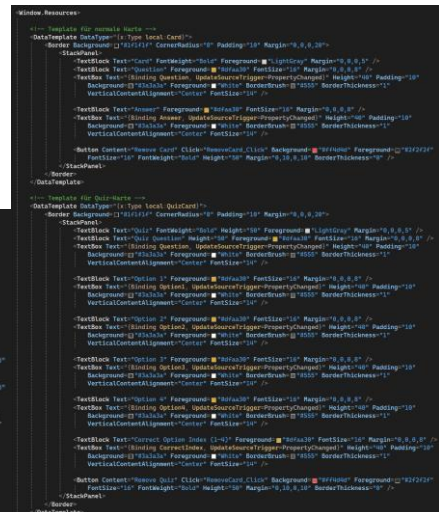


Abbildung 3-42: AddCards.xaml DataTemplate

Änderungen Speichern und ans Backend senden

Die Daten werden mit einem http-post an den Server gesendet. Die Authentifizierungs-Idee ist grundsätzlich dieselbe, wie bei allen get-requests. Dem http-post wird ein JSON mitgegeben, welches mit der Methode CreateJsonFromCards() erstellt wird. Mithilfe von LINQ wird die Cards-Collection gefiltert und jeweils eine neue anonyme Objektliste erstellt, die die relevanten Eigenschaften enthält, z.B. question, answer, option1, correctIndex usw.

Zusätzlich werden pro Karte zwei weitere feste Felder ergänzt: is_fav wird standardmäßig auf false gesetzt und status auf "needs_practice". Diese Informationen beschreiben z.B., ob eine Karte als Favorit markiert wurde oder ob sie erneut geübt werden muss.

Die eigentliche Umwandlung in JSON übernimmt dann der System.Text.Json.JsonSerializer. Mit den Optionen PropertyNamingPolicy = JsonNamingPolicy.CamelCase und WriteIndented = true wird sichergestellt, dass das JSON leserlich und API-konform formatiert ist.

In der Methode SendCardsToApiAsync() wird nach dem Erstellen vom JSON die SendCardsAsync-Methode aufgerufen. Diese Methode sendet das JSON mit den nötigen Parametern an das PHP-Script. Wenn die Aktion erfolgreich durchgeführt werden konnte, ist der responseBody einfache { "Success":true }. Mehr zur serverseitigen Verarbeitung weiter unten.

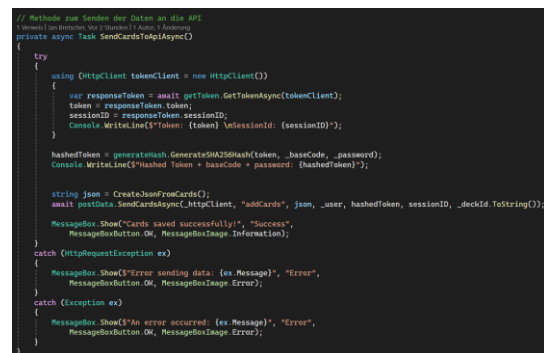


Abbildung 3-44: Methode: SendCardsToApiAsync()

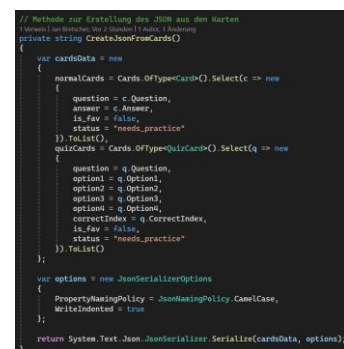


Abbildung 3-45: AddCards.xaml.cs erstellt JSON

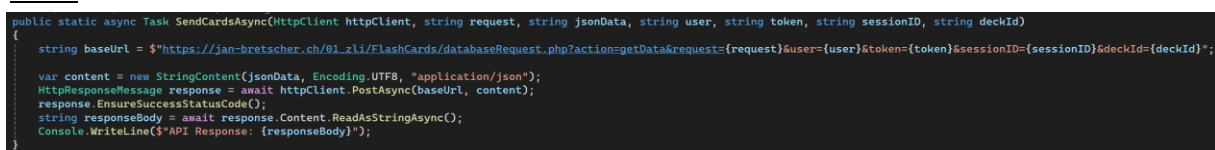


Abbildung 3-46: postData.cs

3.2.8.2 DatabaseRequest.php v2.3

Dem http-Request wird als Parameter request=addCards mitgegeben. Es wird entsprechend der Funktion 'addCards' aufgerufen. Auch diese Funktion ist sehr ähnlich aufgebaut, wie alle anderen, welche etwas mit der Datenbank zu tun haben.

```
$sql = "DELETE FROM cards WHERE deck_id = ?";
$stmt = mysqli_prepare($dbh, $sql);
mysqli_stmt_bind_param($stmt, "i", $deckId);
mysqli_stmt_execute($stmt);
mysqli_stmt_close($stmt);
$sql = "DELETE FROM quiz WHERE deck_id = ?";
$stmt = mysqli_prepare($dbh, $sql);
mysqli_stmt_bind_param($stmt, "i", $deckId);
mysqli_stmt_execute($stmt);
mysqli_stmt_close($stmt);

// Process normal cards
if (isset($_input['normalCards'])) {
    foreach ($_input['normalCards'] as $card) {
        $question = $card['question'] ?? '';
        $answer = $card['answer'] ?? '';
        $isFav = $card['is_fav'] ? 1 : 0; // Convert boolean to integer
        $status = $card['status'] ?? 'needs_practice';

        // Validate inputs
        if (empty($question) || empty($answer)) {
            throw new Exception("Frage oder Antwort darf nicht leer sein");
        }

        $sql = "INSERT INTO cards (deck_id, question, answer, is_fav, status) VALUES (?, ?, ?, ?, ?)";
        $stmt = mysqli_prepare($dbh, $sql);
        mysqli_stmt_bind_param($stmt, "issii", $deckId, $question, $answer, $isFav, $status);
        mysqli_stmt_execute($stmt);
        mysqli_stmt_close($stmt);
    }
}

if (isset($_input['quizCards'])) {
    foreach ($_input['quizCards'] as $quizCard) {
        $question = $quizCard['question'] ?? '';
        $isFav = $quizCard['is_fav'] ? 1 : 0; // Convert boolean to integer
        $status = $quizCard['status'] ?? 'needs_practice';
        $correctIndex = $quizCard['correctIndex'] ?? 1;

        // Validate inputs
        if (empty($question)) {
            throw new Exception("Frage darf nicht leer sein");
        }

        // Insert into quiz table
        $sql = "INSERT INTO quiz (deck_id, question, is_fav, correct_answer, status) VALUES (?, ?, ?, ?, ?)";
        $stmt = mysqli_prepare($dbh, $sql);
        mysqli_stmt_bind_param($stmt, "issii", $deckId, $question, $isFav, $correctIndex, $status);
        mysqli_stmt_execute($stmt);
        $quizId = mysqli_insert_id($dbh);
        mysqli_stmt_close($stmt);

        // Insert options into quiz_options table
        $firstOption = $quizCard['option1'] ?? '';
        $secondOption = $quizCard['option2'] ?? '';
        $thirdOption = $quizCard['option3'] ?? '';
        $fourthOption = $quizCard['option4'] ?? '';

        // Validate options
        if (empty($firstOption) || empty($secondOption) || empty($thirdOption) || empty($fourthOption)) {
            throw new Exception("Alle Optionen müssen ausgefüllt sein");
        }
        if ($correctIndex < 1 || $correctIndex > 4) {
            throw new Exception("correctIndex muss zwischen 1 und 4 liegen");
        }

        $sql = "INSERT INTO quiz_options (quiz_id, first_option, second_option, third_option, fourth_option) VALUES (?, ?, ?, ?, ?)";
        $stmt = mysqli_prepare($dbh, $sql);
        mysqli_stmt_bind_param($stmt, "isss", $quizId, $firstOption, $secondOption, $thirdOption, $fourthOption);
        mysqli_stmt_execute($stmt);
        mysqli_stmt_close($stmt);
    }
}
```

Abbildung 3-47: databaseRequest.php Funktion addCards

Als erstes wird das mit dem post mitgegebene JSON ausgelesen und in der Variablen `$input` gespeichert.

```
$input = json_decode(file_get_contents('php://input'), true);
if (!$input) {
    return ['error' => 'Invalid JSON input'];
}
```

Danach werden alle Cards gelöscht, welche dem Deck zugewiesen sind, welches der als Parameter mitgegebenen deckId entspricht. Dies wird für cards und quiz gemacht. Das JSON hat eine folgende Grundstruktur:

Abbildung 3-48: JSON aus dem post herauslesen

```
{
  "normalCards": [
    {
      "question": "test",
      "answer": "card",
      "is_fav": false,
      "status": "needs_practice"
    },
    {
      "question": "quiz",
      "answer": "card",
      "is_fav": false,
      "status": "needs_practice"
    }
  ],
  "quizCards": [
    {
      "question": "this",
      "option1": "is ",
      "option2": "a",
      "option3": "test",
      "option4": "quiz",
      "correctIndex": 0,
      "is_fav": false,
      "status": "needs_practice"
    }
  ]
}
```

Es wird geprüft, ob es einen Eintrag für 'normalCards' oder 'quizCards' gibt. Wenn dort ein Inhalt vorhanden ist, für jedes weitere Array die variablen Werte ausgelesen und dann mit dem entsprechenden SQL-Query in die richtige Tabelle eingefügt. Beim insert von Quiz ist es etwas aufwendiger, da die Tabelle quiz und quiz_options befüllt werden müssen.

3.2.9 Codeerklärung Learn-Cards V2.4

Die Learn-Funktion habe ich mit zwei verschiedenen Files umgesetzt. Zum einen das Interaktive lernen und zum anderen eine Auswertung von den Fragen.

Es gibt zwei Typen von Karten, entweder eine Frage und eine Antwort oder eine Frage und 4 Antwortmöglichkeiten. Je nach Typ wird das UI entsprechend angepasst. Bei einem Quiz sind es einfach vier Buttons und einer ist der richtige. Bei einer normalen Karte steht die Frage und man muss die Antwort in ein Textfeld schreiben.

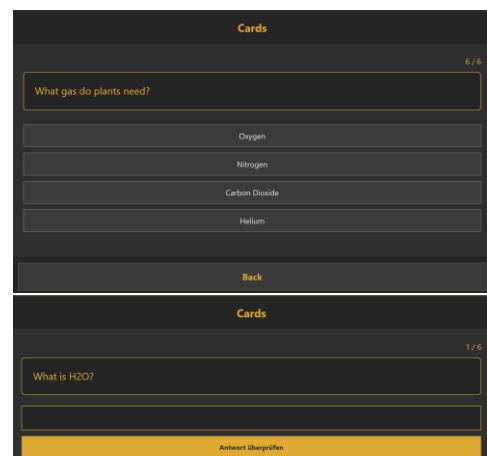


Abbildung 3-49: LearnDeck UI von card & quiz

In der Auswertung sieht man, wie viele Fragen man richtig bzw. falsch hatte und die Erfolgsquote in Prozent. Unten sind alle Fragen aus dem Deck aufgelistet und die mit der jeweiligen Farbe markiert, je nachdem, ob sie richtig oder falsch gelöst wurden. Unten ist die richtige Lösung noch aufgeschrieben.

3.2.9.1 LearnDeck.xaml und LearnDeck.xaml.cs

Zu beginn wird diese Methode aufgerufen, welche zuerst alle Daten des Decks beim Server abfragt und in eine neue Liste 'addCards' und 'filteredCards' speichert.

```
allCards = JsonConvert.DeserializeObject<List<Cards>>(responseData.ToString());
filteredCards = new List<Cards>(allCards);
this.DataContext = filteredCards;
```

Abbildung 3-51: Abgefragte Decks in Liste speichern

```
private async void LoadDataAndStartLearning()
{
    await getCards();

    if (filteredCards?.Count > 0)
    {
        LoadCurrentCard();
    }
    else
    {
        QuestionTextBlock.Text = "Keine Karten verfügbar";
    }
}
```

Abbildung 3-52: LearnDeck.xaml.cs Laden der Daten

```
private void LoadCurrentCard()
{
    if (filteredCards == null || filteredCards.Count == 0) return;

    QuestionTextBlock.Text = currentCard.question;

    if (currentCard.type == "card")
    {
        // Normale Karte
        QuizOptionsPanel.Visibility = Visibility.Collapsed;
        AnswerInputPanel.Visibility = Visibility.Visible;
        AnswerTextBox.Text = "";
    }
    else if (currentCard.type == "quiz")
    {
        // Quiz-Karte
        AnswerInputPanel.Visibility = Visibility.Collapsed;
        QuizOptionsPanel.Visibility = Visibility.Visible;

        // Setze die Quiz-Optionen
        Option1Button.Content = currentCard.first_option;
        Option2Button.Content = currentCard.second_option;
        Option3Button.Content = currentCard.third_option;
        Option4Button.Content = currentCard.fourth_option;

        // Zurücksetzen der Auswahl
        foreach (var child in QuizOptionsPanel.Children)
        {
            if (child is Button button)
            {
                button.Background = new SolidColorBrush(Color.FromRgb(59, 59, 59));
            }
        }

        // Update Fortschrittsanzeige
        ProgressTextBlock.Text = $"{currentCardIndex + 1} / {filteredCards.Count}";
    }
}
```

Abbildung 3-54: LearnDeck.xaml.cs Methode fürs anzeigen der aktuellen Karte

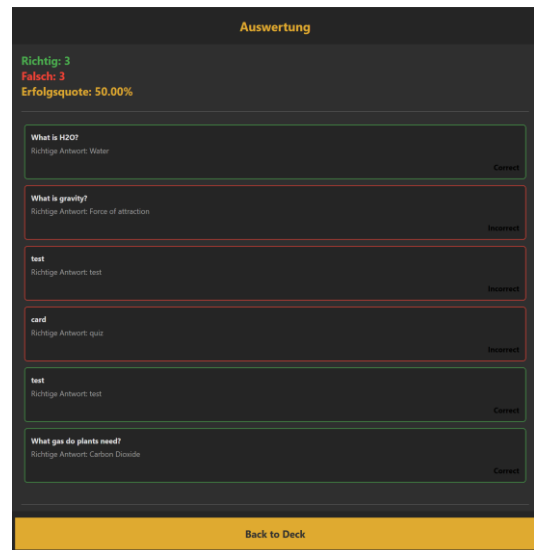


Abbildung 3-50: Auswertung UI

Wenn es Karten in der gefilterten Liste hat, dann wird LoadCurrentCard() ausgeführt. Diese Methode überprüft oben nochmals, ob filteredCards nicht leer ist. Wenn die Liste mind. einen Wert beinhaltet, dann wird der TextBlock 'QuestionTextBlock' mit der aktuellen Frage befüllt. Die aktuelle Frage wird in den

```
private int currentCardIndex = 0;
private List<bool> results = new List<bool>();
12 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
private Cards currentCard => filteredCards[currentCardIndex];
```

Abbildung 3-53: learnDeck.xaml.cs allgemeine Definitionen

allgemeinen Variablen aus der Liste filteredCards und dem currentCardIndex herausgelesen. Wenn der Eintrag der Liste mit dem aktuellen Index den Typ 'card' hat, dann wird das QuizOptionsPanel versteckt und das AnswerInputPanel wird sichtbar. So kann man aus dem Code behind einfach diese Trennung durchführen. Wenn der Typ = 'quiz' ist, dann wird es umgekehrt dargestellt oder versteckt. Hier müssen die Buttons zusätzlich noch mit den vier Auswahlmöglichkeiten befüllt werden. Danach werden alle Buttons grau gefüllt (=zurückgesetzt). Ganz zum Schluss wird currentCardIndex und filteredCards.count dargestellt, damit man eine Übersicht der Karten, während dem Lernen hat.

Die vier Option-Buttons führen schlussendlich alle dieselbe Funktion aus, nämlich QuizOptionButton_Click. Jeder Button gibt jedoch einen anderen Parameter mit. Dieser wird mit dem Schlüsselwort «Tag» gesetzt. Die Parameter sind die ganzen Zahlen zwischen 1 und 4.

```
<!-- Quiz-Optionen -->
<StackPanel x:Name="QuizOptionsPanel" Grid.Row="2" Orientation="Vertical" Margin="0,0,20">
  <Button Cursor="Hand" x:Name="Option1Button" Tag="1" Margin="0,5" Padding="10"
    Background="□" #3b3b3b" Foreground="■" #e0e0e0" FontSize="14"
    Click="QuizOptionButton_Click"/>
  <Button Cursor="Hand" x:Name="Option2Button" Tag="2" Margin="0,5" Padding="10"
    Background="□" #3b3b3b" Foreground="■" #e0e0e0" FontSize="14"
    Click="QuizOptionButton_Click"/>
  <Button Cursor="Hand" x:Name="Option3Button" Tag="3" Margin="0,5" Padding="10"
    Background="□" #3b3b3b" Foreground="■" #e0e0e0" FontSize="14"
    Click="QuizOptionButton_Click"/>
  <Button Cursor="Hand" x:Name="Option4Button" Tag="4" Margin="0,5" Padding="10"
    Background="□" #3b3b3b" Foreground="■" #e0e0e0" FontSize="14"
    Click="QuizOptionButton_Click"/>
</StackPanel>
```

Abbildung 3-55: LearnDeck.xaml QuizOptionButtons

Dieser Tag wird in der Funktion im Code behind in die Variable optionNumber gespeichert. Der angeklickte Button wird mit einer gelben Farbe markiert (233, 170, 48). Nach einer halben Sekunde wird die CheckAnswer()-Methode aufgerufen, mit der Tag-Nummer als Parameter. Beim Typ «card» wird direkt beim Drücken auf Submit diese Methode aufgerufen.

```
private void SubmitAnswerButton_Click(object sender, RoutedEventArgs e)
{
    if (currentCard.type == "card")
    {
        CheckAnswer(AnswerTextBox.Text);
    }
}
```

Abbildung 3-57: LearnDeck.xaml.cs SubmitAnswerButton geklickt

Hier wird die angegebene Antwort überprüft. Es wird auch hier zwischen den beiden Typen unterschieden. Bei einer normalen Karte wird der eingegebene Text von überflüssigen Spaces (Trim()) befreit und mit der originalen Antwort verglichen. Dies wird als Boolean in isCorrect festgehalten. Wenn der Typ ein quiz ist, dann wird der Tag, welcher momentan noch ein String ist zu einem Integer konvertiert und mit dem correct_answer-Int aus der Datenbank verglichen. Auch hier wird ein boolischer Wert in isCorrect gespeichert. Danach wird der Liste 'results' der Boolean angehängt. Wenn es nochmals eine Karte im Deck gibt, dann wird currentCardIndex + 1 gezählt und die nächste Karte von vorne wieder aufgerufen. Wenn jedoch keine Karte im Deck existiert, wird die Auswertung angezeigt. Hierfür wird ein neues Fenster geöffnet nämlich AuswertungLernen.xaml.

```
private void QuizOptionButton_Click(object sender, RoutedEventArgs e)
{
    var button = (Button)sender;
    var optionNumber = int.Parse((string)button.Tag);

    // Visuelles Feedback für die Auswahl
    foreach (var child in QuizOptionsPanel.Children)
    {
        if (child is Button btn)
        {
            btn.Background = new SolidColorBrush(Color.FromRgb(59, 59, 59));
        }
    }
    button.Background = new SolidColorBrush(Color.FromRgb(223, 170, 48));

    // Nach kurzer Verzögerung die Antwort überprüfen
    var timer = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(500) };
    timer.Tick += (s, args) =>
    {
        timer.Stop();
        CheckAnswer(optionNumber.ToString());
    };
    timer.Start();
}
```

Abbildung 3-58: LearnDeck.xaml.cs QuizOptionButton geklickt.

```
private void CheckAnswer(string userAnswer)
{
    bool isCorrect = false;

    if (currentCard.type == "card")
    {
        isCorrect = userAnswer.Trim().Equals(currentCard.answer, StringComparison.OrdinalIgnoreCase);
    }
    else if (currentCard.type == "quiz")
    {
        int selectedOption = int.Parse(userAnswer);
        isCorrect = selectedOption == currentCard.correct_answer;
    }

    results.Add(isCorrect);

    if (currentCardIndex < filteredCards.Count - 1)
    {
        currentCardIndex++;
        LoadCurrentCard();
    }
    else
    {
        ShowEvaluation();
    }
}
```

Abbildung 3-56: LearnDeck.xaml.cs CheckAnswer

3.2.9.2 AuswertungLernen.xaml und AuswertungLernen.xaml.cs

Hier wird von LearnDeck.xaml die Liste der results und die Liste der cards mitgegeben.

Zuerst werden verschiedene allgemeine Werte berechnet, wie die Anzahl richtiger/falscher Antworten und der Erfolgsquote in Prozent. Danach wird eine neue Liste erstellt, in welche

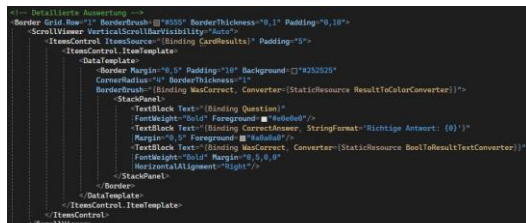


Abbildung 3-59: AuswertungLernen.xaml

```
CorrectCount = results.Count(r => r);
IncorrectCount = results.Count(r => !r);
Percentage = results.Count > 0 ? (CorrectCount / (double)results.Count) * 100 : 0;

CardResults = new List<CardResult>();
for (int i = 0; i < results.Count; i++)
{
    CardResults.Add(new CardResult
    {
        Question = cards[i].question,
        CorrectAnswer = GetCorrectAnswer(cards[i]),
        WasCorrect = results[i]
    });
}

DataContext = this;
```

Abbildung 3-60: AuswertungLernen.xaml.cs Statics Berechnung

die Frage, die richtige Antwort und ob es richtig beantwortet, wurde gespeichert wird. Aus dem xaml-Code werden die Daten aus CardResults genommen und so dargestellt.

3.2.10 Codeerläuterung Import & Export V2.5

Für die Import Funktion konnte ich einiges von addCards verwenden. Es ist hierdurch sehr ähnlich, da sowieso ein JSON gesendet wird, um Karten hinzuzufügen. Nun wird ein JSON-File ausgelesen und direkt gesendet. Einiges aufwendiger war der Import und Export für csv zu ermöglichen.

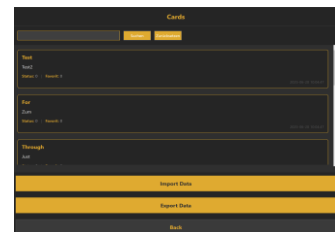


Abbildung 3-61: FileImpExp UI

3.2.10.1 FileImpExp.xaml.cs und FileImpExp.xaml

Import: importCards

Diese importCards-Methode wird aufgerufen, wenn man auf den 'Import Data'-Button drückt. Den FilePicker konnte ich aus der Microsoft-Library nehmen. Dies ist der Standard-Filepicker, den man von allen Applikationen kennt. Beim Filter wird definiert, welche Dateitypen erlaubt sind. Bei mir sind es .JSON und .CSV. Mit ShowDialog() wird das Fenster gezeigt. Danach wird überprüft, ob eine Datei ausgewählt wurde. Der Pfad wird gelesen und der Dateityp wird abgetrennt und in Extension gespeichert. Wenn Extension 'csv' lautet, wird die Methode ConvertCsvToJson aufgerufen, um das Format anzupassen. Wenn ein Json geöffnet wurde, sind keine weiteren Umwandlungen mehr nötig. Der Text wird aus dem File gelesen und als String gespeichert. Danach wird SendImportedCards aufgerufen, welches fast die gleiche Methode wie addCards ist. Beide senden ein JSON mit den Karteninhalten an den Server.

```
private async void importCards()
{
    // Configure open file dialog box
    var dialog = new Microsoft.Win32.OpenFileDialog
    {
        FileName = "cards",
        DefaultExt = ".json",
        Filter = "All supported files (*.json;*.csv)|*.json|JSON files (*.json)|*.json|CSV files (*.csv)|*.csv"
    };

    bool? result = dialog.ShowDialog();

    if (result == true)
    {
        string filePath = dialog.FileName;
        string extension = System.IO.Path.GetExtension(filePath)?.ToLower();

        try
        {
            string jsonContent = string.Empty;

            if (extension == ".csv")
            {
                jsonContent = CardConverter.ConvertCsvToJson(filePath);
            }

            if (extension == ".json")
            {
                string filename = dialog.FileName;
                jsonContent = File.ReadAllText(filename);
            }

            if (!string.IsNullOrEmpty(jsonContent))
            {
                await SendImportedCards(jsonContent);
                getCards();
            }
            else
            {
                MessageBox.Show("No valid cards found in the file.", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Error importing cards: {ex.Message}", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

Abbildung 3-62: fileImpExp.xaml.cs imprtCards-Methode

Import: ConvertCsvToJson

Das Csv ist so aufgebaut:

```
type;question;correctAnswerIndex;answer;options;fav
card;Was ist 2+2?;;4;;true
card;Was ist 2+2?;;4;;true
card;test;;card;;false
card;quiz;;card;;false
card;Was ist die Hauptstadt von Deutschland?;;Berlin;;false
card;Wie viele Planeten hat unser Sonnensystem?;;8;;false
card;Welches chemische Element hat das Symbol 'O'?;;Sauerstoff;;false
card;In welchem Jahr begann der Zweite Weltkrieg?;;1939;;false
```

Dieses Csv muss in JSON umgewandelt werden, damit man es einfach und mit derselben Methode an den Server senden kann. Zuerst werden die Linien der Datei gezählt und danach 2 neue Listen instanziiert. Für jede Linie im Csv wird beim ';' getrennt und die einzelnen Werte ausgelesen. Wenn der type='normalCards' ist, dann werden die relevanten Felder für eine Karte ausgelesen, und wenn der typ='quizCards' ist, dann werden die fürs Quiz relevanten Werte gelesen. Beim Quiz gibt es ein Array im Csv, welches durch ',' getrennt ist. dieses muss hier zuerst auch noch aufgeteilt werden. Fürs result werden diese beiden Arrays zusammengehängt und mit dem JsonSerializer zu einem JSON umgebaut und zurückgesendet.

```
public class CardConverter
{
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public static string ConvertCsvToJson(string csvPath)
    {
        var lines = File.ReadAllLines(csvPath);
        var normalCards = new List<NormalCard>();
        var quizCards = new List<QuizCard>();

        foreach (var line in lines)
        {
            if (string.IsNullOrEmpty(line)) continue;

            var parts = line.Split(';');
            if (parts.Length < 6) continue;

            string type = parts[0].Trim();
            string question = parts[1].Trim();
            string correctIndexStr = parts[2].Trim();
            string answer = parts[3].Trim();
            string options = parts[4].Trim();
            bool isFav = parts[5].Trim().ToLower() == "true";

            if (type == "normalCards")
            {
                normalCards.Add(new NormalCard
                {
                    question = question,
                    answer = answer,
                    is_fav = isFav,
                    status = "needs_practice"
                });
            }
            else if (type == "quizCards")
            {
                var opts = options.Split(',').Select(o => o.Trim()).ToArray();
                if (opts.Length < 4) continue; // ensure 4 options
                int correctIndex = int.TryParse(correctIndexStr, out var i) ? i : 0;

                quizCards.Add(new QuizCard
                {
                    question = question,
                    option1 = opts[0],
                    option2 = opts[1],
                    option3 = opts[2],
                    option4 = opts[3],
                    correctIndex = correctIndex,
                    is_fav = isFav,
                    status = "needs_practice"
                });
            }
        }

        var result = new
        {
            normalCards,
            quizCards
        };

        return JsonSerializer.Serialize(result, new JsonSerializerOptions { WriteIndented = true });
    }
}

2 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
public class NormalCard
{
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string question { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string answer { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public bool is_fav { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string status { get; set; }
}

2 Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
public class QuizCard
{
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string question { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string option1 { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string option2 { get; set; }
    1 Verweis | 0 Änderungen | 0 Autoren, 0 Änderungen
    public string option3 { get; set; }
```

Abbildung 3-63: ConvertCsvToJson()

Export: exportCards

Auf für den Export kann ich die Microsoft-Library verwenden. Auch hier erlaube ich .JSON und .csv Dateitypen. Es wird genau gleich wie oben zwischen den verschiedenen Dateiformaten unterschieden. Wenn ein csv ausgewählt wurde, dann wird wieder eine weitere Methode aufgerufen, welche ein csv erstellt. `filteredCards` ist eine Liste, welche alle angezeigten Karten eines Decks beinhaltet. Wenn man also im UI nach einer Karte sucht, ist in `filteredDecks` nur eine Karte und wenn dann noch exportiert wird, hat man ein JSON-File mit genau einer Karte drin, obwohl das Deck vielleicht 20 Karten total beinhaltet. Aus der Liste werden nun alle Karten, welche kein quiz sind und nur einzelne Attribute davon exportiert. Dasselbe wird dann auch noch für quiz gemacht. Das gib dann zwei listen, welche zusammengesetzt werden in `exportData`. Diese werden mit `SerializeObject` zu einem json-Format geformt.

```
private void exportCards()
{
    var dialog = new Microsoft.Win32.SaveFileDialog
    {
        FileName = $"cards_{deckId}",
        DefaultExt = ".json",
        Filter = "All supported files (*.json;*.csv)|*.json;*.csv|JSON files (*.json)|*.json|CSV files (*.csv)|*.csv"
    };

    bool? result = dialog.ShowDialog();

    if (result == true)
    {
        string filePath = dialog.FileName;
        string extension = System.IO.Path.GetExtension(filePath)?.ToLower();

        try
        {
            if (extension == ".csv")
            {
                string csvContent = ConvertCardsToCsv();
                File.WriteAllText(filePath, csvContent);
                MessageBox.Show("Cards exported successfully!", "Success", MessageBoxButton.OK, MessageBoxImage.Information);
            }
            else if (extension == ".json")
            {
                // Filter normal cards (type != "quiz")
                var normalCards = filteredCards
                    .Where(card => card.type != "quiz")
                    .Select(card => new NormalCardExport
                    {
                        question = card.question,
                        answer = card.answer,
                        is_fav = card.is_fav == 1,
                        status = card.status
                    })
                    .ToList();

                // Filter quiz cards (type == "quiz")
                var quizCards = filteredCards
                    .Where(card => card.type == "quiz")
                    .Select(card => new QuizCardExport
                    {
                        question = card.question,
                        option1 = card.first_option,
                        option2 = card.second_option,
                        option3 = card.third_option,
                        option4 = card.fourth_option,
                        correctIndex = card.correct_answer ?? 1,
                        is_fav = card.is_fav == 1,
                        status = card.status
                    })
                    .ToList();

                // Create the export data structure
                var exportData = new CardExportModel
                {
                    normalCards = normalCards,
                    quizCards = quizCards
                };

                string jsonContent = JsonConvert.SerializeObject(exportData, Formatting.Indented);
                File.WriteAllText(dialog.FileName, jsonContent);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Error exporting cards: {ex.Message}", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

Abbildung 3-64: FileImpExp.xaml.cs exportCards()

Export: ConvertCardsToCsv

Es wird ein `StringBuilder` verwendet, um effizient ein Json zu erstellen. Zu diesem Leeren String wird als erstes die header-Zeile hinzugefügt. Danach wird für jede zu exportierende Karte die wichtigen Werte aus der `filteredCards`-Liste gelesen. Wenn es sich um ein quiz handelt, dann müssen die Optionen noch separat getrennt werden. Dies wird mit der `EscapeCsvFiled`-Methode gemacht. Am Schluss wird der `StringBuilder` mit `.ToString()` zurückgegeben, als fertigen CSV-String.

```
private string ConvertCardsToCsv()
{
    var csvBuilder = new StringBuilder();
    csvBuilder.AppendLine($"type;question;correctAnswerIndex;answer;options;fav");

    foreach (var card in filteredCards)
    {
        string type = card.type;
        string question = EscapeCsvField(card.question ?? "");
        string correctAnswerIndex = card.correct_answer?.ToString() ?? "";
        string answer = EscapeCsvField(card.answer ?? "");
        string options = "";
        bool isFav = card.is_fav == 1;

        if (type == "quiz")
        {
            options = EscapeCsvField($"{card.first_option ?? ""},{card.second_option ?? ""},{card.third_option ?? ""},{card.fourth_option ?? ""}");
        }

        csvBuilder.AppendLine($"{type};{question};{correctAnswerIndex};{answer};{options};{isFav.ToString().ToLower()}");
    }

    return csvBuilder.ToString();
}

// Helper: EscapeCsvField: Escape CSV field
private string EscapeCsvField(string field)
{
    if (string.IsNullOrEmpty(field))
        return "";

    // Wenn das Feld ein Symbol, Name oder Anführungszeichen enthält, muss es in Anführungszeichen gesetzt werden
    if (field.Contains(";") || field.Contains(",") || field.Contains("\""))
    {
        // Anführungszeichen in Feld verdoppeln
        field = field.Replace("\"", "\"\"");
        return $"\"{field}\"";
    }

    return field;
}
```

Abbildung 3-65: FileImpExp.xaml.cs ConvertCardsToCsv & EscapeCsvFiled

3.2.11 Login/Registrierung V3.0

Dies sind die Benutzerverwaltungs Pages (Login und Registrierung). Es war eine Herausforderung, die Registrierung sicher durchzuführen, da normalerweise bei der Authentifizierung das Passwort aus der Datenbank ausgelesen wird und mit den anderen Komponenten zusammen gehashed wird. In diesem Fall besteht noch kein Eintrag in der Datenbank, deshalb ist dies so leider nicht möglich.

Die Lösung ist:

Der Hash wird normal berechnet, mit Token, Salt und Passwort. Als Content des http-Post's wird ein JSON mitgegeben, welches die benötigten Daten für die Datenbank beinhaltet. Dies sind username, E-Mail und password als Hash. Der Server generiert dann aus dem mitgegebenen Passwort-Hash, dem Salt und dem Sessiontoken auch einen Hash und vergleicht diesen mit dem in der URL als Parameter mitgegebenen Hash. In der Datenbank wird nun also ein Hash gespeichert und nicht das Passwort als Plain Text. Beim Login funktioniert die Authentifizierung ganz normal.

Im App.xaml.cs ist nun definiert, dass wenn man die App startet, wird Register.xaml aufgerufen.

Abbildung 3-66: Register & Login UI

3.2.11.1 Register.xaml.cs und Login.xaml.cs

Register: Button_click

Wenn man den Button «Registrieren» klickt, dann wird als erstes verglichen, ob die beiden Passwörter übereinstimmen. Wenn ja, wird es gehashed und die anderen Angaben in den Variablen gespeichert, welche oben definiert wurden. Danach wird die registerUser()-Funktion aufgerufen. Wenn die Passwörter jedoch nicht übereinstimmen, wird dies in einer MessageBox angezeigt.

```
private void RegisterButton_Click(object sender, RoutedEventArgs e)
{
    if (first_pw.Password == second_pw.Password)
    {
        _password = generateHash.GenerateSHA256Hash(first_pw.Password);
        _email = email.Text;
        _user = username.Text;
        registerUser();
    }
    else
    {
        MessageBox.Show("Die Passwörter stimmen nicht überein.", "Fehler", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Abbildung 3-67: Register.xaml.cs Buttonclick-Event

Register: registerUser

In der registerUser-Methode wird zuerst ein Token angefordert, welcher dann mit den anderen Komponenten gehashed wird. Danach wird ein json erstellt, welches username, E-Mail und password enthält, welches dann im http-Post mitgesendet wird. Die Methode SendNewUserAsync() wird aufgerufen, welche einen Normalen Post an die URL macht. Die Backend-Funktionen werden hier beschrieben.

```
private async void registerUser()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionId = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionId}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _password);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        string json = JsonConvert.SerializeObject(new
        {
            username = _user,
            email = _email,
            password = _password
        });

        await postData.SendNewUserAsync(_httpClient, json, hashedToken, sessionId);
    }
    catch (Exception ex)
    {
    }
}
```

Abbildung 3-68: Register.xaml.cs registerUser-Methode

Login: Button_Click

Beim Event vom Login-Button wird aus dem Passwort als erstes einen Hash erstellt, da dieser gleich sein sollte, wie in der Datenbank. Der username wird auch aus dem Textfeld gelesen und in der Variablen gespeichert.

```
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
    _password = generateHash.GenerateSHA256Hash(password.Password);
    _user = username.Text;
    verifyAccount();
}
```

Abbildung 3-69: Login.xaml.cs LoginButton_Click-Event

Login: verifyAccount()

Fürs Login wird dieselbe Abfrage verwendet, wie z.B. bei `getDecks`. Wenn die Rückgabe aus dem Backend positiv war, dann werden die Logindaten (username, password) in den `Properties.Settings.Default` gespeichert. Dies ist ein fixer Speicher, der von jedem Namespace aus der App erreichbar ist. Dort wird der Salt-Wert auch gespeichert. Wenn das Login erfolgreich war, wird das Index-Fenster geöffnet.

```
private async void verifyAccount()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionID = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionID}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _password);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await sendRequest.SendRequest(requestClient, "verifyAccount", _user, hashedToken, sessionID, "0");

            if (responseData is bool && (bool)responseData == true)
            {
                Properties.Settings.Default.username = _user;
                Properties.Settings.Default.password = _password;
                Properties.Settings.Default.Save();
                var indexWindow = new Index();
                indexWindow.Show();
                this.Close();
            }
            else
            {
                MessageBox.Show("Login fehlgeschlagen. Bitte überprüfen Sie Ihre Anmeldedaten.", "Fehler", MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

Abbildung 3-70: Login.xaml.cs verifyAccount()-Methode

3.2.11.2 databaseRequest.php

`$action==='createUser'`

Wenn die `action==='createUser'` ist, dann wird als erstes das mitgegebene JSON gelesen und in `$data` gespeichert. Dieses wird dann auseinandergenommen und die verschiedenen Variablen werden gespeichert. Der Hash wird danach hieraus berechnet. Somit wird der Session-Token und der Salt benötigt, damit man einen User erstellen kann. Diese beiden Hashes werden verglichen und dann wird die `createUser`-Funktion aufgerufen.

```
if ($action === 'createUser') {
    $data = json_decode(file_get_contents('php://input'), true);

    fwrite($fh, print_r($data, true));

    if (isset($data['username'], $data['email'], $data['password'])) {
        $user = $data['username'];
        $email = $data['email'];
        $password = $data['password'];
    } else {
        http_response_code(400);
    }

    $fullToken = $_SESSION['token'] . $baseCode . $password;
    $serverHash = hash('sha256', $fullToken);

    // Client-Hash mit Server-Hash vergleichen
    if ($clientToken !== $serverHash) {
        session_destroy();
        fwrite($fh, date('DATE_RFC2822') . " : Authentifizierung fehlgeschlagen : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
        echo json_encode(['error' => 'Authentifizierung fehlgeschlagen']);
        exit;
    }

    $result = createUser($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $email, $password, $fh);

    echo json_encode($result);

    // Session nach erfolgreicher Anfrage zerstören
    session_destroy();
}
```

Abbildung 3-71: databaseRequest.php `$action==='createUser'`

Funktion createUser()

In dieser Funktion wird mit einem INSERT INTO der neue user erstellt, mit den Werten, welche im JSON dem http-Post mitgegeben wurden. Ab sofort kann die Authentifizierung normal ablaufen. Zuerst wird jedoch noch überprüft, ob der user bereits vorhanden ist. Wenn alles geklappt hat, wird ein json-Array zurückgesendet, welches die Bestätigung 'success: true' angibt. Genauere Ausführungen zur Datenbankverbindung oder den Abfragen werden [hier](#) beschrieben.

```
function createUser($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $email, $password, $fh) {
    $dbh = createDatabaseConnection($mysql_host, $mysql_user, $mysql_password, $mysql_database, $fh);

    if (is_array($dbh)) {
        return $dbh; // Fehler bei der Datenbankverbindung
    }

    // Prüfen ob Benutzer bereits existiert
    $checkSql = "SELECT id FROM users WHERE username = ? OR email = ?";
    $checkStmt = mysqli_prepare($dbh, $checkSql);

    if (!$checkStmt) {
        mysqli_close($dbh);
        fwrite($fh, date(DATE_RFC2822) . " : SQL Prepare (Check) fehlgeschlagen\n");
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_bind_param($checkStmt, "ss", $user, $email);
    mysqli_stmt_execute($checkStmt);
    $checkResult = mysqli_stmt_get_result($checkStmt);

    if (mysqli_num_rows($checkResult) > 0) {
        mysqli_stmt_close($checkStmt);
        mysqli_close($dbh);
        return ['error' => 'Benutzername oder E-Mail bereits vergeben'];
    }
    mysqli_stmt_close($checkStmt);

    // Benutzer erstellen
    $sql = "INSERT INTO users (username, email, password) VALUES (?, ?, ?)";
    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        mysqli_close($dbh);
        fwrite($fh, date(DATE_RFC2822) . " : SQL Prepare fehlgeschlagen\n");
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_bind_param($stmt, "sss", $user, $email, $password);

    if (!mysqli_stmt_execute($stmt)) {
        mysqli_stmt_close($stmt);
        mysqli_close($dbh);
        fwrite($fh, date(DATE_RFC2822) . " : Fehler beim Erstellen des Benutzers: " . mysqli_stmt_error($stmt) . "\n");
        return ['error' => 'Fehler beim Erstellen des Benutzers'];
    }

    $userId = mysqli_insert_id($dbh);

    mysqli_stmt_close($stmt);
    mysqli_close($dbh);

    return [
        'success' => true,
        'userId' => $userId,
        'message' => 'Benutzer erfolgreich erstellt'
    ];
}
```

Abbildung 3-72: databaseRequest.php Funktion createUser()

Request=verifyAccount

Wenn der Parameter 'request' = verifyAccount ist, muss nicht viel gemacht werden. Die Hashes werden verglichen, und wenn diese übereinstimmen, wird als Rückgabe true gesendet. Dies reicht völlig aus als Rückgabe, da nur eine Bestätigung benötigt wird, ob das Passwort richtig ist, und ob der User existiert.

```
// Server-seitigen Hash generieren
$fullToken = $_SESSION['token'] . $baseCode . $password;
$serverHash = hash('sha256', $fullToken);

// Client-Hash mit Server-Hash vergleichen
if ($clientToken !== $serverHash) {
    session_destroy();
    fwrite($fh, date(DATE_RFC2822) . " : Authentifizierung fehlgeschlagen : " . $_SERVER['HTTP_CLIENT_IP'] . "\n");
    echo json_encode(['error' => 'Authentifizierung fehlgeschlagen']);
    exit;
}

switch ($requestMessage) {
    case 'verifyAccount':
        $result = true;
        break;
}
```

Abbildung 3-73: databaseRequest.php request=verifyAccount

3.2.11.3 App.xaml.cs

Dies sind alle Einstellungs-Variablen, welche ich definiert habe. Der salt ist eine Anwendungs-Variable. Dies heisst, dass sie readOnly ist. Diese kann man aus dem Code nicht bearbeiten. Die anderen sind dem Benutzerbereich zugewiesen. Diese werden während der Laufzeit des Codes bearbeitet.

Dies ist die erste Methode, welche beim Start der App aufgerufen wird. Hier werden die Systemvariablen, welche im Benutzerbereich definiert, sind abgefragt. Danach wird überprüft, ob sie einen Wert beinhalten. Wenn nicht, wird das Login-Fenster geöffnet. Wenn sie gespeichert wurden, wird überprüft, wie lange die Variablen bereits definiert sind. Wenn sie länger als ein Monat sind (30 Tage), dann werden sie auch gelöscht und man kommt auch zur Login-Page. Wenn dies jedoch auch nicht der Fall ist und alle Daten aktuell sind, wird direkt die Index-Seite geöffnet.

Name	Typ	Bereich	Wert
salt	string	Anwendung	4gdrsh92z7
username	string	Benutzer	
password	string	Benutzer	
LastLoginDate	System.DateTime	Benutzer	

Abbildung 3-75: Properties.Settings.default

```
protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);

    string username = FlashCards.Properties.Settings.Default.username;
    string password = FlashCards.Properties.Settings.Default.password;
    DateTime lastLoginDate = FlashCards.Properties.Settings.Default.LastLoginDate;

    Window startWindow;

    if (!string.IsNullOrEmpty(username) && !string.IsNullOrEmpty(password))
    {
        if (lastLoginDate != DateTime.MinValue && (DateTime.Now - lastLoginDate).TotalDays > 30)
        {
            FlashCards.Properties.Settings.Default.username = string.Empty;
            FlashCards.Properties.Settings.Default.password = string.Empty;
            FlashCards.Properties.Settings.Default.LastLoginDate = DateTime.MinValue;
            FlashCards.Properties.Settings.Default.Save();

            startWindow = new Login();
            MessageBox.Show("Ihre Anmeldedaten sind abgelaufen. Bitte melden Sie sich erneut an.",
                "Anmeldung");
        }
        else
        {
            // Benutzer ist bekannt und Anmeldezeitraum ist noch gültig → Login aufrufen
            startWindow = new Index();
        }
    }
    else
    {
        // Noch kein Benutzer vorhanden (oder Daten wurden gerade gelöscht) → Registrierung
        startWindow = new Login();
    }

    startWindow.Show();
}
```

Abbildung 3-74: App.xaml.cs OnStartup-Methode

3.2.12 Karte als Favorit markieren

Bei Jeder Karte ist nun ein Stern-Icon Vorhanden, welches ein Button ist. Wenn der Stern gelb ist, heisst dies, dass die Karte als favorit markiert wurde. Wenn er weiss ist, wurde sie nicht markiert. Momentan kann man noch nichts mit diesen Favoriten anfangen, es wäre jedoch toll, wenn man die Favoriten Einzel lernen könnte. Dies werde ich aus zeitlichen Gründen jedoch wahrscheinlich nicht fertig erledigen können. Die Möglichkeit die Karten zu markieren ist bei der Deck-Übersicht und bei der Import-/Export-Seite vorhanden.

3.2.12.1 Deck.xaml.cs und sendRequest.cs

FavoriteButton_Click

```
private async void FavoriteButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        Button button = (Button)sender;
        int cardId = (int)button.Tag;

        var card = allCards.FirstOrDefault(c => c.id == cardId);
        if (card != null)
        {
            card.is_fav = card.is_fav == 1 ? 0 : 1;

            if (this.DataContext == null)
            {
                this.DataContext = filteredCards;
            }

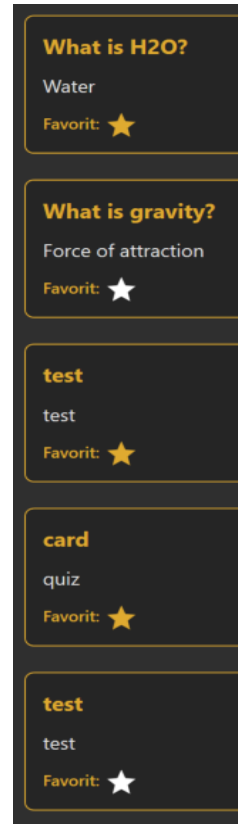
            await createTokenHash();

            using (HttpClient requestClient = new HttpClient())
            {
                var responseData = await sendRequest.SendRequest(requestClient,
                    "updateCardFavorite", _user, hashedToken, sessionID,
                    _deckId.ToString(), cardId.ToString(), card.is_fav.ToString(),
                    card.type.ToString());

                Console.WriteLine(responseData.message);
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error updating favorite: {ex.Message}");
    }
}
```

Abbildung 3-76: Deck.xaml.cs FavoriteButton_Click-Event

Beim Button_click (click auf das Stern-Icon) wird die Karten Id mitgegeben, welche als erstes überprüft wird, ob sie überhaupt vorhanden ist. Wenn sie vorhanden ist, dann wird der Wert geändert (ändern zwischen 1 & 0). Danach wird wie gewohnt einen Token angefordert, der gehashed wird. Da ich in dieser Klasse in zwei



Bsp. für Fav-Markierung

Methoden die Tokens benötige, habe ich diese verallgemeinert in der createTokenHash()-Methode. Es wird ein SendRequest aufgerufen, welcher sehr viele Parameter benötigt, wie der fav-Wert, der card-Type oder die cardId. In sendRequest wird diese ganze anfrage bearbeitet und ans Backend gesendet.

sendRequest

sendRequest ist eine ganz normale http-Abfrage, welche diese vielen Parameter mit gibt. Die Verarbeitung im Backend mit der Datenbank ist sehr ähnlich wie createUser, einfach wird kein SQL-Insert, sondern ein UPDATE verwendet.

```
internal static async Task<dynamic> SendRequest(HttpClient client, string request, string user, string token, string sessionID, string deckId, string cardId, string is_fav, string type)
{
    var response = await client.GetStringAsync(
        $"{baseUrl}?action=getData&request={request}&user={user}&token={token}&sessionID={sessionID}&deckId={deckId}&cardId={cardId}&isFav={is_fav}&type={type}");
    return JsonConvert.DeserializeObject(response);
}
```

Abbildung 3-77: sendRequest.cs SendRequest-Methode

3.2.13 Bearbeiten vom Konto

Dies ist die Overview Seite des eigenen Kontos. Der Username und die E-Mail werden aus der Datenbank abgefragt und hier dargestellt. Es gibt hier drei mögliche Aktionen: die persönlichen Angaben anpassen (Username, E-Mail, Passwort), den Account löschen oder sich abzumelden. Um die persönlichen Angaben zu ändern kann man einfach die Textfelder ausfüllen/anpassen und Save changes klicken. Wenn alles erfolgreich ablief, wird man abgemeldet und kann sich mit den neuen Details anmelden.

3.2.13.1 Account.xaml und Account.xaml.cs

Die Methode `getUserData()` fragt im Backend die aktuellen Daten für den aktuell angemeldeten User ab. Diese Information werden in einem Objekt einer weiteren Klasse 'UserData' gespeichert und via Bindung im xaml angezeigt. Das Ganze funktioniert sehr ähnlich wie das Anzeigen der Decks oder Karten.

Save changes Button

Wenn man etwas ändert und 'Save changes' drückt, wird `SavePassword_Click`-Methode ausgeführt. Hier war es sehr kompliziert und wichtig, dass die Variablen aussagekräftig benannt wurden. Hier wird zuerst das bisherige Passwort, welches der User zur Bestätigung eingeben muss gehashed und dann mit dem in den `Properties.Settings...` gespeicherten Passwort verglichen und die beiden neuen, um zu schauen, dass sie die selben sind. Wenn dies stimmte, wird die E-Mail-Adresse noch validiert und wenn diese auch gültig ist, wird sie auch gespeichert. Danach wird die `updateUser()`-Methode aufgerufen.

Abbildung 3-78: UI von der Account Overview Seite

```
private void SavePassword_Click(object sender, RoutedEventArgs e)
{
    string emailPattern = @"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$";
    string hashedOldPW = generateHash.GenerateSHA256Hash>PasswordBoxOld.Password);

    if (hashedOldPW == _OldPassword && PasswordBoxNew1.Password ==
        PasswordBoxNew2.Password)
    {
        _password = generateHash.GenerateSHA256Hash>PasswordBoxNew1.Password);
    }
    else
    {
        MessageBox.Show("Die Passwörter stimmen nicht überein oder das alte Passwort wurde falsch eingegeben.", "Fehler", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }

    if (Regex.IsMatch(email.Text, emailPattern))
    {
        _email = email.Text;
    }
    else
    {
        MessageBox.Show("Die E-Mail ist ungültig", "Fehler", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }

    _user = username.Text;
    registerUser();
}
```

Abbildung 3-79: Account.xaml.cs Save-Button-Event

updateUser()

Die updateUser-Methode ist sehr ähnlich, wie registerUser. Sie unterscheiden sich einfach in der Requestanforderung und dem JSON, welches mitgesendet wird. In diesem File gibt es viele Eigenschaften doppelt. Zum Beispiel wird der alte Benutzer und das alte Passwort für die Authentifizierung benötigt. Also habe ich nun `_OldUser`, `_OldPassword`, `_password` und `_user`. Dies führte bei mir schnell zu einem Durcheinander und ich musste lange nach einem Fehler suchen und das Problem war, dass ich das falsche Passwort gehasht habe und somit konnte ich keine Interaktionen mit der Datenbank durchführen. Ich erstelle also mit dem `_OldPassword` einen Hash, denn der Server erstellt den Hash mit dem aus der Datenbank, und dieses entspricht momentan noch dem `_OldPassword`. Der `_OldUser` muss mitgesendet werden, damit der Server weiss, welcher Datensatz geändert werden muss. Die `UpdateUserAsync`-Methode ist sozusagen dasselbe, wie die SendNewUserAsync-Methode. Der Rückgabewert wird dann überprüft und wenn es sich um eine Success-Meldung handelt, kommt ein Bestätigungs-Fenster und das Login-Fenster wird geöffnet und alle Properties-Einstellungen werden gelöscht. Wenn jedoch ein Error zurückkommt, kommt eine Fehlermeldung und nichts weiteres passiert.

```
private string token;
private string sessionId;
private string hashedToken;
private string _salt = Properties.Settings.Default.salt;
private string _oldUser = Properties.Settings.Default.username;
private string _oldPassword = Properties.Settings.Default.password;
private string _password;
private string _user;

private readonly HttpClient _httpClient;
```

Abbildung 3-81: Account.xaml.cs Eigenschaften

```
private async void updateUser()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionId = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionId}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _oldPassword);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        string json = JsonConvert.SerializeObject(new
        {
            username = _user,
            email = _email,
            password = _password,
            OldPassword = _oldPassword,
            OldUser = _oldUser
        });

        string response = await postData.UpdateUserAsync(_httpClient, json, hashedToken, sessionId);
        var result = Newtonsoft.Json.JsonConvert.DeserializeObject<dynamic>(response);

        if (result.success == true)
        {
            System.Windows.MessageBox.Show("Updated successfully!", "Erfolg", System.Windows.MessageBoxButton.OK, System.Windows.MessageBoxImage.Information);

            Properties.Settings.Default.Reset();
            Properties.Settings.Default.Save();
            var loginWindow = new Login();
            loginWindow.Show();
            this.Close();
        }
        else
        {
            string errorMessage = result.error ?? "Unknown error occurred";
            System.Windows.MessageBox.Show($"Update failed: {errorMessage}", "Error", System.Windows.MessageBoxButton.OK, System.Windows.MessageBoxImage.Error);
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
    Console.WriteLine($"Error: {ex.Message}");
}
```

Abbildung 3-80: Account.xaml.cs updateUser-Methode

Im Grunde funktioniert die updateUser-Funktion im `databaseRequest.php` ebenfalls sehr ähnlich, wie createUser, deshalb gehe ich hier nicht weiter darauf ein.

3.2.14 Mitarbeiter für ein Deck einladen

Es war schwierig eine richtige SQL-Abfrage zu schreiben, welche den gewünschten Zweck erfüllt. Beim Erstellen von einem Deck gibt es einen Dropdown-Button. Wenn man darauf clickt, werden verschiedene Benutzernamen angezeigt, welche man hinzufügen kann. Man kann jedoch nicht jeden Benutzer hinzufügen, sondern nur konnten, welche einem Folgen. Ein Beispiel:

User1 folgt User2

User2 folgt User1, User3, User4

User3 folgt User4

Das heisst, dass bei User1 nur User2 zur Auswahl steht, weil User2 ihm folgt. Bei User2 wird nur User1 angezeigt, User3 kann User2 einladen Und User4 kann User3 und User2 einladen.

Dies vermeidet die ungewollte Hinzufügung von Konten. Jeder Benutzer kann selbst entscheiden, wem er folgen möchte und somit auch wer ihn als Mitarbeiter hinzufügen kann.

3.2.14.1 createDeck.xam.cs, createDeck.xaml und databaseRequest.php v3.2

xaml UI ComboBox

Eine ComboBox wird für ein DropDown-Button verwendet. In App.xaml definiere ich einen allgemeinen style, welchen ich für alle ComboBoxen verwenden kann. Die ItemSource sind alle diese Daten, welche dargestellt werden in der Auswahl. Das SelectedItem ist der ausgewählte Wert.

```
<ComboBox Style="{StaticResource YellowComboBoxStyle}"
  ItemsSource="{Binding BenutzerNamenListe}"
  SelectedItem="{Binding SelectedBenutzer, Mode=TwoWay}" />
```

Abbildung 3-82: createDeck.xaml DropDown-Button

Benutzer abfragen

Als erstes, wenn die Seite neu aufgerufen wird, wird getUsers() ausgeführt. Dies ist eine ganz normale http-get Funktion, welche die Benutzer aus dem Backend abfragt. Als Rückgabewert kommt ein JSON, welches alle gefilterten Benutzernamen beinhaltet. Diese werden in in einem neuen Listen-Objekt allUsers gespeichert. Diese Liste wird der BenutzerNamenListe angehängt, welche einen Setter und Getter beinhalten. Aus dem UI werden die Werte für die Auswahl dann aus der BenutzerNamenListe genommen. Der OnPropertyChanged-Event aktualisiert das UI mit den aktuellen Daten, wenn der SelectedBenutzer oder BenutzerNamenListe geändert wurde. In SelectedBenutzer wird der Wert gespeichert, welcher im Dropdown ausgewählt wurde.

```
private async void getUsers()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionId = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionId}");

            hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _password);
            Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

            using (HttpClient requestClient = new HttpClient())
            {
                var responseData = await sendRequest.SendRequest(requestClient, "getUsersCollaborator", _user, hashedToken, sessionId, "0");
                List<UserCollaborator> allUsers = JsonConvert.DeserializeObject<List<UserCollaborator>>(responseData.ToString());
                BenutzerNamenListe = allUsers.Select(u => u.username).ToList();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

Abbildung 3-84: createDeck.xaml.cs getUsers()

```
private List<string> _benutzerNamenListe;
2 Verweise | Jan Bretscher, Vor 7 Stunden | 1 Autor, 1 Änderung
public List<string> BenutzerNamenListe
{
    get => _benutzerNamenListe;
    set
    {
        _benutzerNamenListe = value;
        OnPropertyChanged(nameof(BenutzerNamenListe));
    }
}

private string _selectedBenutzer;
2 Verweise | Jan Bretscher, Vor 7 Stunden | 1 Autor, 1 Änderung
public string SelectedBenutzer
{
    get => _selectedBenutzer;
    set
    {
        _selectedBenutzer = value;
        OnPropertyChanged(nameof(SelectedBenutzer));
    }
}

public event PropertyChangedEventHandler PropertyChanged;
```

Abbildung 3-83: createDeck.xaml.cs getter & setter

Get Users in Backend (php)

Die getUsers-Funktion wird für mehrere verschiedene Abfragen verwendet. Bei der Following funktionalität müssen auch alle Benutzer angezeigt werden, jedoch nach anderen Kriterien. Für die Benutzer müssen wie oben beschrieben alle Benutzer angezeigt werden, welche einem folgen.

```
function getUsers($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $type, $fh) {
    $dbh = createDatabaseConnection($mysql_host, $mysql_user, $mysql_password, $mysql_database, $fh);

    if ($type == 'collaborator') {
        $sql = "SELECT u.username
                FROM users u
                JOIN follows f ON u.id = f.follower_id
                WHERE f.followed_id = (SELECT id FROM users WHERE username = ?) AND username != ?";

        $stmt = mysqli_prepare($dbh, $sql);

        if (!$stmt) {
            mysqli_close($dbh);
            return ['error' => 'SQL Prepare fehlgeschlagen'];
        }

        mysqli_stmt_bind_param($stmt, "ss", $user, $user);
    } else if ($type == 'follow') {
        $sql = "SELECT u.username
                FROM users u
                WHERE u.id != (SELECT id FROM users WHERE username = ?)
                AND NOT EXISTS (
                    SELECT 1
                    FROM follows f
                    WHERE f.follower_id = (SELECT id FROM users WHERE username = ?)
                    AND f.followed_id = u.id
                )";

        $stmt = mysqli_prepare($dbh, $sql);

        if (!$stmt) {
            mysqli_close($dbh);
            return ['error' => 'SQL Prepare fehlgeschlagen'];
        }

        mysqli_stmt_bind_param($stmt, "ss", $user, $user);
    }

    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);

    if (!$result) {
        mysqli_stmt_close($stmt);
        mysqli_close($dbh);
        return ['error' => 'Fehler bei der Abfrage'];
    }

    $users = [];
    while ($row = mysqli_fetch_assoc($result)) {
        $users[] = $row;
    }

    mysqli_stmt_close($stmt);
    mysqli_close($dbh);
}
```

Abbildung 3-85: databaseRequest.php getUsers-Funktion

Neues Deck mit Collaborator speichern

Für die Speicherung des Mitarbeiters war keine grosse Änderung mehr nötig. Der Action `addDeck` wird

```
var responseData = await addDeck.AddDeck(requestClient, "addDeck", _user, hashedToken, sessionId,
    removeHashtagColor.RemoveHashtagColor(FirstColor.ToString()), removeHashtagColor.RemoveHashtagColor(SecondColor.ToString()), title.Text, alt.Text, SelectedBenutzer);
```

Abbildung 3-87: createDeck.xaml.cs addDeck mit Mitarbeiter

```
if ($stmt2 && mysqli_stmt_bind_param($stmt2, "iss", $deckId, $startColor, $endColor) && mysqli_stmt_execute($stmt2)) {
    mysqli_stmt_close($stmt2);

    if ($collaborator != null) {
        $sql3 = "INSERT INTO collaborators (deck_id, user_id) VALUES (?, (SELECT id FROM users WHERE username = ?))";
        $stmt3 = mysqli_prepare($dbh, $sql3);

        if ($stmt3 && mysqli_stmt_bind_param($stmt3, "is", $deckId, $collaborator) && mysqli_stmt_execute($stmt3)) {
            mysqli_stmt_close($stmt3);
        } else {
            $error = 'Fehler beim Hinzufügen von Mitarbeitern: ' . ($stmt3 ? mysqli_stmt_error($stmt3) : mysqli_error($dbh));
        }
    }
} else {
    $error = 'Fehler beim Hinzufügen der Deck-Farben: ' . ($stmt2 ? mysqli_stmt_error($stmt2) : mysqli_error($dbh));
}
```

Abbildung 3-86: databaseRequest.php collaborator erstellen

nun der ausgewählte Benutzer mitgeschickt. Dieser wird im Backend empfangen und als dritte Abfrage durchgeführt. Es muss geprüft werden, ob ein Username als Parameter mitgegeben wurde. Dies ist eine Freiwillige Funktion, welche nicht notwendig ist. Wenn ein Benutzername mitgesendet wurde wird in der Tabelle Collaborators einen neuen Eintrag erstellt.

3.2.15 Konten Folgen

Die Follow-Funktion hat einige Ähnlichkeiten zur Collaborator Funktionalität. Im Backend ist alles fast dieselbe Funktion mit anderen SQL-Abfragen. Das Ganze mit dem Folgen von anderen Konten ist in der Account Overview verwaltet. Hier hat es ein Dropdown, wo man einen Account auswählen kann und diesem dann auch folgen. Bei diesem Dropdown werden einem alle registrierten Benutzer angezeigt, ausser die, welchen man bereits folgt. Jeder Benutzer kann jedem folgen. Unten wird angezeigt, wer diesem Account folgt, und wem man bereits folgt. Somit ist die Following-Administration fast komplett. Nun fehlt nur noch das Löschen von Beziehungen. Dies passte leider nicht mehr in den zeitlichen Rahmen dieses Projektes.

Abbildung 3-88: Account Overview UI

3.2.15.1 Account.xaml.cs und Account.xaml v3.3

Follower-Daten abfragen

```

getNotFollowing();
getFollowers();
getFollowing();
}

2. Verweise | 0 Änderungen | 0 Autoren, 0 Änderungen
private async void getNotFollowing()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionID = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionID}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _oldPassword);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await sendRequest.SendRequest(requestClient, "getUsersNotFollowing", _oldUser, hashedToken, sessionID, "0");

            List<UserFollow> allUsers = JsonConvert.DeserializeObject<List<UserFollow>>(responseData.ToString());
            BenutzerNamenListe = allUsers.Select(u => u.username).ToList();
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}

```

Abbildung 3-89: Account.xaml.cs getNotFollowing-Methode (allgemeines Bsp.)

Zuoberst werden alle Methoden aufgerufen, welche benötigt werden für das Anzeigen der Follow-Beziehungen. Alle funktionieren im Prinzip gleich. Sie unterscheiden sich nur bei den Anweisungen. Hier ist die Anfrage für 'getUsersNotFollowing'. Diese Daten werden benötigt für das Dropdown. Dort sollten nur diese User angezeigt werden, welchen der aktuelle noch nicht folgt.

Die anderen Methoden fragen die Benutzernamen der Follower und von denen, welche vom aktiven Benutzer gefolgt werden, ab.

Follow Aktion

Beim Drücken auf den Folgenbutton wird die addFollow()-Methode aufgerufen, welche wiederum einen http-get ausführt. Als Parameter werden AddFollow, der aktuelle User und der ausgewählte User aus dem Dropdown mitgegeben. Nachdem dieser Request abgelaufen wird, werden die Daten neu geladen, damit alles aktualisiert ist.

```

private async void addFollow()
{
    try
    {
        using (HttpClient tokenClient = new HttpClient())
        {
            var responseToken = await getToken.GetTokenAsync(tokenClient);
            token = responseToken.token;
            sessionID = responseToken.sessionID;
            Console.WriteLine($"Token: {token} \nSessionId: {sessionID}");
        }

        hashedToken = generateHash.GenerateSHA256Hash(token, _salt, _oldPassword);
        Console.WriteLine($"Hashed Token + baseCode + password: {hashedToken}");

        using (HttpClient requestClient = new HttpClient())
        {
            var responseData = await sendRequest.AddFollow(requestClient, "addFollow", _oldUser, hashedToken, sessionID, SelectedBenutzer);

            getNotFollowing();
            getFollowers();
            getFollowing();
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}

```

Abbildung 3-90: Account.xaml.cs addFollow

3.2.15.2 databaseRequest.php v3.2

Die verschiedenen Datenbankabfragen, wie oben beschrieben, werden so organisiert. Je nach mitgegebener Abfrage wird ein anderes Schlüsselwort der aufgerufenen Funktion mitgegeben. Je nachdem, ob followers, following oder nicht following User abgefragt werden müssen. Die Funktion ist für alles jedoch dieselbe.

```
case 'getUsersFollowers':
    $result = getUsers($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, 'followers', $fh);
    break;

case 'getUsersFollowing':
    $result = getUsers($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, 'following', $fh);
    break;

case 'getUsersNotFollowing':
    $result = getUsers($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, 'notFollowing', $fh);
    break;
```

Abbildung 3-91: databaseRequest.php administration von Follower DB-Abfragen

Hier war es sehr anspruchsvoll, die richtige SQL-Query zu schreiben, sodass das richtige Ergebnis rauskommt. Zuerst gab es für mich einige Logische Probleme. Es gibt die Spalten follower_id und followed_id. Es war dem entsprechend sehr anspruchsvoll herauszufinden, wie ich abfragen muss, damit das richtige rauskommt.

```
if ($type == 'followers') {
    $sql = "SELECT u.username
            FROM users u
            JOIN follows f ON u.id = f.follower_id
            WHERE f.followed_id = (SELECT id FROM users WHERE username = ?) AND username != ?";

    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        mysqli_close($dbh);
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_bind_param($stmt, "ss", $user, $user);
} else if ($type == 'notFollowing') {

    $sql = "SELECT u.username
            FROM users u
            LEFT JOIN follows f ON u.id = f.followed_id AND f.follower_id = (SELECT id FROM users WHERE username = ?)
            WHERE f.followed_id IS NULL
            AND u.username != ?";

    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        mysqli_close($dbh);
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_bind_param($stmt, "ss", $user, $user);
} else if ($type == 'following') {

    $sql = "SELECT u.username
            FROM users u
            JOIN follows f ON u.id = f.followed_id
            WHERE f.follower_id = (SELECT id FROM users WHERE username = ?) AND username != ?";

    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        mysqli_close($dbh);
        return ['error' => 'SQL Prepare fehlgeschlagen'];
    }

    mysqli_stmt_bind_param($stmt, "ss", $user, $user);
}
```

Abbildung 3-92: databaseRequest.php follower-Abfragen

Als Parameter werden hier der Username des aktuellen Benutzers und der des 'zu Folgenden' Benutzers mitgegeben. Diese Daten werden dann in die passende Tabelle geschrieben.

```

////////////////////////////////////
// function addFollow
////////////////////////////////////

function addFollow($mysql_host, $mysql_user, $mysql_password, $mysql_database, $user, $follow, $fh) {
    // Datenbankverbindung herstellen
    $dbh = createDatabaseConnection($mysql_host, $mysql_user, $mysql_password, $mysql_database, $fh);

    if (!$dbh) {
        return ['error' => 'Fehler beim Herstellen der Datenbankverbindung'];
    }

    // Validierung der Eingabeparameter
    if (empty($user) || empty($follow)) {
        mysqli_close($dbh);
        return ['error' => 'Benutzername oder Follower-ID fehlt'];
    }

    // SQL-Abfrage vorbereiten
    $sql = "INSERT INTO follows (follower_id, followed_id) VALUES ((SELECT id FROM users WHERE username = ?), (SELECT id FROM users WHERE username = ?))";
    $stmt = mysqli_prepare($dbh, $sql);

    if (!$stmt) {
        $error = 'Fehler beim Vorbereiten der Abfrage: ' . mysqli_error($dbh);
        mysqli_close($dbh);
        return ['error' => $error];
    }

    // Parameter binden und ausführen
    if (!mysqli_stmt_bind_param($stmt, "ss", $user, $follow) || !mysqli_stmt_execute($stmt)) {
        $error = 'Fehler beim Ausführen der Abfrage: ' . mysqli_stmt_error($stmt);
        mysqli_stmt_close($stmt);
        mysqli_close($dbh);
        return ['error' => $error];
    }

    // ID des eingefügten Datensatzes abrufen
    $followId = mysqli_insert_id($dbh);

    // Statement und Verbindung schließen
    mysqli_stmt_close($stmt);
    mysqli_close($dbh);

    return ['success' => true, 'followId' => $followId];
}

```

Abbildung 3-93: databaseRequest.php addFollow-Funktion

3.3 Testplan/Testfälle mit Ergebnissen

TEST-ID	TESTFALL	TESTSCHRITTE	ERWARTETES ERGEBNIS	TATSÄCHLICHES ERGEBNIS	STATUS
T01	Benutzerregistrierung	1. Register.xaml öffnen 2. Gültige Daten eingeben 3. "Registrieren" klicken	Neuer Benutzer in Datenbank, Weiterleitung zu Login	Erfolgreiche Registrierung	✓
T02	Login mit korrekten Daten	1. Login.xaml öffnen 2. Registrierte Credentials eingeben 3. "Login" klicken	Zugriff auf Index.xaml mit persönlichen Decks	Erfolgreicher Login	✓
T03	Deck erstellen	1. Angemeldet sein 2. "Create Deck" wählen 3. Titel/Farben eingeben, speichern	Neues Deck in DB mit Creator-ID, Anzeige in Index	Deck wird korrekt erstellt	✓
T04	Karte hinzufügen	1. Deck öffnen 2. "Add Card" wählen 3. Frage/Antwort eingeben, speichern	Neue Karte im ausgewählten Deck sichtbar	Karte wird persistiert	✓
T05	Quiz-Karte hinzufügen	1. Deck öffnen 2. "Add Quiz" wählen 3. Frage/4 Optionen/Korrekte Antwort eingeben	Quiz mit Optionen im Deck	Quiz wird mit Optionen gespeichert	✓
T06	Deck löschen	1. Rechtsklick auf Deck 2. "Delete" wählen 3. Bestätigen	Deck und zugehörige Karten aus DB entfernt	Kaskadierendes Löschen funktioniert	✓
T07	Karten lernen (Quiz-Modus)	1. Deck mit Quiz-Karten öffnen 2. "Learn" wählen 3. Antwort auswählen	Sofortiges Feedback (richtig/falsch), Statistik	Korrekte Auswertung	✓
T08	Import von Karten (JSON)	1. "Import" wählen 2. JSON-Datei auswählen 3. Ziel-Deck bestätigen	Karten erscheinen im ausgewählten Deck	Daten werden geparkt und importiert	✓
T09	Export als CSV	1. Deck öffnen 2. "Export" wählen 3. CSV-Format und Speicherort wählen	CSV-Datei mit Deck-Inhalten wird erstellt	Formatierung korrekt	✓
T10	Favoriten markieren	1. Karte öffnen 2. Stern-Icon anklicken	Gelber Stern, is_fav=1 in Datenbank	Status wird persistent gespeichert	✓
T11	Mitarbeiter zu Deck einladen	1. Deck erstellen 2. Follower aus Dropdown wählen 3. Hinzufügen	User erscheint in collaborators-Tabelle	Kollaboration möglich	✓

3.4 Funktionsbeschreibung

1. Benutzerverwaltung

- **Registrierung & Login:**
 - Benutzer können ein persönliches Konto anlegen und sich mit Benutzername/Passwort anmelden.
 - Die Anmeldedaten werden sicher (SHA-256 gehasht) in der Datenbank gespeichert.
- **Konto-Einstellungen:**
 - Passwort- und E-Mail-Änderung möglich.
 - Option zum Löschen des Kontos.

2. Lernkarten erstellen & verwalten

- **Decks anlegen:**
 - Benutzer erstellen thematische Stapel (z. B. "Mathe-Formeln" oder "Englisch-Vokabeln").
 - Jedes Deck hat einen Titel, eine Beschreibung und einen Farbverlauf zur besseren Unterscheidung.
- **Karten hinzufügen:**
 - **Frage-Antwort-Karten:** Klassische Lernkarten mit einer Frage und einer Antwort.
 - **Quiz-Karten:** Multiple-Choice-Fragen mit vier Antwortmöglichkeiten.
- **Karten bearbeiten & löschen:**
 - Bestehende Karten können nachträglich angepasst oder entfernt werden.

3. Lernen & Abfragen

- **Lernmodus:**
 - Karten werden nacheinander angezeigt.
 - Bei Quiz-Karten wird die richtige Lösung nach der Auswahl angezeigt.
- **Statistik & Erfolgskontrolle:**
 - Nach jeder Lernsession wird eine Auswertung mit richtigen/falschen Antworten angezeigt.
 - Karten können als "gelernt" oder "wiederholen" markiert werden.

4. Import & Export

- **Daten austauschen:**
 - Karten und Decks können als **JSON oder CSV** exportiert werden.
 - Bestehende Lernmaterialien können im gleichen Format importiert werden.
- **Kompatibilität:**
 - Die Export-Funktion ermöglicht die Nutzung der Karten in anderen Apps oder die Sicherung der Daten.

5. Collaborator & Follow Funktion

- **Mitarbeiter einladen:**
 - Benutzer können andere Nutzer als "Mitarbeiter" zu ihren Decks hinzufügen, um gemeinsam Inhalte zu bearbeiten.
- **Follower-System:**
 - Nutzer können anderen folgen, um deren öffentliche Decks zu sehen.

3.5 Mögliche Erweiterungen

Aus zeitlichen Gründen konnte ich leider nicht alle Funktionen umsetzen. Während dem Programmieren kamen mir immer noch weitere Ideen in den Sinn, was auch noch praktisch und hilfreich für die Userexperience wäre. Ich musste mir dann jedoch immer sagen, dass ich mich mehr oder weniger an meine Planung halten muss, damit ich rechtzeitig mit allen Zielen fertig werde. Ich finde es jedoch immer schwierig und schade, wenn man etwas aus zeitlichen Gründen limitieren muss. Hier eine kleine Liste mit möglichen Erweiterungen, die vielleicht noch nützlich sein könnten:

- **Unfollow-Button** → wenn ich als Benutzer jemandem folge, sollte es auch möglich sein, dass ich wieder entfolge.
- **Share Deck** → momentan kann man ein Deck insofern teilen mit jemandem, dass jemand zum Mitarbeiter wird. Es wäre toll, wenn man das Deck ohne Schreibrechte mit jemandem teilen kann.
- **Edit Decks** → Decks sollte nach dem Erstellen immer noch bearbeitet werden können, wie etwa neue Mitarbeiter hinzufügen, Farbe, Titel, Beschreibung ändern.
- **Multiple collaborators** → die Möglichkeit sollte bestehen, dass man mehr als nur einen Mitarbeiter pro Deck hinzufügen kann.
- **Profile Icon** → es wäre toll, wenn man ein Profilbild hinzufügen könnte.
- **Learn Favorites** → die Karten, welche als Favorit markiert wurde, sollte man einzeln lernen können.
- **Save Statics** → die Statistik könnte in der Datenbank gespeichert werden, damit man Diagramme erstellen kann.
- **Mobile/Web** → eine Mobile- oder Web-Applikation, welche mit denselben Benutzern arbeitet.

3.6 Fazit

Ich bin ziemlich zufrieden mit dem Endprodukt. Ich konnte alle meine geforderten Issues erfüllen. Am vierten Tag hatte ich sehr Probleme weiterzukommen. Dies war auch der Grund, warum ich die ganze Zeit immer etwas zurückgefallen bin und dies nie aufholen konnte. In den letzten drei Tagen konnte ich noch einiges aufholen und wurde deshalb zum Glück fertig. Meine Planung war ziemlich realistisch. Es gibt nun noch einige Dinge, welche noch erweitert werden könnten. Für die Grundfunktionalität ist jedoch nichts mehr nötig. Die Applikation kann man so gut verwenden. Auch die Präsentation konnte ich in der Zeit gut vorbereiten. Es war gut, dass ich die Dokumentation nach jedem Issue nachgeführt habe. Somit konnte ich am Schluss auch noch einiges an Zeit und Zeitdruck sparen.

Bei diesem Projekt konnte ich meine C# Kenntnisse einiges erweitern. Bisher habe ich die grösste Zeit mit JavaScript gearbeitet. JavaScript ist eine ziemlich andere Programmiersprache wie C#, deshalb ist es nun umso besser, dass ich mich diese vier Wochen gut auf meine Lehre bei der PrimeSoft vorbereiten konnte. Auch habe ich nun eine Windows-Applikation, welche ziemlich nützlich, vor allem für Studenten, ist. Ich würde mich jederzeit wieder für ein Solches Projekt entscheiden. Es hatte von vielen Bereichen etwas zu profitieren. Ich musste mich mit Server und Clientseitiger Programmierung auseinandersetzen. Für die Verbindung vom Backend und Frontend arbeitete ich mit http-requests. Hier musste ich mich um die Sicherheit kümmern, damit nicht jeder eine Datenbankabfrage machen kann. Zum Schluss musste ich mich intensiv mit der Administration von Datenbanken und der Abfrage von Datenbanken befassen.

Mein Code ist teilweise sehr unübersichtlich. Vor allem in databaseRequest.php hat es einige sehr unschöne Stellen. Ich habe falsch begonnen, deshalb habe ich so weitergearbeitet, um nicht viel Zeit zu verlieren. Das Management mit den API-Endpunkten ist sehr unschön, dass dies alles über dieselbe Datei läuft. Es wäre besser gewesen, wenn ich für jeden der CRUD-Requests in einer anderen Datei verwaltet hätte. Auch die http-Abfragen sind nicht alle sehr schön. Vieles läuft über die URL-Parameter. Dies funktioniert zwar, es wäre jedoch schöner, wenn ich bei createDeck einen POST verwenden würde, und kein GET. Diese Möglichkeit mit dem POST ist mir jedoch erst zu spät gekommen, als ich createDeck bereits über GET gemacht habe. Deshalb liess ich die So wie es war. Hier gibt es also auch noch einiges an Verbesserungspotential.

3.7 Arbeitsjournal

3.7.1 Tag 1 - 04.06.25

Wir starteten heute mit der Individuellen Abschlussarbeit des BLJ's. Jörg führte uns gut und ausführlich in dieses Thema ein und erklärte uns die wichtigsten Details. Danach konnte ich direkt mit der Planung beginnen, weil ich mir voraus bereits überlegt habe, was ich machen möchte. Ich überlegte mir auch schon vor dem Beginn des Projektes, wie ich das ganze Projekt umsetzen kann. Somit konnte ich zielstrebig meine Milestones und Issues Definieren. Nach dem Mittag wurde mein Projekt freigegeben und ich konnte starten. Ich fokussierte mich heute auf die Dokumentation. Ich habe ein Dokument erstellt, mit den nötigen Untertiteln befüllt und alles beschrieben, was ich bis jetzt konnte. Dies sind die gesamte Einleitung und Planung. Weiter habe ich ein erster Entwurf von einem ERD für die Datenbank gezeichnet. Ich finde es ziemlich gut und denke, dass es meinen Anforderungen an die Datenbank entspricht. Der MySQL-Server, der als Host meiner Datenbank dient, befindet sich bei mir zu Hause. Um darauf zugreifen zu können, muss ich mich normalerweise im lokalen Netzwerk befinden. Dieses

Problem konnte ich mit einer VPN-Verbindung umgehen, um auch vom ZLI aus darauf zugreifen zu können. Obwohl die VPN-Verbindung einwandfrei funktionierte, konnte ich zunächst nicht auf den MySQL-Server zugreifen. Das Problem lag an den Firewall-Regeln des MySQL-Servers, die das Gateway von OpenVPN (dem VPN-Server-Dienst) nicht zuließen. Ich musste daher eine neue Regel hinzufügen, um den Zugriff auf die Datenbank auch über den VPN-Server zu ermöglichen.

3.7.2 Tag 2 - 05.06.25

Ich startete heute mit dem Aufsetzen der Datenbank. Das ERD, welches ich gestern gezeichnet habe, habe ich heute noch vervollständigt und dann begonnen einen passenden SQL-Code dazu zu schreiben. Den SQL-Code zu schreiben war nicht sehr anspruchsvoll, da ich alle Details der Relationen, Entitäten, Attribute und Flags bereits gestern im ERD umgesetzt und mir überlegt habe. Die einzige Schwierigkeit war die Syntax. Ich konnte den SQL-Code problemlos auf den MySQL-Server laden und meine Datenbank so aufsetzen. Mit AI habe ich dann Testdatensätze erstellt, um nun alles gut testen zu können. Weiter habe ich einige nützliche CRUD-Operationen mit der Datenbank getestet, damit ich sicherstellen konnte, dass meine Überlegungen funktionieren. Alles hat so funktioniert, wie ich es erwartet und geplant habe. Ich habe nun auch bereits einige nützliche SQL-Queries, welche ich danach für die Applikation richtig verwenden kann. All diese Schritte habe ich auch sauber dokumentiert. Ich bin nun mit meiner Dokumentation auf gleichem stand, wie ich auch beim Projekt weit bin. Dies möchte ich übers ganze Projekt hindurch so beibehalten. Ich bin sehr erleichtert, dass die Datenbank im Backend so unkompliziert aufgesetzt werden konnte und ich mit dieser nun gut weiterarbeiten kann. Es hat mich auch erleichtert, dass die VPN-Verbindung nun auch einwandfrei funktioniert und ich aus dem ZLI aus mit der richtigen Datenbank arbeiten kann.

3.7.3 Tag 3 - 06.06.25

Ich habe zuerst die falsche Projektvorlage gewählt. Ich habe 'WPF-App (.Net Framework) Visual Basic' gewählt. Ich möchte mein Projekt jedoch nicht mit Visual Basic, sondern mit C# umsetzen. Ich habe den Fehler jedoch schnell bemerkt, weil es .vb und .cs Dateien waren. Ich muss mich nun noch etwas an diese Entwicklungsumgebung gewöhnen, weil ich bisher immer mit VS-Code gearbeitet habe. Dies ist jedoch gut so, weil ich später im Betrieb dann auch mit Visual Studio 2022 arbeiten werde.

Das Hauptziel, welches ich mir heute gestellt habe, konnte ich leider nicht erreichen. Ich wollte eine Verbindung zur Datenbank aus dem C#-Projekt herstellen. Das Problem ist, dass ich meine Datenbank-Abfragen auf dem Client machen wollte. Hierfür gäbe es eine NuGet-Library, welche die Verbindung erleichtern würde. Da ich jedoch nur vom Lokalen Netzwerk auf die Datenbank zugreifen kann, ist dies keine Lösung, denn nicht jeder, der diese Applikation verwendet hat auch eine VPN-Verbindung in unser Heimnetzwerk. Ich habe heute sehr lange an diesem Problem getüftelt, jedoch ohne Erfolg. Als Test-Verbindung wollte ich einen Docker Container aufsetzen und auf die Lokale Datenbank zugreifen. Auch dies ist jedoch gescheitert. Somit habe ich heute nichts Sinnvolles umsetzen können, was sehr frustrierend war. Mir kam dann jedoch die rettende Lösung. Es ist zwar etwas aufwändiger, jedoch ist es sauberer umgesetzt und funktioniert sicher. Ich werde nun ein Serverseitiges Script mit PHP schreiben, welches die Datenabfragen durchführt. Dieses File liegt auf dem Webserver, auf welchen man von extern zugreifen kann. Mit http-Requests werde ich dann die Daten von meiner WPF-Applikation an die Datenbank senden, und wieder zurück. Heute war kein guter Tag, und ich habe sehr viel Zeit verloren. Ich werde deshalb noch zu Hause weiter entwickeln, damit ich nicht zu sehr hinter den Zeitplan falle. Dass ich ihn nun einhalten kann, ist momentan jedoch sehr unwahrscheinlich.

3.7.4 Tag 4 - 11.06.25

Ich dokumentierte heute alles für die Client-Server-Kommunikation. Dies kostete mich einen halben Tag, jedoch ist meine Dokumentation nun auf einem ziemlich aktuellen stand. Das ganze Sicherheitskonzept

und die Umsetzung des PHP-Scripts habe ich zu Hause begonnen und hier fertiggestellt. Es war herausfordernd, ein gutes Sicherheitssystem zu entwickeln, damit nicht jeder die Datenbank abfragen kann. Ich bin nun jedoch sehr zufrieden mit meiner Lösung, welche ich gut ausgeklügelt finde. Genauer zum Lösungsweg ist oben beschrieben. Ich arbeitete danach an der Indexseite. Die Datenbankabfrage lief nun einwandfrei, was mir schonmal zeigt, dass mein Plan funktioniert, hatte mit den Tokens und dem PHP-Script. Mein grosses Problem heute war das anzeigen der Decks im XAML-Window. Ich hatte die Ausgabe der Datenbank in einem JSON-String im Code-behind (MainWindow.xaml.cs) nun musste ich jedoch die Daten aus dem JSON im UI-Darstellen. Dies war nicht ganz einfach und ich habe wieder einiges an Zeit zahlen müssen. Ich bin jedoch gut voran gekommen heute und konnte das Problem gut lösen. Noch bin ich knapp im Zeitplan, ich habe dann jedoch nicht mehr viel Zeit für die nächsten Issues für morgen. Ich werde einfach weiterarbeiten und so gut wie möglich versuchen den Zeitplan wieder einzuholen.

3.7.5 Tag 5 - 12.06.25

Ich habe heute viel für die logischen Abfragen für die Decks und Cards gemacht. Weiter habe ich auch die Projektbenennung etwas umgebaut. Nun hat das MainWindow.xaml keine Bedeutung mehr. Anstelle habe ich Index.xaml als erstes Window erstellt, das beim Applikaionsstart gestartet wird. Es gab heute einige Herausforderungen, welche ich meistern musste. Das aufwändigste war, das JSON-Array richtig in die ListDecks Klasse zu schreiben und das Ganze dann im UI-Anzeigen zu können. Auch grosse Herausforderungen waren die richtigen SQL-Queries, um dir richtigen Daten abzufragen oder zu löschen. Ich denke, dass ich nun einiges neues lernen konnte und die Basis und das know how habe, um die weiteren Seiten zu entwickeln. Ich habe beim Serverseitigen Script die delete-Funktion hinzugefügt, um Decks zu löschen. Ich hatte sehr lange einen Fehler mit Statuscode 500, ich wusste jedoch nicht genau wo im PHP-Code. Nach langem Suchen sah ich meinen blöden Fehler, dass ich einen Parameter beim Funktionsaufruf vergessen habe. Es war ein gemischtes Gefühl, als ich den Fehler erkannt habe. Zum einen war ich frustriert, dass ich so lange an etwas Kleinem, einfachem gesucht habe, zum andern war ich erleichtert, dass ich nicht viel anpassen musste, um dies zu fixen. Ich bin immer noch etwas hinter dem Zeitplan, denke jedoch, dass wenn ich so weiterarbeite und keine Zwischenfälle dazu kommen, werde ich nicht noch weiter nach hinten fallen

3.7.6 Tag 6 - 13.06.25

Der heutige Tag war von Höhen und Tiefen geprägt. Ich bin zwar noch ein Stück weiter nach hinten gefallen, jedoch bin ich mit dem aktuellen Produkt bereits sehr zufrieden. Die neue Funktion createDeck funktioniert nun einwandfrei. Ich hatte hier einige Schwierigkeiten, welche ich lösen musste, wie mit dem HashTag, welcher die Parameter beim http-Request gestört hatte. Es dauerte sehr lange, um diesen Fehler zu erkennen, nun bin ich jedoch umso glücklicher, dass dies nun gut funktioniert und ich nun fürs nächste Mal weiss, worauf ich bei http-requests achten muss. Ich bin auch froh, dass der ColorPicker problemlos implementiert werden konnte, und dass dieser wunderbar funktioniert. Am Ende vom heutigen Tag habe ich viel erreicht und einige Probleme behoben. Backendseitig im PHP ist es nun kein grosser Aufwand, um weitere Datenbankabfragen-Funktionen zu erstellen. Ich kann die bestehenden verwenden und etwas anpassen. Von der Dokumentation bin ich auch auf einem sehr aktuellen Stand. Ich denke, dass ich meine Zeit gut einteile, dass ich immer wieder dokumentieren kann. Nun bin ich endgültig hinter dem Zeitplan. Ich bin bei einem Issue erst in der Hälfte, welchen ich heute Mittag eigentlich abschliessen sollte. Ich arbeite einfach normal weiter und werde gegen Schluss ein paar Issues aus zeitlichen Gründen streichen müssen. Ich werde auf jeden Fall darauf achten, dass die wichtigen Funktionen damit das Programm seinen Zweck erfüllt, abzuschliessen.

3.7.7 Tag 7 - 18.06.25

Ich stellte heute die AddCards Funktion fertig. Ich habe meinen Issue etwas vergrößert, sodass es nun nicht nur eine create-Funktionalität, sondern gleich auch eine edit-Funktionalität beinhaltet. Die vorhandenen Karten werden auch angezeigt, sodass man diese Inhalte bearbeiten könnte. Man kann neue cards oder quiz erstellen und diese speichern. Mit einem http-post wird ein JSON mit den notwendigen Inhalten an den Server geschickt. Die Funktionalität fürs Markieren von Favoriten oder fürs Setzen eines Lernstatus an Karten ist noch nicht implementiert. Ich werde dies vorerst überspringen und bei genügend Zeit am Schluss bearbeiten. Grundsätzlich bin ich momentan etwas hinter dem Zeitplan. Ich denke jedoch, dass einige der folgenden Issues eher kürzer werden, da ich diese bereits vorher etwas umgesetzt habe, wie zum Beispiel die Login-Funktion oder das Importieren/Exportieren der Karten/Decks. Im Laufe der Arbeit bin ich bisher nicht weiter zurück gefallen im Zeitplan, was ich sehr gut und wichtig finde. Auch gut ist, dass meine Dokumentation nun sehr aktuell ist, was heisst, dass ich am Schluss auch hier etwas Zeit sparen kann.

3.7.8 Tag 8 - 19.06.25

Mit dem Morgen war ich heute sehr zufrieden. Ich arbeitete sehr produktiv und erreichte mein Ziel etwas vor dem Zeitplan. Das Design fürs Lernen sieht sehr ansprechend aus und hat alle nötigen Funktionen. Herausfordernd war, die Karten je nach Typ (card/quizCard) richtig darzustellen. Dies ist oben genauer beschrieben. Bei der Auswertung finde ich es wichtig, dass man sieht, welche Fragen man richtig hatte, und welche nicht. Dies war auch nicht ganz einfach, jedoch machbar. Beim neuen Seitenaufruf muss der Parameter der Resultate mitgegeben werden und dann waren noch weitere Funktionen nötig, um dies ansprechend darzustellen. Ich finde auch die Berechnung der richtigen bzw. falschen Antworten ein sehr nützliches und interessantes Feature. Heute Nachmittag habe ich noch viel Dokumentiert von dem, was ich heute Morgen umgesetzt habe. Weiter habe ich mein PHP-Script optimiert, sodass nun das Logfile immer zuverlässig befüllt wird. Der Fehler lag wirklich daran, dass ich zu wenig Berechtigungen hatte, um das File zu beschreiben. Dies änderte ich heute und ich erweiterte das Script mit noch einigen Logdaten mehr. Ich habe auch ein neues Fenster erstellt, welches die Vorlage für den Import/Export darstellt. Morgen muss ich hierfür noch die Logik implementieren.

3.7.9 Tag 9 - 20.06.25

Ich konnte heute Morgen sehr effizient am neuen Feature arbeiten. Ich habe die kompletten Fileimport und Fileexport Funktionen geschrieben. Nun ist der import als json und csv und der export als json und csv möglich. Das ganze nur mit json wäre sehr einfach gewesen, da ich die Daten sowieso als json dem Server sende und viel mit json arbeite. Die grosse Herausforderung war es, das ganze noch für csv zu ermöglichen. Es war viel aufwendiger dies so zu implementieren. Ich habe am Morgen die Dokumentation auch auf einen aktuellen Stand bringen können. Ich konzentrierte mich am Nachmittag auf den Login und Registrierungsfunktion. Es ist mir erstaunlich gut gelungen und ich bin sehr zufrieden. Ich hätte nicht gedacht, dass sich dies so schnell umsetzen lässt. Nun kommt man beim Start der Applikation auf die Registrierungsseite, bei der man einen neuen User erstellen kann. Hier war es etwas knifflig, wie ich es umsetzen wollte, da ich zu Sicherheit immer das Passwort mit dem Token und dem Salt hashe. Nun habe ich jedoch noch kein passwort für diesen User, weshalb ich mir etwas Neues ausdenken musste. Hier eine genauere Erläuterung. Auf der Register-Seite kann man über einen Link welche einem auf die Login Seite führt. Hier läuft die Authentifizierung normal ab, wie ich es mir von Beginn an ausgedacht habe. Das Passwort und der username wird dann unter Properties.Settings.Default.username gespeichert, sodass dies individuell auf jeder Seite gespeichert wird. Ich bin heute sehr gut vorangekommen und habe meinen Zeitplan fast aufgeholt. Ich bin sehr zufrieden mit meiner Arbeit und meinem Fortschritt von heute. Ich habe heute jedoch sehr wenig dokumentiert, was ich am Mittwoch dringend nachholen muss.

3.7.10 Tag 10 - 25.06.25

Heute lief es erneut ziemlich gut. Am Morgen habe ich mich auf die Dokumentation konzentriert. Ich musste meine Arbeit vom Freitag noch das meiste dokumentieren. Nun ist die Login- und Registrierungs-Funktion komplett dokumentiert. Danach erweiterte ich den Login/Registrier-Funktion mit wenigen Details, wie dem Startup Screen, oder dass man direkt zum Login weitergeleitet wird, wenn man sich registriert hat. Wenn man sich registriert, kommt zuerst eine Bestätigung, dass man erfolgreich ein neues Konto erstellt hat. Danach wird man auf die Login-Seite weitergeleitet. Wenn man sich anmeldet, werden die Daten in der Applikation gespeichert. Diese Login-Informationen werden entweder gelöscht, wenn man sich abmeldet, oder nach einem Monat. Dies dient als Sicherheitsmassnahme, dass man sich mindestens jeden Monat neu anmelden muss. Wenn die Daten (noch) gespeichert sind, wird beim Ausführen der exe direkt die Index-Seite angezeigt. Wenn die Daten jedoch nicht gespeichert sind, wird der Login-Screen angezeigt. Weiter fügte ich das Favourite-Markieren Feature hinzu. Nun kann man einzelne Karten als Favorit markieren und dieser Wert wird als Tiny-Int gespeichert. Ich denke, dass ich den Follow-Issue nicht machen werde, da ich diese Funktion nicht so wichtig finde. Was ich toll fände, wenn ich es in der Zeit noch schaffen würde, ist die Collaborator-Funktion, beider man jemanden zu einem Deck einladen kann. Die Zeit ist nun jedoch sehr knapp und ich weiss nicht, ob ich mit diesem Feature fertig werde.

3.7.11 Tag 11 - 26.06.25

Ich konnte heute nochmals einen Tag im Homeoffice arbeiten. Dies lief sehr effizient und reibungslos ab. Zuerst fokussierte ich mich auf die Dokumentation, dass ich sie aktuell halte. Ich musste von Gestern noch einiges nach dokumentieren, wie das Favoritenmarkieren. Danach erweiterte ich die App mit einem neuen Feature, um den Account zu bearbeiten. Zuerst wollte ich nur die Accountlöschen-Funktion einbauen. Ich fand dann jedoch auch noch praktisch, wenn man die E-Mail, das Passwort oder den Username auch anpassen könnte. Deshalb erweiterte ich ebenfalls mit diesem Feature und dokumentierte auch hier alles ganz genau. Durch dass ich noch weitere Bearbeitungs-Funktionen eingebaut habe, fiel ist etwas im Zeitplan nach hinten. Ich wollte auch die Funktion für das Hinzufügen von Collaboratoren und Followern einbauen. Dies startete ich erfolgreich, jedoch wurde ich in der Zeit nicht ganz fertig. Morgen ist bereits der letzte Tag. Ich gebe mir am Morgen noch etwas Zeit für diese Funktionen und das Dokumentieren. Am Nachmittag möchte ich dann die Dokumentation noch fertig stellen und eine PPP erstellen. Morgen wird ein sehr gefüllter Tag, an dem ich keine Sekunde verschwenden darf.

3.7.12 Tag 12 - 27.06.25

Heute war ein sehr stressiger Tag. Am Morgen finalisierte ich die Follow und Collaborator-Funktion und dokumentierte das Ganze. Ich musste mich sehr beeilen und einige wichtige Funktionen wie Unfollow konnte ich leider noch nicht implementieren. Das Grundgerüst steht jedoch und es ist eine brauchbare App. Zu meinem Erstaunen konnte ich alle Issues erfolgreich erledigen und sogar ein optionales Issue habe ich erreicht (Collaborators). Am Nachmittag konzentrierte ich mich auf das Fertigstellen der Dokumentation und dem Erstellen einer Präsentation. Auch dies lief eher stressig ab, ich wurde jedoch fertig. Die Dokumentation hat eine angemessene Länge und ich bin der Meinung, dass alles beschrieben wurde. Auch die Präsentation ist nun fertig. Ich befasste mich heute auch noch mit dem Code, dass jede Datei ein Header hat und alles gut formatiert ist.

4 Anhang

4.1 Quellenverzeichnis

4.1.1 Online-Ressourcen

- **W3Schools** – Allgemeine Referenz für Programmierung
URL: <https://www.w3schools.com>
- **Stack Overflow** – Plattform für Lösungen zu Programmierproblemen.
URL: <https://stackoverflow.com>
- **Microsoft Dokumentation** – Offizielle Dokumentation für .NET, C# und WPF.
URL: <https://docs.microsoft.com>
- **GitHub** – Quellcode-Repository und Projektmanagement.
URL: <https://github.com>
- **PHP Dokumentation** – Offizielle Dokumentation für PHP.
URL: <https://www.php.net/docs.php>
- **MySQL Dokumentation** – Offizielle Dokumentation für MySQL.
URL: <https://dev.mysql.com/doc/>

4.1.2 KI-Tools

- **ChatGPT** – Unterstützung bei Code-Optimierung und Problemlösungen.
URL: <https://chat.openai.com>
- **Gemini** – KI-Tool für Code-Generierung und Dokumentation.
URL: <https://gemini.google.com>
- **Deepseek** – KI-basierte Unterstützung für Recherche und Code-Analyse.
URL: <https://www.deepseek.com>

4.1.3 Bibliotheken und Frameworks

- **.NET Framework** – Offizielles Framework für WPF-Entwicklung.
URL: <https://dotnet.microsoft.com>
- **Newtonsoft.Json** – Bibliothek für JSON-Verarbeitung in C#.
URL: <https://www.newtonsoft.com/json>
- **WPF-Color-Picker** – Bibliothek für die Farbauswahl in WPF.
URL: <https://github.com/dsafa/wpf-color-picker>

4.2 Glossar

Begriff	Definition
WPF (Windows Presentation Foundation)	Grafik-Framework für Windows-UI-Entwicklung mit Trennung von Design (XAML) und Logik (C#).
.NET Framework	Microsoft-Softwareplattform für Anwendungsentwicklung mit Laufzeitumgebung und Bibliotheken.
MySQL	Relationales Datenbankmanagementsystem (RDBMS) mit SQL-Schnittstelle.
ERD (Entity-Relationship-Diagramm)	Visuelle Darstellung von Datenbankentitäten und ihren Beziehungen.
CRUD-Operationen	Grundlegende Datenbankoperationen: Create, Read, Update, Delete.
SQL (Structured Query Language)	Sprache zur Verwaltung relationaler Datenbanken.
JSON (JavaScript Object Notation)	Datenformat für strukturierte Datenspeicherung und -übertragung.
CSV (Comma-Separated Values)	Tabellen-Dateiformat mit kommasetrennten Werten.
API (Application Programming Interface)	Schnittstelle für Softwarekommunikation (z.B. zwischen App und Datenbank).
PHP	Serverseitige Skriptsprache für Webentwicklung, dient als Middleware.
VPN (Virtual Private Network)	Sichere Netzwerkverbindung über unsichere Infrastrukturen (z.B. Internet).
XAML (Extensible Application Markup Language)	XML-basierte Sprache für WPF-Benutzeroberflächen.
LINQ (Language Integrated Query)	C#-Abfragesprache für Datenquellen (Datenbanken, Collections).
SHA-256	Kryptografischer Hash-Algorithmus für sichere Passwortspeicherung.
NuGet	Paketmanager für .NET-Bibliotheken und Abhängigkeiten.

4.3 Checkliste Individuelles Abschlussprojekt Dokumentation 2025

Wichtige Hinweise	Dokumentation ist für Fachpersonen verständlich	✓
	Eigenleistung und Unterstützungen sind klar deklariert	✓
Allgemein	Kopfzeile Projektname Titel	✓
	Fusszeile Datum, Version, AutorIn, Seitenzahl	✓
	Seitenlayout: Keine Überlappung Seitenränder, Quer/Hochformat	✓
	Beschriftung der Bilder/Grafiken	✓
	Einsatz von aussagekräftiger Grafiken (Netzwerkplan, DB Schema)	✓
Titelblatt	Klar und übersichtlich gestaltet	✓
	Überbegriff: Individuelle Abschlussprojekt BLJ	✓
	Projektname (aussagekräftig / max 20 Zeichen)	✓
	Name, Abgabedatum, Name der Lehrfirma	✓
	Version des Dokuments	✓
Inhaltsverzeichnis	Kapitel nummeriert (z.B.: 2.1; 2.1.1 usw.)	✓
	Seitenzahlen korrekt angegeben	✓
	Formatierung überprüft	✓
Einleitung	Änderungstabelle/Versionierung (tabellenform)	✓
	Aufgabenstellung und Projektbeschreibung	✓
	Mögliche Risiken vor Projektbeginn	✓
Planung	Terminplan (Gantt Diagramm oder Screenshot Github Project) vorhanden	✓
	Entscheidungswege und Möglichkeiten (Entscheidungsmatrix)	✓
Hauptteil	Detaillierte Beschreibung des Vorgehens und der Zwischenschritte	✓
	Ergebnisse der Arbeit	✓
	Entscheidungen sind formuliert	✓
	Arbeitsjournal vorhanden	✓
	Testplan/Testfälle mit Ergebnissen	✓
	Persönliches Fazit	✓
Anhang	Quellenangaben und Literaturverzeichnis vorhanden	✓
	Glossar/Begriffserklärungen vorhanden (<i>Github / Dokument</i>)	✓
	Bildverzeichnis vorhanden	✓
	Programm-Code, Scripts, Foto-Dokumentation (<i>Github / Dokument</i>)	✓
	Relevante KI-Chat-Prompts/Auszüge	X
	Testplan/Testfälle (optional)	X
	Ausgefüllte Checkliste	✓
Code/Konfigurationen	Link zum Github Repository vorhanden	✓
	Programm-Code folgt Clean-Code Richtlinien	✓
	Wichtige Module, Klassen, Funktionen sind kommentiert	✓
	Aussagekräftige Namen für Dateien, Klassen, Funktionen, DB Felder	✓
	Programm-Dateien Header mit Autor, Datum, Version, Beschreibung	✓

4.4 Bilderverzeichnis

Abbildung 2-1: Roadmap	6
Abbildung 3-1: ERD der FlashCard Datenbank.....	9
Abbildung 3-2: Die passende Projekt Vorlage auswählen	15
Abbildung 3-3: Neues Projekt wird erstellt	15
Abbildung 3-4: Titel und Speicherort für Projektmappe.....	15
Abbildung 3-5: Datenbankabfrage Netzwerkschema	16
Abbildung 3-6: MainWindow.xaml.cs	17
Abbildung 3-7: getToken.cs.....	18
Abbildung 3-8: generateHash.cs	19
Abbildung 3-9: sendRequest.cs.....	19
Abbildung 3-10: CodeSnippet databaseRequest.php allgemeine Variabeln.....	20
Abbildung 3-11: CodeSnippet databaseRequest.php Sessionverwaltung.....	20
Abbildung 3-12: CodeSnippet databaseRequest.php action == 'getToken'	21
Abbildung 3-13: CodeSnippet databaseRequest.php 'action==getData'	21
Abbildung 3-14: CodeSnippet databaseRequest.php getCards	22
Abbildung 3-15: Index.xaml.cs getDecks()-Funktion & DeckList-Class	23
Abbildung 3-16: Index.xaml Final-UI.....	23
Abbildung 3-17: Index.xaml Ausschnitt von Decksanzeige	24
Abbildung 3-18: Index.xaml.cs Routing nach Deck.xaml	24
Abbildung 3-19: Index.xaml.cs Suchfunktion-Logik.....	24
Abbildung 3-20: Index.xaml für die Suchfunktion	24
Abbildung 3-21: Index.xaml Ausschnitt für Löschfunktion	25
Abbildung 3-22: Index.xaml.cs Löschfunktion Event	25
Abbildung 3-23: Index.xaml.cs Konstruktoren	26
Abbildung 3-24: SQL-Abfrage für cards und quiz	26
Abbildung 3-25: Deck.xaml.cs routing auf Indexpage	26
Abbildung 3-26: Deck.xaml UI mit dargestellten Karten.....	26
Abbildung 3-27: databaseRequest.php request===deleteDeck.....	26
Abbildung 3-28: Deck erstellen UI	27
Abbildung 3-29: Colorpicker UI	27
Abbildung 3-30: Code für colorpicker.....	27
Abbildung 3-31: Xaml für Farbvorschau	27
Abbildung 3-32: ColorToBrushConverter Klasse.....	27
Abbildung 3-33: CreateDeck.xaml.cs createDeck()-Methode	28
Abbildung 3-34: removeHashtagColor.cs Regex-Funktion.....	28
Abbildung 3-35: Parametervariablen definieren	28
Abbildung 3-36: action = addDeck	28
Abbildung 3-37: databaseRequest.php addDeck-Funktion.....	29
Abbildung 3-38: addCards UI	29
Abbildung 3-39: AddCards.xaml.cs Klassen für die Darstellung der Daten.....	29
Abbildung 3-40: AddCards.xaml.cs Buttonfunktionen.....	30
Abbildung 3-41: AddCards.xaml.cs getCards -> bereits bestehende Karten abrufen	30
Abbildung 3-42: AddCards.xaml DataTeplate	31
Abbildung 3-43: AddCards.xaml Anzeige	31
Abbildung 3-44: Methode: SendCardsToApiAsync()	31
Abbildung 3-45: AddCards.xaml.cs erstllt JSON.....	31
Abbildung 3-46: postData.cs.....	31
Abbildung 3-47: databaseRequest.php Funktion addCards.....	32
Abbildung 3-48: JSON aus dem post herauslesen	33

Abbildung 3-49: LearnDeck UI von card & quiz.....	33
Abbildung 3-50: Auswertung UI	34
Abbildung 3-51: Abgefragte Decks in Liste speichern.....	34
Abbildung 3-52: LearnDeck.xaml.cs Laden der Daten	34
Abbildung 3-53: learnDeck.xaml.cs allgemeine Definitionen.....	34
Abbildung 3-54: LearnDeck.xaml.cs Methode fürs anzeigen der aktuellen Karte	34
Abbildung 3-55: LearnDeck.xaml QuizOptionButtons	35
Abbildung 3-56: LearnDeck.xaml.cs CheckAnswer	35
Abbildung 3-57: LearnDeck.xaml.cs SubmitAnswerButton geklickt	35
Abbildung 3-58: LearnDeck.xaml.cs QuizOptionButton geklickt.	35
Abbildung 3-59: AuswertungLernen.xaml.....	36
Abbildung 3-60: AuswertungLernen.xaml.cs Statics Berechnung	36
Abbildung 3-61: FileImpExp UI.....	36
Abbildung 3-62: fileImpExp.xaml.cs imprtCards-Methode	36
Abbildung 3-63: ConvertCsvToJson()	37
Abbildung 3-64: FileImpExp.xaml.cs exportCards().....	38
Abbildung 3-65: FileImpExp.xaml.cs ConvertCardsToCsv & EscapeCsvFileds	38
Abbildung 3-66: Register & Login UI.....	39
Abbildung 3-67: Register.xaml.cs Buttonclick-Event	39
Abbildung 3-68: Register.xaml.cs registerUser-Methode.....	39
Abbildung 3-69: Login.xaml.cs LoginButton_Click-Event	40
Abbildung 3-70: Login.xaml.cs verifyAccount()-Methode	40
Abbildung 3-71: databaseRequest.php \$action==='createUser'	40
Abbildung 3-72: databaseRequest.php Funktion createUser().....	41
Abbildung 3-73: databaseRequest.php request=verifyAccount	42
Abbildung 3-74: App.xaml.cs OnStartup-Methode.....	42
Abbildung 3-75: Properties.Settings.default	42
Abbildung 3-76: Deck.xaml.cs FavoriteButton_Click-Event.....	43
Abbildung 3-77: sendRequest.cs SendRequest-Methode.....	43
Abbildung 3-78: UI von der Account Overview Seite.....	44
Abbildung 3-79: Account.xaml.cs Save-Button-Event.....	44
Abbildung 3-80: Account.xaml.cs updateUser-Methode.....	45
Abbildung 3-81: Account.xaml.cs Eigenschaften	45
Abbildung 3-82: createDeck.xaml DropDown-Button	46
Abbildung 3-83: createDeck.xaml.cs getter & setter.....	46
Abbildung 3-84: createDeck.xaml.cs getUsers()	46
Abbildung 3-85: databaseRequest.php getUsers-Funktion.....	47
Abbildung 3-86: databaseRequest.php collaborator erstellen.....	48
Abbildung 3-87: createDeck.xaml.cs addDeck mit Mitarbeiter	48
Abbildung 3-88: Account Overview UI	48
Abbildung 3-89: Account.xaml.cs getNotFollowing-Methode (allgemeines Bsp.)	49
Abbildung 3-90: Account.xaml.cs addFollow.....	49
Abbildung 3-91: databaseRequest.php administration von Follower DB-Abfragen	50
Abbildung 3-92: databaseRequest.php follower-Abfragen	50
Abbildung 3-93: databaseRequest.php addFollow-Funktion	51